

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Távközlési és Mesterséges Intelligencia Tanszék

Kosárladbda játékosok mozgásának elemzése mélytanulási módszerekkel

Szakdolgozat

Készítette
Czene Zsombor

Konzulens
Dr. Toka László
Szmida Patrik Péter
Mihályi Balázs Márk

2026. május 27.

Tartalomjegyzék

Kivonat i

Abstract ii

1. Bevezetés 1
1.1. Motiváció és előzmények . . . 1
1.2. Sportanalitika és gépi tanulás a kosárlabdában . . . 2
1.2.1. Felügyelt, feature-engineering alapú modellek . . . 2
1.2.2. Felügyelet nélküli reprezentációtanulás . . . 2
1.2.3. Önfelügyelt multi-task tanítás . . . 3
1.3. Az alkalmazott modellarchitektúra elméleti alapjai . . . 3
1.3.1. Transformer-architektúra és önfigyelmi mechanizmus . . . 3
1.3.2. BERT és a kétirányú maszkolt nyelvi modellezés . . . 4
1.3.3. Gráf-attention alapú hálók, GAT és GATv2 . . . 4
1.3.4. Tracking adatok szekvenciális tokenizációja . . . 5
1.4. A feladatkiírás értelmezése . . . 5
1.5. A dolgozat felépítése . . . 6

2. A tervezés részletes leírása 7
2.1. Adatforrások bemutatása . . . 7
2.2. Az adatletöltés lépései . . . 7
2.3. Szekvencia generálás és jellemzők kinyerése . . . 7
2.3.1. Konfiguráció és alapparaméterek . . . 8
2.3.2. A szekvencia-alapú feldolgozás lépései . . . 10
2.3.2.1. Labdabirtoklási határok kinyerése . . . 11
2.3.3. Játékosposzt-szótár felépítése . . . 11
2.3.4. Egységes időalapú indexelés bevezetése . . . 11
2.3.5. Feldolgozási ciklus a labdabirtoklás-határok alapján . . . 12
2.3.5.1. Játékidő ugrásainak kezelése . . . 12
2.3.5.2. Pályán lévő aktív játékosok ellenőrzése . . . 12
2.3.6. Tenzormátrix felépítése . . . 13
2.3.7. Mátrix-szintű utófeldolgozási lépések . . . 13
2.3.7.1. Savitzky–Golay simítás . . . 13
2.3.7.2. Támadási irány normalizálása . . . 14
2.3.7.3. Frame-szintű sebességértékek számítása . . . 14
2.3.7.4. Származtatott jellemzők kiszámítása . . . 14
2.3.8. A mátrixon végzett csúszóablakos felbontás . . . 16
2.3.9. Metaadatok összeállítása és kimeneti tenzor mátrix mentése . . . 16
2.4. Modell architektúra bemutatása . . . 17
2.4.1. Adatbetöltés . . . 17
2.4.1.1. Memmap-builder implementációja . . . 17

2.4.1.2. Dataset implementáció . . . 18
2.4.2. Adatok normalizációja és augmentáció . . . 18
2.4.2.1. Y-tengely menti tükrözés . . . 18
2.4.2.2. Időbeli eltolás . . . 19
2.4.2.3. Gauss-zaj hozzáadása . . . 19
2.4.2.4. Augmentáció szerepe a SimCLR-tanításban . . . 19
2.4.3. Modell architektúra . . . 19
2.4.3.1. Térbeli figyelem . . . 21
2.4.3.2. Időbeli, temporális tömörítés . . . 22
2.4.3.3. Szekvenciális kontextus, időbeli figyelem . . . 23
2.4.3.4. Phase Pooling . . . 24
2.4.3.5. Bottleneck réteg bevezetése . . . 25
2.4.4. Önfelügyelt multi-task tanítási feladatok . . . 25
2.4.4.1. Masked Player Prediction . . . 25
2.4.4.2. Masked Language Modelling . . . 26
2.4.4.3. Velocity Head . . . 26
2.4.4.4. Court Zone Prediction . . . 27
2.4.4.5. SimCLR kontrasztív tanítás . . . 28
2.4.5. Veszteségfüggvények implementációja . . . 29
2.4.5.1. MPP, MLM és Velocity loss . . . 29
2.4.5.2. CZP loss . . . 29
2.4.5.3. SimCLR NT-Xent loss . . . 30
2.4.6. Tanítási folyamat részletei . . . 30
2.4.6.1. Optimalizáló és tanulási ráta ütemező . . . 30
2.4.6.2. Mixed Precision alkalmazása . . . 30
2.4.6.3. Validáció és Early Stopping implementációja . . . 31
2.4.6.4. Silhouette Próba . . . 31
2.4.7. Embeddingek kinyerése . . . 31
2.5. Riportáló weblap felület bemutatása . . . 32
2.5.1. Tervezési döntések és célok . . . 32
2.5.2. Adatbetöltő réteg és kompatibilitási absztrakció . . . 32
2.5.3. Felhasználói felület bemutatása . . . 33
2.5.4. Mock-adat alapú fejlesztés és tesztelhetőség . . . 35
2.6. A webes környezet tesztelése . . . 35

3. Modell eredményeinek kiértékelése 37
3.1. A tanítási folyamat lefutása . . . 37
3.2. Az embedding-tér kiértékelési módszertana . . . 38
3.2.1. Kétszintű klaszterezési pipeline . . . 38
3.2.2. Vizualizáció UMAP-pal . . . 39
3.3. Alkalmazott kiértékelési metrikák és eredmények . . . 39
3.4. Eredmények . . . 39
3.4.1. K-Means klaszterezés eredménye . . . 39
3.4.2. Hierarchikus klaszterezés . . . 41
3.4.3. Klaszterezés stabilitása . . . 41
3.4.4. Vizuális elemzés az embedding-térben . . . 41

4. Összefoglalás és továbbfejlesztési lehetőségek 46
4.1. Továbbfejlesztési lehetőségek . . . 46

Irodalomjegyzék 48

Függelék **51**
F.1. Nyilatkozat generatív mesterséges intelligencia alkalmazásáról 51
F.2. A tanítás további veszteséggörbéi 54
F.3. Függelékben elhelyezett kódrészletek 55

HALLGATÓI NYILATKOZAT

Alulírott *Czene Zsombor*, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2026. május 26.

Czene Zsombor
hallgató

Kivonat

A kosárlabda-sportanalitikában a SportVU mozgáskövető (tracking) rendszer megjelenésével hatalmas adatmennyiség növekedés jelent meg. Ezek az adatok a korábbi eseményalapú (PBP) statisztikákhoz képest kevesebb szemantikai információt hordoznak, cserébe viszont részletesen leírják a játék térbeli és dinamikai jellemzőit. A tracking adatok kézi címkézése a nagy adatmennyiség miatt szinte lehetetlenné vált, ami megnehezíti a felügyelt tanulási módszerek alkalmazását. Szakdolgozatomban azt vizsgáltam, hogy egy BERT-alapú, önfelügyelt tanulási megközelítés mennyire alkalmas a kosárlabda játékhelyzeteinek és taktikáinak megértésére. Az NBA 2015–16-os szezonjának nyilvánosan elérhető SportVU tracking adatait és az NBA.com play-by-play eseményadatait használtam fel. Az adatokat labdabirtoklás-alapú szekvenciákra bontottam, csúszóablakos módszert alkalmazva. Az általam tervezett architektúra újítása, hogy a térbeli és az időbeli információ feldolgozását dedikált modellekkel szétválasztja. A játékosok közötti térbeli kapcsolatokat egy GATv2-alapú gráf-neurális háló, az időbeli összefüggéseket pedig egy BERT alapú architektúra tanulja meg. A modellt öt párhuzamos önfelügyelt pretext feladaton tanítottam a robusztus reprezentáció kialakítása érdekében. Az eredményeket a kimeneti embedding-tér klaszterezésével értelmeztem, melynek során az egyes csoportokban eltérő taktikai mintázatok és felállások különültek el. Ezt klaszterezési metrikákkal és a szekvenciák vizuális ellenőrzésével is alátámasztottam. Végül az eredmények megjelenítésére egy webes elemzőfelületet is készítettem. Eredményül elmondható, hogy a megközelítésem képes önfelügyelt módon, multi-task tanulással címkézés nélkül a kosárlabdára vonatkozó taktikai tudás tanulására, ugyanakkor az ipari felhasználáshoz további fejlesztést igényelne.

Abstract

With the introduction of the SportVU motion-tracking system, basketball sports analytics has seen an enormous growth in the volume of available data. Compared to earlier event-based (PBP) statistics, this data carries less semantic information, but in exchange it describes the spatial and dynamic characteristics of the game in detail. Due to the large volume of data, manual labelling of tracking data has become nearly impossible, which hinders the application of supervised learning methods. In my thesis, I investigated how suitable a BERT-based, self-supervised learning approach is for understanding the game situations and tactics of basketball. I used the publicly available SportVU tracking data of the NBA 2015–16 season together with the play-by-play event data published by NBA.com. I divided the data into possession-based sequences using a sliding-window method. The novelty of the architecture I designed is that it separates the processing of spatial and temporal information using dedicated models. The spatial relationships between players are learned by a GATv2-based graph neural network, while the temporal dependencies are learned by a BERT-based architecture. I trained the model on five parallel self-supervised pretext tasks in order to form a robust representation. I interpreted the results by clustering the output embedding space, in the course of which distinct tactical patterns and formations emerged in the individual groups. I also supported this with clustering metrics and the visual inspection of the sequences. Finally, I created a web-based analytics interface for presenting the results. In conclusion, my approach is able to learn tactical knowledge related to basketball in a self-supervised manner, through multi-task learning and without labelling, although it would require further development for industrial use.

1. fejezet

Bevezetés

1.1. Motiváció és előzmények

A sportanalitika az utóbbi években egyre fontosabb szerepet tölt be a sikeres élsport és a sikeres piacalapú sportgazdaság mindennapjaiban. A fejlődő analitikai és elemzőeszközök mára nélkülözhetetlenné váltak a sikeres csapatok életében. Az egyre bővülő adatforrások és a modern statisztikai mutatók megjelenése háttérbe szorította a manuális elemzéseket az automatikus elemzőrendszerek javára. Az NBA-ben ez a folyamat a play-by-play (PBP) eseményadatok publikálásával kezdődött, majd ez a forradalom a 2013–14-es szezonban a ligaszinten bevezetett SportVU videóalapú mozgáskövető rendszerrel bontakozott ki, ami hat kamera segítségével teszi lehetővé, hogy minden pályán lévő játékos és a labda háromdimenziós koordinátái másodpercenként 25-ször rögzítésre kerüljön.

Az adatmennyiség ilyen nagymértékű növekedése helyezte előtérbe a mesterséges intelligencia alapú megoldások bevezetését a manuális elemzések helyett. A SportVU által szolgáltatott adatok nem aggregált, vagy eseményenkénti statisztikák összességét, hanem a mérkőzések tényleges tér-időbeli dinamikáját nyújtják. Ez nagyságrendekkel több és jóval kevesebb szemantikai információval rendelkező adatot jelent, mint a korábbi PBP adatok. Emellett az adatok értelmezése és elemzése korántsem egyértelmű a klasszikus statisztika eszközeivel. A meglévő PBP adathalmazok főként a játékot meghatározó eseményekre fókuszálnak, mint a dobások, eladott labdák, lepattanók vagy személyi hibák. A játékosok és a labda közötti interakciókat és dinamikát nem kísérelték meg rögzíteni, ahogyan ezt a SportVU adatok teszik [1].

A mozgáskövetési (tracking) adatok manuálisan elvégzett címkézése viszont megnehezíti a felügyelt gépi tanulási módszerek implementálhatóságát. Ennek a folyamata rendkívül időigényes, valamint szakértő elemzők aprólékos munkáját igényli. Ezt kikerülve célszerű mérlegelni az önfelügyelt tanulási megoldások alkalmazhatóságának lehetőségét, ahol a címkék helyett a tracking adatok struktúrája alapján megy végbe a tanulási folyamat a reprezentáció kinyerése érdekében.

Ezt a gondolatmenetet folytatva érdemes megvizsgálni a kosárlabda tracking adatok jellegét is. Egyrészt megjelenik a térbeli információ, a csapatok felállásával, a taktikai játékok jellemző geometriájával, másrészt rendkívül fontos az adatok időbeli, temporális jellegét is szem előtt tartani. A különböző játékszituációkban a játékosok különböző mozgásdinamikával rendelkeznek. Egy gyorsindítás és egy felállt taktikai játék során teljesen más a pályán lévő játékosok mozgásának sebessége. Valamint önmagában az időbeliség kontextusa is rendkívül fontos a játékok kialakulásában. Ezt a kettősséget az elemzésre szánt modellarchitektúrának is tükröznie kell, ezért az általam implementált rendszer párhuzamosan egy gráf-neurális hálók térbeli információfeldolgozó képességére, valamint a Transformer-architektúra figyelmi mechanizmusára épít.

1.2. Sportanalitika és gépi tanulás a kosárlabdában

A kosárlabda sportanalitikában alkalmazott gépi tanulási modellek között több különböző megközelítés került kidolgozásra az elmúlt években. A klasszikus jellemzőkinyerés (feature engineering) alapú megközelítések és a felügyelt mélytanulást alkalmazó modellek mellett megjelentek a mai modern technikáknak számító önfelügyelt és reprezentációtanulási módszertanok is.

1.2.1. Felügyelt, feature-engineering alapú modellek

A legegyszerűbb iránynak a felügyelt tanuláson alapuló megoldások tekinthetőek. Ezek közös jellemzője, hogy az elemzők által kézzel készített, domain specifikus jellemzőkkel, címkékkel kell ellátni a tanító adathalmazt. [2] elzárás-leválás (pick-and-roll) szituációk védekezési reakcióinak osztályozását valósította meg. Ehhez egy logisztikus regressziót alkalmazó, a labdás, az elzárást adó játékos, valamint a védőik közötti távolságokra épülő modellt implementáltak. Fontos megemlíteni, hogy a szerzőknek rendelkezésre állt a védekezési típusok szerint kategorizált tanítóhalmaz a felügyelt tanulás végrehajtásához.

Shah és Romijnders 2016-os publikációjukban a hárompontos dobások kimenetelének predikciójára tanítottak rekurrens neurális hálót (Recurrent Neural Network, RNN). Ez a modell a labda szekvenciális trajektóriáira építve becsülte meg a dobások eredményét. Fontos megállapításuk, hogy a tisztán pozícióalapú szekvenciális modell reprezentáció szempontjából jobb eredményeket hozott, mint a kézzel készített jellemzőkre épülő, feature-engineering alapú elődjeik [3].

Wang és Zemel implementációjukban a játékosok elemzését támadó oldalról közelítették meg. Ehhez a SportVU adatokon egy RNN-alapú osztályozó modellt implementáltak [4]. Ez a megközelítés a támadó játékosok koordinátáinak szekvenciái alapján tanult, valamint a címkézés a csapatok által használt taktikai playbook-ból (taktikai gyűjtemény) kinyert kategóriák alapján került meghatározásra. A legfontosabb eredményük, hogy a címkézés kézzel elvégzett hosszadalmas folyamata nélkül, implementálták a felügyelt tanulást megvalósító osztályozásukat.

A felügyelt módszerek érezhetően korlátokat jelentenek. Egyrészt a címkézés folyamata rendkívül időigényes és elemzők szaktudását igényli. Másrészt a modell csakis a tanítóhalmazban szereplő címkék osztályait lesz képes felismerni. Minden új vagy kimaradt taktikai elem új címkézést igényel, különben a modell a létező osztályok közé fogja besorolni az adott taktikát.

1.2.2. Felügyelet nélküli reprezentációtanulás

Az imént említett korlátokra jelenthet megoldást a reprezentációtanulás technikája. Ennek a módszernek a kosárlabda vonatkozásában az a célja, hogy a bemeneti koordinátaszekvenciákból egy olyan vektoros beágyazási teret (embedding-teret) tanuljon meg, ahol a hasonló mozgásmintázatok vektorai geometriailag is közel kerüljenek egymáshoz.

A Nistala és Guttag által 2019-ben publikált modell [5] egy felügyelet nélküli megoldás, ami 32 dimenziós trajektória-alapú embedding-eket alkalmazott az egyes támadó játékosok labdabirtoklásonkénti útvonalainak és mozgásmintáinak megértésére. Az eredményül kapott embedding-térben előállított vektorok klaszterezése során láthatóvá váltak különböző releváns taktikai elemek szegmentációi.

Ding és Huang 2020-ban bemutatott munkájukban gráf-figyelmi (gráf-attention) alapú megközelítést alkalmaztak. A játékosok és a labda egy teljes gráf csomópontjaiban lettek elhelyezve és a csomópontok közötti kapcsolatokat és interakciókat pedig gráf-attention mechanizmussal modellezték. Valamint egy időbeli konvolúciós hálózat, azaz egy *Temporal Convolutional Network (TCN)* végezte a jövőbeli mozgástrajektóriák becslését, ez a mo-

dellelem felelt az architektúra időbeli kiterjesztéséért. A szerzők fontos megállapítása volt, hogy a gráf-attention és a TCN modelleket kombináló architektúra hosszabb időablakokra is jól reagált, valamint hatékonyabb tanulási folyamatot eredményezett, mint a korábbi megoldások [6].

Az itt bemutatott megközelítések fontos tanulsága, hogy a kialakuló embedding-tér szerkezetét a választott tanítási feladat (későbbi célfeladat, azaz downstream task, vagy előtanítási feladat, azaz pretext task) típusától és céljától függően másképp befolyásolja. Egy trajektória-rekonstrukciós, egy hasonlóságkereső vagy egy mozgáspredikciós feladat más-más reprezentációt eredményez. A következő bekezdés modelljeinek alapját ez a megállapítás ösztönzi, melyek az itt olvasható pretext feladatok egymást kiegészítő, párhuzamos implementációjával próbálkoznak robusztusabb reprezentáció kinyerése érdekében.

1.2.3. Önfelügyelt multi-task tanítás

A manapság alkalmazott reprezentáció alapú tanítás egyik továbbfejlesztésének tekinthető a többfeladatos (multi-task) alapú megvalósítás, ami a több különböző pretext feladat párhuzamos végrehajtását jelenti.

A Wang és társai 2024-ben publikált *HoopTransformer* modell egy axiális-figyelmű Transformer architektúra, két önfelügyelt pretext tanítási feladat együttes optimalizálásával. A Motion Prediction (MP) a bemeneti szekvenciák első fele alapján próbálja prediktálni a mozgások folytatását. A Motion Reconstruction (MR) pedig a játékosok nagy sebességű mozgásának maszkolásával végez rekonstrukciót [7]. A modellt hasonlóan a megoldásomhoz a nyilvánosan is elérhető 2015–16–os NBA szezon SportVU adatain implementálták. Fontos megállapítás, hogy a két feladat együttes tanítása robusztusabb és pontosabb eredményeket hozott, mint az általuk vizsgált korábbi implementációk.

Shao és társai 2025-ben publikált *Hoop-MSSL* modellje az előző önfelügyelt, multi-task alapú megoldást fejlesztette tovább, három pretext feladat beépítésével. A mozgás trajektóriák frame-szintű rekonstrukciója, a játékosok taktikákban betöltött szerepének azonosítása (például a labdás és elzáró ember), valamint szekvencia szintű kontrasztív tanulás jelentek meg új pretext feladatokként [8]. Ezek együttes alkalmazásával a modell még általánosabb reprezentációt adott a tanítási folyamat végére.

Az általam vizsgált források (*HoopTransformer* és *Hoop-MSSL*) [7, 8] jelentik a kosárlabda analitikában alkalmazott mélytanulás és reprezentációtanulás csúcsát. Emiatt közvetlen kiindulási alapot adnak a saját modellem implementációjához. Fontos tanulság a multi-task alapú pretext feladatok bevezetése és a kontrasztív tanulás alkalmazása. A különböző, már kipróbált és működő modell elemek kombinálására fókuszáltam, ugyanakkor ezek mellett számos újítást próbáltam bevezetni az architektúrába. Ezek közül az egyik legfontosabb a térbeliség és időbeliség tanulásáért felelős komponensek szétválasztása és együttes alkalmazása, valamint újszerű pretext feladatok implementálása volt.

1.3. Az alkalmazott modellarchitektúra elméleti alapjai

Ebben a fejezetben rövid áttekintésre kerülnek az általam implementált architektúra elemeinek működése.

1.3.1. Transformer-architektúra és önfigyelmi mechanizmus

A Transformer architektúra [9] célja az önfigyelmi mechanizmus bevezetésével alternatívát nyújtani a szekvenciális adatok feldolgozására a rekurrens (RNN, LSTM) és a konvolúciós hálókkal szemben. A Transformer a tokenizált szekvenciák alapján képez súlyozott összeget, a súlyokat pedig ezen tokenek tartalma alapján frissíti. A tokenek segítségével a

teljes szekvencia tartalmát képes kódolni, így a szűk környezet helyett hosszabb, a teljes szekvencián átívelő függőségek tanulására is képes. Ezzel lehetőség nyílik a szekvencián belüli teljes környezeti kontextus megértésére, ellenben a rekurrens modellcsaláddal, ahol a tágabb környezet információi kevesebb hangsúlyt kapnak.

A Multi-Head Attention változat ezt a folyamatot több „fejen" (head) végzi el, így a modell egyszerre képes különböző szemantikai vagy szerkezetű kapcsolatokat megfigyelni és megtanulni. A Transformer alapértelmezésben nem rendelkezik információval a bemenetek sorrendjéről, emiatt fontos a pozícióalapú kódolás beépítése a bemenetbe. Ehhez a [9]. forrásban szinusz–koszinusz trigonometrikus függvényekre épülő megoldást javasolnak a szerzők, így én is ezt a megközelítést választottam az implementáció során.

A modellem a Transformer encoder-only változatát alkalmazza a BERT-szerű implementáció miatt. Ennek értelmében nem történik meg a dekódolási lépés, hanem a szekvenciák párhuzamosan áthaladnak az encoder-rétegeken, majd a kétirányú (bidirekcionális) kontextus alapján történik a frissítés a tanulási lépések során.

1.3.2. BERT és a kétirányú maszkolt nyelvi modellezés

A BERT (Bidirectional Encoder Representations from Transformers) modellt Devlin és társai mutatták be 2019-ben [10]. Kezdetben nyelvi feladatokon és a természetes nyelvfeldolgozás (Natural Language Processing, NLP) területén alkalmazták.

Az alapmegközelítés szerint egy Transformer Encoder kétirányú önfelügyelt előtanítása pretext feladatokkal kiegészítve megfelelő reprezentációs tudást eredményez. A publikált architektúra fő pretext feladata a Masked Language Modelling (MLM), ami a bemeneti szekvencia szöveg tokenjeinek véletlenszerűen kiválasztott 15%-át egy speciális `[MASK]` tokenre cserélte, majd a célja a maszkolt rész rekonstrukciója a kontextus alapján. Az alkalmazott maszkolási stratégia során az esetek 80%-ában ténylegesen a `[MASK]` token kerül alkalmazásra, 10%-ban a maszkolt rész egy véletlenszerűen választott másik tokenre cserélődik, a maradék 10%-ban pedig érintetlenül marad. Erre azért van szükség, hogy a modell ne csak a `[MASK]` tokent lássa a tanulási folyamat során, ugyanis ez a token a kiértékelési lépések során nem kerül használatra, tehát kizárólag ennek a maszk tokennek az alkalmazása torzítaná a tanítást követő lépéseket.

A másik fontos építőelem a szekvenciák elején elhelyezett tanulható `[CLS]` token. Ez egy szekvencia aggregált reprezentációját valósítja meg, és az encoder utolsó rétegének kimenete alapján a teljes szekvencia-szintű reprezentációt foglalja magába. A `[CLS]` token kulcsfontosságú, ugyanis a downstream osztályozási fejek implementációja is erre támaszkodik.

A BERT modellem implementációja során ezt a maszkolási elvet próbáltam alkalmazni, amit a szerzők is megvalósítottak. Fontos különbség, hogy a kosárlabda tracking adatok alkalmazása során a tokenek nem szavak, hanem a tracking adatok frame-jei, időpillanatai, a mondatok analógjának pedig a labdabirtoklási szekvenciák tekinthetők. A maszkolási stratégiákat ennek értelmében a tér-időbeli koordinátákra, és a játékosokhoz rendelt különböző jellemzőkre alkalmaztam.

1.3.3. Gráf-attention alapú hálók, GAT és GATv2

A gráf-neurális hálózatok (Graph Neural Networks, GNN) célja a gráf-struktúrájú adatokon végzett gépi tanulás megvalósítása. A háló minden csomópont kontextusát a gráfstruktúra szerinti szomszédai alapján iteratívan frissíti. A tanulási folyamat során egy konkrét csomópont a többi csomópont egyre tágabb környezetének információját ismerheti meg.

A Graph Attention Network (GAT) modellt Veličković és társai 2018-ban mutatták be [11]. Ebben a GNN továbbfejlesztésének ötlete az volt, hogy a csomópontok közötti in-

formációáramlást súlyokkal szabályozzák, ezzel kontrollálva, hogy egy adott csúcs mennyit tanul a szomszédaitól, így bevezetve a figyelmet a csúcspontok között.

Brody és társai 2022-es munkájukban viszont rámutattak [12], hogy a GAT csak statikus figyelmet képes megtanulni. Ez azt jelenti, hogy ha az egyik csomópont számára egy adott szomszéd jelenti a legfontosabb kontextust, akkor a többi csomópont számára is ez az adott szomszéd csomópont lesz globálisan a legfontosabb a figyelmi rangsorban. A GATv2 változatban a nemlineáris LeakyReLU aktivációs függvény és a végső projekcióért felelős függvény sorrendjének felcserélését javasolták, amivel a modell tényleges dinamikus figyelmet valósít meg, és a csomópontokra eltérő fontossági rangsort képes tanulni.

A modellem térbeli figyelmi blokkja ezt a GATv2-alapú modellt alkalmazza, melynek bemenete a pályán lévő játékosokból és a labda alapján felépített 11-csúcsú teljes gráf.

1.3.4. Tracking adatok szekvenciális tokenizációja

A BERT adaptációja kosárlabdában

Fontos kitérni arra, hogyan adaptáltam a BERT NLP token-alapú megközelítését a SportVU tracking adatokra. Az NLP-ben a mondatot diszkrét tokenek sorozatára bontjuk fel. A tanulási feladat ezen tokenek közötti összefüggések megértése a rendelkezésre álló kontextus alapján. A tracking adatok esetében egy hasonló token és mondat megfeleltetést kell tenni, annak érdekében, hogy az említett BERT architektúrával képesek legyünk a tanítást megvalósítani. Egy mondatnak egy labdabirtoklás feleltethető meg. A szavak (mint tokenek) analógjai a csúszóablakos módszerrel kivágott, majd konvolúcióval tömörített kisebb szekvenciaelemek. Ez a megfeleltetés egyben indokolja is a labdabirtoklás-határok mentén történő vágást, ugyanis a maszkolt rekonstrukció pretext feladatai esetén fontos, hogy a rekonstruált szekvenciák önmagukban értelmes egészet alkossanak. Ez megfeleltethető a feldolgozott szövegben megjelenő mondathatárokkal.

A cél tehát, hogy a megfelelően előkészített SportVU adatokból a modellem önfelügyelt módon képes legyen a labdabirtoklás-határok mentén előállított szekvenciákból támadástípusokat és taktikákat megérteni. Alapvető megkötés, hogy a támadó csapat a modell minden fázisában aszimmetrikusan felülsúlyozásra került, hogy a védelem ne domináns, csak a támadásokat befolyásoló fontos tényezőként jelenjen meg.

A *HoopTransformer* [7] axiális figyelem segítségével próbálja tanulni a térbeli és időbeli kontextust a Transformer-architektúrán belül, a *Hoop-MSSL* [8] pedig a maszkolásra és a szerepek azonosítására építi a térbeli információk kinyerését. A saját megközelítésem újítása, hogy a modellemben a térbeli és időbeli dinamikai jellemzők tanulása architekturális szinten is különválik. A térbeliséget frame-szinten egy *GATv2* [12] alapú gráf-neurális háló kódolja a játékosok között, és egy Transformer Encoder [9, 10] felel az időbeli temporális kontextus tanulásáért.

A pretext feladatok definiálásánál bővítettem a tanulási feladatok körét. A teljes architektúra párhuzamosan öt egymástól független feladatot tanul, a Masked Player Prediction-t, a Masked Language Modelling-et (ami az eredeti BERT modellben alkalmazott NLP módszer analógja a tér-időbeli SportVU adatokon), a Velocity Prediction-t, a Court Zone Prediction-t, valamint a SimCLR [13] kontrasztív tanulást. Ezekkel a multi-task önfelügyelt tanulás [14] elvét még erősebben próbáltam érvényesíteni, mint az eddig bemutatott megvalósítások.

1.4. A feladatkiírás értelmezése

A szakdolgozat kiírásában is említett cél a BERT modell alkalmazhatóságának alátámasztása a kosárlabda játékkontextusának megértésére. A modell tanítását az NBA.2015–2016–

os szezonjának SportVU tracking adatai, valamint az NBA.com által publikusan közölt eseményről eseményre rögzített, play-by-play (PBP) statisztikai adatok segítségével végeztem el. Követelményként megjelenik, hogy az implementált architektúra egy mérnökileg fejlesztett rendszert alkosson, és az eredmények megjelenítését egy webes alkalmazáson keresztül tegyem lehetővé.

Az *irodalomkutatás* bevezetést nyújt a kosárlabda sportanalitika fejlődésének irányzataiba és a létező eszközök módszertanába, ezáltal kontextusba helyezi a fejlesztési lépéseim kezdeti alapjait.

A *feltáró adatelemzés* elsősorban a 2. fejezetben kapott szerepet a letöltött adatok struktúrájának megértésében és feldolgozásában, valamint a különböző forrású adatok egységes sémájának kialakításában.

Az *adatok előkészítése* során megtörtént a modell tanításához szükséges adatszerkezet előállítása és tisztítása. Ez az egyik legfontosabb pont a projekt szempontjából, ugyanis az inputadatok minősége alapjaiban határozza meg a modell tanulási képességét. Az előkészítés során végbementek a szükséges szűrési lépések az adatok minőségének biztosítása érdekében, valamint a megfelelő transzformációk a bemeneti adatstruktúra előállítása céljából.

A *modell tanítása* a BERT-alapú modell implementációját és tanítását jelenti, melynek részletes architekturális felépítését a 2.4 fejezet részletezi.

A *modellvalidáció* során a cél az elkészült modell eredményeinek kiértékelése mérhető metrikák és vizuális kiértékelés segítségével.

A *webes elemzőalkalmazás implementációja és tesztelése* feladat értelmében az egyszerű felhasználók és elemzők által élvezhető és hasznos kimenetet szolgáltató webes megjelenítőt készítettem. Az eszköz a betanított modell eredményeit felhasználva jeleníti meg a modell kimenetét. A webes rendszer tesztelését pedig unit tesztek segítségével hajtottam végre.

1.5. A dolgozat felépítése

Az 1. fejezet tárgyalja az elvégzendő feladat motivációját és az ehhez szükséges irodalomkutatást. Részletezi az implementált rendszer működését és az alkalmazott eszközök módszertanát, valamint tartalmazza a feladatkiírás pontosítását.

A 2. fejezet három részre bontja a megvalósított rendszer dokumentációját. A 2.2. szakasz megemlíti az adatletöltési folyamat legfontosabb lépéseit. A 2.3. szakasz bemutatja a szekvenciák előállítását, valamint a modell bemenetének előkészítését. A leírás részletessége az adattranszformációval töltött munka jelentős mennyiségét is szemlélteti. A 2.4. szakasz részletezi a modell architektúrám felépítését és elemeinek részletes működését, az alkalmazott augmentációs módszereket, a veszteségfüggvények definiálását és a tanulási folyamatot segítő eszközök implementációját. A 2.5. szakaszban bemutatásra kerül a modell kiértékelésének eredményeit szemléltető webes rendszer tervezésének, felépítésének és tesztelésének dokumentációja.

A 3. fejezetben a modell eredményeinek kiértékelése foglal helyet, ahol részletezésre kerülnek az alkalmazott metrikák, valamint sor kerül vizuális elemzésre is.

Zárásként a 4. fejezet összefoglalja az elvégzett munkámat és eredményeit, valamint jövőbeli továbbfejlesztési lehetőségek is felsorolásra kerülnek.

2. fejezet

A tervezés részletes leírása

2.1. Adatforrások bemutatása

Az adatletöltő modul feladata a nyers input adatok lokális összegyűjtése és az adatelőkészítéshez szükséges séma előállítása. Az 1.1 szakaszban említett SportVU és NBA API adatok szolgáltatják a folyamat bemenetét. Ahogy már említettem, a tracking adatok méretéből adódóan a feldolgozási módszerek túlmutatnak a legegyszerűbb Python-os adatkezelő csomagok használatán mint a Pandas. A Polars könyvtár [15] alkalmazása mellett számos hatékonyságnövelő megoldást implementáltam, melyek az adatkezelés hatékonyságát és az erőforrások optimális kihasználtságát hivatottak optimalizálni, ekkora adatméret mellett. A felhasznált tracking adatok a [16] github repository-ból érhetők el, ami a 2015–16-os NBA szezon meccseit tartalmazza, 7-Zip formátumban. Az archívumok mérkőzésenként egy-egy JSON fájlt tömörítenek és ezek tartalmazzák a hozzá tartozó tracking adatokat. A JSON sémában megjelennek a már említett 25 fps felbontással rendelkező háromdimenziós játékos- és labdakoordináták, az aktuális negyed, támadóidő, valamint az egyéb csapatokra vonatkozó és játékos szintű metaadatok. Fontos, hogy minden mérkőzés rendelkezik egy egyedi azonosítóval `game_id` néven. Ez a JSON fájlok elnevezésében is megjelenik a könnyebb kezelhetőség érdekében.

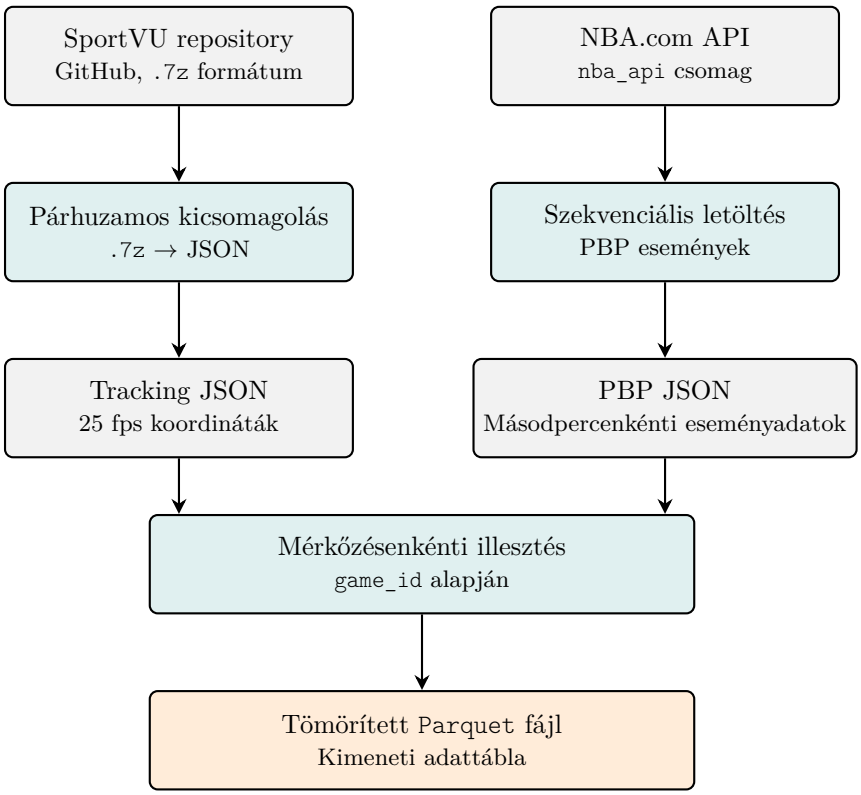
Az NBA által publikált historikus statisztikai PBP adatok az `nba_api` Python csomag [17] segítségével érhetők el. A PBP adatok szintén rendelkeznek mérkőzésenként egyedi `game_id` azonosítóval, ami konzisztens a SportVU rendszer azonosítóival. Az `nba_api`-ban implementált API kliens segítségével az nba.com-on nyilvánosan publikált historikus adatok egyszerűen letölthetők, ugyanakkor egy teljes szezon letöltéséhez robusztus kapcsolatkiépítésre és adatkezelésre volt szükség.

2.2. Az adatletöltés lépései

Az adatletöltési folyamat a 2.1. ábrán látható, amely összefoglalja az elvégzett lépéseket. A 7-Zip archívumokból párhuzamosítva kicsomagolásra kerül mindegyik JSON fájl. Ezt követően szekvenciálisan letöltésre kerülnek a `nba_api` csomag segítségével a PBP eseményeket tartalmazó fájlok. Végül mérkőzésenként illesztésre kerülnek ezek a különböző forrásból származó adatok, figyelembe véve az eltérő felbontásukat is. Az elkészült kimenet egy tömörített Parquet fájlba kerül mentésre.

2.3. Szekvencia generálás és jellemzők kinyerése

A következő lépésben az adatelőkészítési és transzformációs lépéseket mutatom be, amely során a letöltés által előállított adathalmazt fix méretű szekvenciákká történő átalakí-



2.1. ábra. Az adatletöltési és előfeldolgozási folyamat lépései.

tása kerül középpontba. A szekvenciák határainak megállapításához egy csúszóablakos módszert implementáltam annak érdekében, hogy a határok ne éles vágásként, hanem finomabb átmenetként jelenjenek meg az input adatokban. Fontos megemlíteni, hogy az ebben a fejezetben tárgyalt előkészítési folyamat kulcsfontosságú a modell hatékony és eredményes tanulásához, valamint működéséhez.

Az elvárt kimenet mérkőzésenként egy tömörített [N, 200, 131] dimenziós NumPy tömböt tartalmazó archívum. N a mérkőzésből előállított szekvenciák számát jelöli, ablakonként 200 frame, valamint a *feature engineering* eredményeként előállt 131 jellemző kerül tárolásra frame-enként.

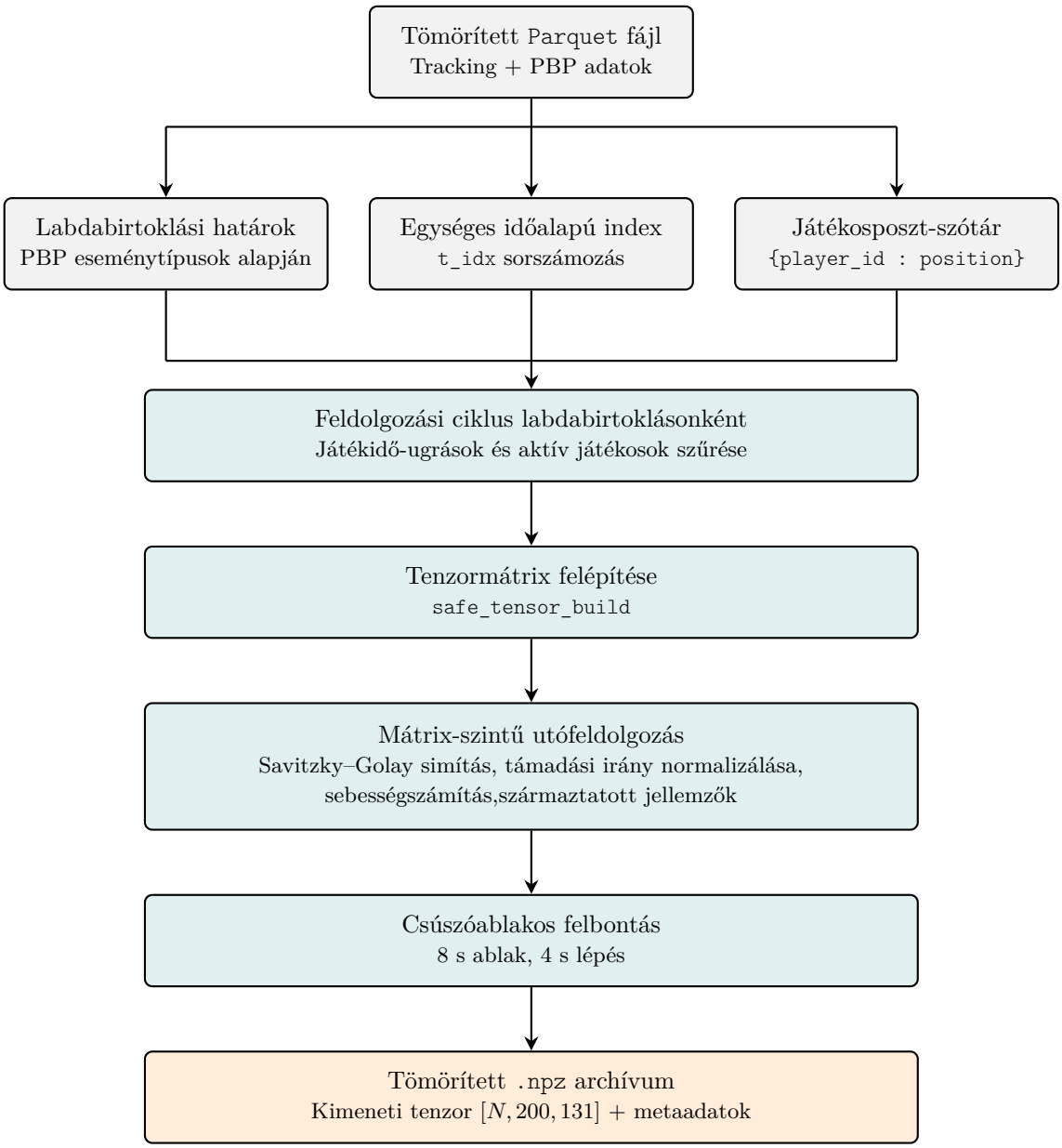
2.3.1. Konfiguráció és alapparaméterek

Az implementáció magyarázata előtt fontos kitérni a rögzített paraméter-konstansokra, melyek az egész modul során állandó értékekként kerülnek hivatkozásra.

A `FPS` változó rögzíti a SportVU tracking rendszer által szolgáltatott tracking adatok 1 másodpercre eső felbontását.

A `COURT_X_MAX` és `COURT_Y_MAX` változók a SportVU rendszer által rögzített adatok koordináta-tartományának felső határait rögzítik lábban megadva. Ugyanakkor fontos megemlíteni, hogy a hivatalos 94 láb × 50 láb NBA pályaméreteknél [18] ezek az értékek nagyobbak, ugyanis a pályán kívüli területet is tartalmazzák. Ez azért hasznos, mert sikertelen játékok során, vagy bedobások esetén a játékosok és a labda is bekerülhetnek ebbe a zónába. Ezek az értékek fontos szerepet töltenek be a pályanormalizáció végrehajtásában.

A csúszóablakos módszerhez rögzítésre kerültek a különböző ablakparaméterek. A módszer célja, hogy az eltérő hosszú labdabirtoklás alapú szekvenciák rögzített méretű, egymással részben átfedésben lévő ablakok szerint kerüljenek felbontásra. Ez egyrészt növeli az adatkészlet méretét, másfelől a fix hosszú szekvenciák egységesen, könnyebben



2.2. ábra. A szekvenciagenerálás és jellemzőkinyerés folyamatának lépései.

kezelhetőek. Végül, ami már a bevezetésben is említésre került, lehetővé teszi, hogy éles határok helyett, összefüggő átmenetekként kezeljük a szekvenciák határait egy labdabirtoklás on belül. A csúszóablakos módszer részletesebb leírására a 2.3.8. alszakaszban kerül sor. A `WINDOW_SEC` változó értéke rögzíti egy támadáson belüli kisebb szekvenciák hosszát, azaz a csúszóablak méretét. A `STEP_SEC` megadja másodpercekben az ablak lépésének nagyságát. A `FIXED_FRAMES` rögzíti az adott ablak frame-ek szerinti hosszát. A `WINDOW_SEC` hosszú ablakra emiatt $\texttt{WINDOW_SEC} \cdot \texttt{FPS} = 8 \cdot 25 = 200$ frame kerül egy ablakba.

Ahhoz, hogy egy azonosított labdabirtoklást lezáró esemény kimenetele biztosan bekerüljön az ablakba, egy 2 másodperces puffert alkalmaztam, amit a `POST_EVENT_BUFFER` változó rögzít.

Az `INPUT_DIM` változó tartalmazza a kimeneti jellemző vektor méretét, ami 131 előállított jellemzőt jelent. Ezek az értékek minden frame-re kiszámításra kerülnek a feldolgozási

lépések során. A frame-enként előállított jellemzővektor szerkezetét a 2.1. táblázat tartalmazza. Az említett konfigurációs konstansokat a F.3.2. kódrészlet foglalja össze.

Oszlop / Offset	**Tartalom**	**Normalizálás**
Labda feature-ök (oszlopok 0–9)		
0	Labda x	$/\,100.0$ ($\rightarrow [0,1]$)
1	Labda y	$/\,53.0$ ($\rightarrow [0,1]$)
2	Labda z (magasság)	$/\,15.0$
3	Labda Δx (sebesség x)	frame-ről frame-re delta
4	Labda Δy (sebesség y)	frame-ről frame-re delta
5–8	Labda zóna one-hot	[paint, midrange, three, invalid]
9	Időbeli pozíció	$f/(n_{\text{frames}}-1),\ 0 \rightarrow 1$
Csapat spacing (oszlopok 10–11)		
10	Team A spacing terület (shoelace)	–
11	Team B spacing terület (shoelace)	–
Játékos feature-ök (Team A: 12–66, Team B: 67–121) *Bázis offset ($p = 0, \ldots, 4$): Team A: $12 + p \times 11$, Team B: $67 + p \times 11$*		
+0	Játékos x	$/\,100.0$
+1	Játékos y	$/\,53.0$
+2	Sebesség Δx	clip(-0.1, 0.1)
+3	Sebesség Δy	clip(-0.1, 0.1)
+4	Távolság a labdától	láb $/\,100.0$
+5	Labdabirtokló flag	1.0, ha < 3.5 láb és labda $z < 0.25$
+6	Legközelebbi kosártól mért távolság	láb $/\,100.0$
+7–+10	Zóna kódolás	[paint, midrange, three, invalid]
Csapat- és interakciós feature-ök (oszlopok 122–130)		
122	Team A legkisebb belső páros távolság	$/\,50.0$
123	Team B legkisebb belső páros távolság	$/\,50.0$
124	Team A paint jelenléti aránya	#paint játékos/5.0
125	Team B paint jelenléti aránya	#paint játékos/5.0
126	Team A x-koordináták szórása	nyers std
127	Team B x-koordináták szórása	nyers std
128	Labda össz-sebesség	$\sqrt{\Delta x^2 + \Delta y^2}$
129	Team A–B legkisebb kereszt-távolság	$/\,50.0$
130	Támadó idő	clamp(0, 24)/24.0

2.1. táblázat. A kimeneti feature vektor felépítése és normalizálási szabályai.

2.3.2. A szekvencia-alapú feldolgozás lépései

A szekvenciák előállítása mérkőzésenként végzett iteratív folyamat. A futtató script végighalad minden mérkőzés 2.2. fejezetben megismert adatain és sorban végrehajtja rajtuk a transzformáló lépéseket, amelyeket a következő alfejezetekben a végrehajtási sorrendjük szerint mutatok be. Az egyes lépések sorrendjét a 2.2. ábra szemlélteti.

Kezdő lépésként minden mérkőzés feldolgozását megelőzően ellenőrzésre kerül, hogy létezik-e az adott mérkőzéshez tartozó adatforrás, majd Polars-szal betöltésre kerülnek a `Parquet` fájlból.

2.3.2.1. Labdabirtoklási határok kinyerése

Már a 2. fejezetben is említésre került, hogy a SportVU tracking adatok önmagukban nem tartalmazzák a pályán bekövetkező eseményeket. Ez indokolta a PBP adatok beépítését is, ugyanis ez a többletinformáció elengedhetetlen a labdabirtoklás határainak a meghatározásához. A tracking adatokból származó tér-idő alapú információ önmagában csak a labda csapatok közötti átadásának detektálását tenné lehetővé. Ugyanakkor a labdabirtoklások véget érhetnek a labda eladása nélkül is, például ha kimegy a labda a pályáról, ha védők személyi hibát követnek el, vagy ha dobás történik, ekkor a labda azonos csapatnál maradhat, de a labdabirtoklás véget ér. Emellett egy dobás után például nem feltétlenül szerencsés a lepattanóért történő harcot a szekvenciákban hagyni, ugyanis ez torzítást vinne a szekvenciákba.

A feldolgozás negyedenkénti bontásban történik, mivel egy negyed eleje és vége egyértelműen labdabirtoklási határt jelent. Egy NBA mérkőzés négy 12 perces negyedből áll, ezek egyenként 720 másodpercet tesznek ki. A `current_start` az adott labdabirtoklás kezdetét jelöli, induláskor ez az érték a negyed elejét jelenti. A határesemények a PBP adatokból kinyert `Made Shot`, `Missed Shot`, `Turnover`, `Foul` és `Violation` eseménytípusok adják. A `Made Shot` sikeres, a `Missed Shot` sikertelen dobást jelent a támadás végén. A `Turnover` eladott labda, a `Foul` személyi hiba elkövetése, a `Violation` pedig más szabálytalanságok esetén kerül bejegyzésre a PBP adatok közé.

A labdabirtoklási határok azonosítása a F.3.1. forráskódban látható `extract_possession_bounds` függvénnyel történik. A módszer végigiterál az adott negyed említett eseménytípusain, majd kiszámolja a labdabirtoklás időtartamának hosszát. Ha ez legalább `WINDOW_SEC` hosszú, akkor ezt a birtoklást eltároljuk, egyébként eldobjuk, mert túl rövid a további elemzéshez, mivel egyetlen csúszóablak hosszát sem tölti ki az azonosított intervallum. Ezek a rövid labdabirtoklások általában zajos játékokat tartalmazhatnak, ezért döntöttem úgy, hogy inkább ne kerüljenek be a modell által megkapott inputadatok közé. A sikeresen megállapított birtoklási határok után a folyamat tovább halad a következő határok beazonosítására, a `current_start` változót frissíti az előző birtoklás végét jelző (`end_clock`) értékével, így ez folytonosan jelzi a következő feldolgozandó labdabirtoklás kezdetét.

2.3.3. Játékosposzt-szótár felépítése

A modell tanulása során biztosítanunk kell, hogy a felépített kimeneti mátrixban a játékosok pályán betöltött posztjuk, szerepük alapján mindig konzisztens sorrendben szerepeljenek. A tracking adatokból csak az érvényes játékos pozíciót tartalmazó rekordok kerülnek megőrzésre. A játékosposztokat egységesen egy `{player_id : position}` szerkezetű szótárban tároltam, amelynek segítségével a későbbiekben a játékosok poszt szerinti sorrendezése is lehetségessé válik.

2.3.4. Egységes időalapú indexelés bevezetése

Az input tracking adatokban a frame-ek nem szekvenciális indexszel, hanem a `(quarter, timestamp)` párral, az aktuális negyed és a hozzá tartozó timestamp értékekkel vannak azonosítva. A modell bemeneti mátrixa viszont szigorúan egymást követő időpillanatokból épül fel. A 2.3.8. szakaszban bemutatott csúszóablakos vágás módszere is folytonos frame-

alapú indexelést követel meg. Ennek következtében bevezetésre került a mérkőzésenként egységes időalapú indexelés, aminek az értékei a `t_idx` mezőben foglalnak helyet.

Az új indexelés kialakítása három lépésben történik. Először a tracking adatok közül kiszűrésre kerülnek nulla vagy `null` x-koordinátájú értéket tartalmazó rekordok, mint érvénytelen, hibás adatok. Ezt követően megtörténik a rekordok `(quarter, timestamp)` alapú rendezése. A tábla egyedi `(quarter, timestamp, game_clock)` hármasai időrend szerint sorrendezésre kerülnek, majd új sorszámot kapnak a `t_idx` mezőben. Ennek a kulcsnak a segítségével minden eredeti tracking rekord rendelkezik azzal az információval, hogy a folytonos idősorrendben pontosan hol helyezkedik el az adott időpillanat.

2.3.5. Feldolgozási ciklus a labdabirtoklás-határok alapján

A 2.3.2.1. szakaszban azonosított labdabirtoklási határok, valamint a 2.3.4. szakaszban bevezetett `t_idx` egységesített időalapú index birtokában következik a feldolgozási ciklus, ami végigiterál az összes azonosított labdabirtokláson, és külön-külön elvégzi az ezekhez tartozó adatok összeállítását. A következő alszakaszok a feldolgozási ciklus lépéseit fejtik ki részletesen.

2.3.5.1. Játékidő ugrásainak kezelése

A mérkőzések során gyakran előfordul, hogy egy szünet után a játékvezetők az óra visszaállítását ítélik meg. Másrészről, ha a játék megáll valamilyen megszakítás miatt, a tracking adatok a szünet közben is érkezhetnek. Ezek az anomáliák akár duplikált vagy nem folytonos időbélyegekkel rendelkező szekvenciákat okoznának, ami egyértelműen torzítaná az időbeli mintázatok felismerését a modell számára.

A jelenség detektálásához a vizsgált ablakon belüli `game_clock` értékek sorrendje alapján kerül sor a vizsgálatra. Az ablak első, időrendben legkorábbi frame-jének óraértéke a referencia (`max_gc`) változó értéke, ennél csak kisebb óraértékek fordulhatnak elő egy helyes szekvenciában, ugyanis a játékidő monoton csökken az idő előrehaladtával. Ezt követően megkeresésre kerülnek azok az indexek, ahol az óraérték a maximumnál több mint $0,05$ másodperccel kisebb (`drop_idx`). A csökkenő óra miatt a `drop_idx` azt a pillanatot jelzi, amikor az óra ténylegesen elindul.

Ha az ablak elején több frame-en keresztül azonos vagy közel azonos órával érkeznek a tracking rekordok, akkor a `drop_idx` egynél nagyobb értéket vesz fel. Ekkor az ablakot a `drop_idx - 25` indextől kell levágni, ez kiszűri a felesleges, álló órájú frame-eket, miközben megőriz egy rövid előzményt is.

2.3.5.2. Pályán lévő aktív játékosok ellenőrzése

Az adatok vizsgálata során megjelentek olyan szekvenciarészletek, amikor adott pillanatban több mint öt játékos jelent meg egy csapatban. A pálya szélén véletlenül detektált játékosok vagy esetleges adathibák miatt fordulhat elő és ez torzítaná a későbbiekben felépített mátrix tartalmát.

A probléma kezelésére egy frame-szintű ellenőrző szűrést alkalmaztam. A szűrés azokat a támadókat és védőket tekinti *stabil* játékosoknak egy adott szekvenciában, akik a frame-ek minimum 40%-ában jelen vannak. A `team_id > 0` feltétel kizárja a -1 azonosítójú labdarekordokat, mivel a szűrés célja kizárólag a játékos-duplikációk kezelése volt. A labda minden esetben mentésre kerül a felépülő kimeneti mátrixba.

2.3.6. Tenzormátrix felépítése

Az input adatok táblázatos relációs alakban kerülnek betöltésre, a modell viszont egy fix méretű kétdimenziós mátrixot vár a bemenetére. A `safe_tensor_build()` műveletem felelős ennek a bemeneti mátrixnak az előállításáért, az eddig említett előzetes feldolgozási lépések kimenetét felhasználva. A lépések során megtörténik a táblázatos séma átalakítása, és frame-enként a 131 feature a mátrix megfelelő oszlopaiba kerül beillesztésre. A kimeneti feature-vektor felépítését a 2.1. táblázat tartalmazza.

A tenzorépítés folyamatát a Numba JIT (*Just-In-Time*) fordító eszközével [19] implementáltam. Ez az eszköz a Python függvényt gépi kódra fordítja, így gyorsítva a ciklusok iterációinak hatékony végrehajtását.

Az inicializálás során először egy nullákkal feltöltött mátrix kerül létrehozásra. A későbbiekben azok az adatok, amelyek 0 értékűek maradnak, hiányzó adatnak minősülnek.

Ezt követően a táblázatos adatok minden egyes sora feldolgozásra kerül egy cikluson keresztül. A hatékonyság érdekében a NumPy `searchsorted` műveletével meghatározásra kerül, hogy az adott `t_idx` érték a `target_indices` tömb melyik pozíciójához tartozik. Ha az adott index nem található a tömbben, akkor a feldolgozás kihagyja az adott rekordot.

A koordináták normalizációja a 2.3.1. alfejezetben rögzített pályaméretek skáláján történik, így minden x és y koordináta a $[0, 1]$ intervallumba kerül. A támadóidő normalizálása során először szűrésre kerülnek az adathibából következő 0-nál kisebb értékek, majd az így megmaradt értékek 24-gyel kerülnek elosztásra, hogy szintén $[0, 1]$ közötti intervallum szerint kerüljenek felhasználásra.

Ha a feldolgozott sor `team_id` mezője 0-nál kisebb, akkor az adott sor a -1 értékű labdarekordhoz tartozik. A labda koordinátái a mátrix első három oszlopába kerülnek beírásra. Játékos rekord esetén a játékos helye a `team_a_arr` vagy `team_b_arr` tömbben kerül megkeresésre, majd a megfelelő x és y koordináták értékei felülírásra kerülnek a mátrix megfelelő celláiban.

2.3.7. Mátrix-szintű utófeldolgozási lépések

A felépített mátrixon további négy utófeldolgozási lépés kerül végrehajtásra, melyek sorrendje a következők szerint történik.

2.3.7.1. Savitzky–Golay simítás

A tracking koordinátákban felléphetnek zajok, mind a kamerarendszer pontatlanságaiból, mind a 25 fps felbontás apró pontatlanságaiból. A differencia-alapú sebesség feature-ök kiszámításához ezeket az apró pontatlanságokat célszerű valamilyen adattranszformációval kezelni, mivel ezek kiugró sebességekhez és anomáliákhoz vezetnének.

Az implementációmban egy Savitzky–Golay szűrő [20] simítja ezeket a zajos értékeket megfelelő polinomillesztéseket alkalmazva, ugyanakkor megtartja a játékosok hirtelen irányváltásait, amiket a modellezés szempontjából rendkívül fontos megőrizni. Ezt a feltételt egyszerűbb mozgóátlag-alapú szűrők nem tudnák garantálni, valamint implementációja a SciPy csomag `savgol_filter` műveletének használatával egyszerűen megvalósítható.

Az implementációban 22 oszlopon történik simítás, a labda, valamint a játékosok x és y koordinátáin. A szűrő csak a létező, hiánytalan indexekre kerül meghívásra, így a hiányos adatokat nem torzítja. A szűrő `window_length=5`, `polyorder=2` paraméterek alkalmazásával 5 frame hosszú ablakokon, másodfokú polinomokkal végzi az illesztést.

2.3.7.2. Támadási irány normalizálása

Egy mérkőzés során a csapatok félidőben térfelet cserélnek, így mindkét csapat támad jobb és bal irányba is egy mérkőzés alatt. A taktikai szegmentáció során viszont érdektelen, hogy egy taktikát bal vagy jobb irányban játszanak, ugyanis a támadó térfélre ezek a taktikák invariánsak. Mindegy, hogy egy *pick and roll* játékot melyik térfélen játsszák le, attól ez a taktikai elem ugyanaz marad. Az azonos irányok kezelése nélkül a modell viszont megpróbálná megkülönböztetni a pálya bal és jobb térfelén történő játékok közötti különbséget, ami nem cél.

Ezt a problémát a támadási irány normalizálásával oldottam meg. A szekvencia középső tartományában a labda átlagos x-koordinátájának vizsgálata alapján, ha az érték kisebb, mint 0,5, akkor a támadás a pálya bal térfele felé történik, egyébként jobbra. A 0,5 a pályanormalizáció utáni $[0, 1]$ értékkészletű normalizált pályakoordináták alapján került meghatározásra. Választásom alapján, ha a támadás balra történik, akkor szükséges egy tükrözés az x-koordináták mentén, hogy minden támadás a jobb térfélen jelenjen meg.

2.3.7.3. Frame-szintű sebességértékek számítása

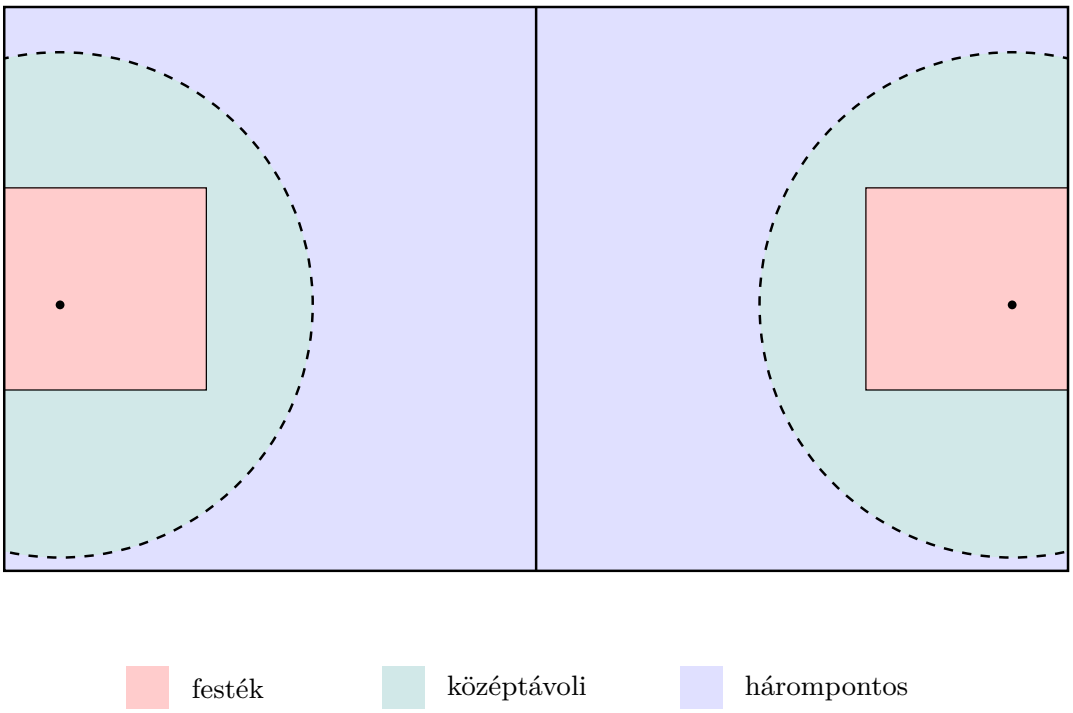
A játékosokhoz és labdához tartozó sebességadatok a 2.3.7.1. alszekcióban végzett simítás után, delta-alapú differenciális módszerrel kerülnek kiszámításra. Csak azokra a koordinátákra számítódnak, ahol mind a megelőző, mind a következő frame-ben rendelkezésre állnak a koordinátaadatok a differenciális művelet elvégzése érdekében. A $[-0{,}1; +0{,}1]$ intervallumra történő levágás (*clip*) biztosítja, hogy az esetleges tracking adat hibákból adódó teleportálások miatt ne kerüljenek kiugró sebességértékek a mátrixba. A clip felső határa körülbelül a valós játékossebesség sokszorosa, ebből adódóan az ennél magasabb értékek valószínűleg teleportálási hibáknak tekinthetőek.

2.3.7.4. Származtatott jellemzők kiszámítása

Ennek a lépésnek a célja, hogy az adatokat, a későbbi modell tanulást segítő extra információval lássam el, melyeknek az eredeti koordinátákból történő kikövetkeztetése sokkal nehezebb feladat lenne ezen változók előállítása nélkül. Az implementációkat szintén `Numba`-alapú gyorsítással hajtottam végre a hatékonyság érdekében. A `safe_tensor_build` művelet eddig csak a koordinátapozíciókat és a támadóidőt dolgozta fel. Ezeket a meglévő értékeket felhasználva, további hasznos jellemzők értékei kerülnek kiszámításra.

Pályazónák meghatározása. Létrehoztam a pályán detektált objektumokhoz tartozó zónafelosztást, amit a `get_zone_one_hot` művelet implementál a 2.3. ábrán látható módon, megvalósítása a F.3.3. kódrészlet mutatja be. A művelet ezeket a területeket domain tudás szerint meghatározott zónarendszer alapján rögzíti a kimeneti mátrixban one-hot kódolt jellemzőkként. Megállapításra kerülnek festék, hárompontos és középtávoli zónák, valamint érvénytelen koordináták, például a pályán kívüli területek.

Játékos-szintű jellemzők. Különböző játékosokhoz tartozó jellemzők is számításra kerülnek. Minden játékoshoz hozzárendelek egy normalizált, labdától mért euklideszi távolságot, valamint egy bináris indikátort, amely a labdabirtoklást jelzi. Ennek az értéke 1, ha egyszerre teljesül, hogy a labda távolsága az adott játékostól kisebb, mint 3,5 láb, valamint a labda magasságát jelző z-koordináta kisebb 3,75 lábnál. Ez a második feltétel kizárja a levegőben lévő labdát dobás vagy passz esetén. Minden játékoshoz kiszámításra kerül a legközelebbi gyűrűtől mért, normalizált távolság is. A fentebb említett `get_zone_one_hot` művelettel megállapított zóna-címkézést a játékosok is megkapják. Végül meghatároztam



2.3. ábra. A feature-vektorban megjelenő zónafelosztás.

azt is, hogy egy adott játékos a támadott gyűrű előtti festékterületen belül vagy kívül tartózkodik-e.

Csapat-szintű jellemzők. A csapatok pályán elfoglalt területét a `calc_spacing_area` határozza meg. Először kiszámítja az adott csapat öt játékosa alapján a koordinátáik centroidját, valamint minden játékos pontjának a centroiddal bezárt szögét. A pontok a szögük alapján kerülnek rendezésre az előjeles területképlet helyes alkalmazása miatt, majd a shoelace formulával (*Gauss-féle területképlet*) [21] kerül meghatározásra a csapat játékosai által kifeszített sokszög területe, amit a pályán történő helyezkedés mérőszámaként alkalmaztam. Minél nagyobb a lefedett terület, annál jobb az adott csapat helyezkedése. A számítást a 2.1. képlet írja le, ahol $n \leq 5$ az érvényes játékosok száma. A képlet minden tagja az adott élhez és az origóhoz tartozó háromszög kétszeres, előjeles területét összegzi, így ez a sokszög területének kétszeresét adja.

$$A \;=\; \frac{1}{2}\left|\sum_{i=1}^{n}(x_i\,y_{i+1} - x_{i+1}\,y_i)\right|, \qquad (x_{n+1}, y_{n+1}) \equiv (x_1, y_1), \tag{2.1}$$

További csapatszintű jellemzőként meghatározásra kerül a paint arány, amely megmutatja, hogy hány játékos tartózkodik a festékterületen, 5-tel normalizálva, valamint ehhez társul még a csapatok x-koordinátáinak szórása is, amely a csapat x-tengely menti kiterjedését mutatja meg a pályán. Végül a csapaton belül vett játékosok páronkénti legkisebb távolsága az elzárások azonosítását segíti, ugyanis ha két támadó közel kerül egymáshoz, ez taktikai szempontból gyakran elzárást jelent. Ezzel együtt kiszámításra kerül a támadók és védők közötti páronkénti minimum-távolság is, amely az emberfogás szorosságáról ad extra információt.

2.3.8. A mátrixon végzett csúszóablakos felbontás

A 2.3.2.1. alfejezetben tárgyalt, labdabirtoklás-határok alapján létrehozott szekvenciák hossza eltérő lehet. A 8 másodperces minimumtól, a teljes támadóidőt kitöltő 24 másodperces labdabirtoklások is előfordulhatnak. A változó hosszúságú szekvenciák viszont megnehezítik egy fix kimeneti dimenziókkal rendelkező mátrix létrehozását. Emiatt a változó hosszú szekvenciákat egységes, fix hosszúságú ablakokra bontottam fel csúszóablakos megközelítés szerint. Fontos megemlíteni, hogy ez a módszer lehetővé teszi a határok átmenetének folytonos reprezentációját az adatokban. A fix pozíció alapú vágások esetén ez a folytonosság nem lenne garantálható.

Az implementáció a 2.3.1. alszakaszban említett konstansok értékeit használja fel, `STEP_SEC` 4 másodperces lépésközzel halad előre, és minden lépésben `WINDOW_SEC` 8 másodperces, azaz 200 frame hosszúságú ablakot jelöl ki feldolgozásra.

2.3.9. Metaadatok összeállítása és kimeneti tenzor mátrix mentése

A későbbiekben betanított modell kiértékeléséhez fontos eltárolni a kimeneti szekvenciákhoz tartozó metaadatokat, azaz, hogy egy adott szekvencia melyik mérkőzésből, negyedből és milyen PBP-eseményből származik, valamint mely játékosok találhatóak meg a pályán. Ezeknek az információknak a kimentéskor történő összegyűjtése később megkönnyíti az elemzési feladatok elvégzését is.

Minden metaadat-bejegyzést egy egyedi szekvenciaazonosítóval láttam el, ami a `{game_id}_pos{N}_w{W}` logikát követi. Az N a labdabirtoklás sorszáma a mérkőzésen, a W pedig a csúszóablak sorszáma az adott labdabirtoklás on belül. Egy bejegyzés tartalmazza a mérkőzés egyedi azonosítóját, az aktuális negyedet, a PBP eseménytípust, valamint a támadó és védő csapat játékosainak egyedi azonosítóit.

A mérkőzés feldolgozásának végén a felépített mátrix egy tömörített `.npz` archívumba kerül kimentésre. A `.npz` a `NumPy` beépített tömörített tárolási formátuma a vektorok és mátrixok helytakarékos tárolására. A metaadatok nem mérkőzésenként kerülnek mentésre, hanem a teljes futás végén egyetlen közös fájlba kerülnek kiírásra, így minden metaadat egyetlen helyen lesz elérhető.

2.4. Modell architektúra bemutatása

Ebben a fejezetben az implementált modell architektúrájának részletes bemutatása következik. A megközelítésem a már említett BERT-alapú [10], önfelügyelt tanulást végző modell felhasználhatóságát vizsgálja, amelynek célja a 2.3. szakaszban előállított $[N, 200, 131]$ dimenziós bemeneti tenzor további feldolgozása és a tanítási folyamat elvégzése. Ennek eredményeként a modell kosárlabda-taktikák szerint elkülönülő embedding-reprezentációkat állít elő, amelyek alkalmasak a szekvenciák taktikai szegmentációjára. A bemeneti szekvenciák viszont nem statikus állapotokat, hanem a csapatok és a labda mozgásának folyamatát jelentik, tér és idő dimenzióban. Így cél, hogy az architektúra tükrözze ezt a kettősséget is.

Ismét fontos hangsúlyozni a 1.2.1 és 1.2.2. alszakaszokban bemutatott felügyelt módszerek és reprezentációtanuló megközelítések eltéréseit, ahol az utóbbi esetén a szekvenciák belső szerkezete alapján végezzük a tanítást. Ennek elérése érdekében az architektúra párhuzamosan öt komplementer, egymást kiegészítő pretext feladatot tanul, ezzel kikényszerítve a robusztus reprezentáció kialakulását, ahogy ez a 1.2.3 és 1.3. szakaszokban már bővebben kifejtésre került.

A modell a tanulási fázis során a 131 dimenziós frame-vektorokon több feldolgozási lépést hajt végre. Az első egy gráf-neurális háló, ami frame-szinten kódolja a játékosok és a labda közötti térbeli kapcsolatokat. A kapott jellemzőket egy konvolúciós réteg időben tömöríti: a `SUBSAMPLE` $= 2$ értékű előzetes mintavételezés és a `TEMPORAL_STRIDE` $= 5$ lépésközű konvolúció hatására a 200 frame-es szekvenciák 20 hosszú tokenre redukálódnak. Ezek a tokenek egy Transformer-encoder bemenetét alkotják, amely az időbeli temporális összefüggések tanulásáért felel. Az encoder 128 dimenziós kimenetén a *Phase Pooling* mechanizmus egy további aggregálást végez. Ez a kimenet a tokenek idő-alapú korai, középső és késői fázisai szerint szétbontva, a `[CLS]` tokennel kombinálva kerül kiszámításra. A modell célja a támadó taktikák reprezentációs tanulása és elemzése, emiatt fontos implementációs döntés a támadó csapat aszimmetrikus felülsúlyozása az architektúra építőelemeinek felépítése során.

2.4.1. Adatbetöltés

2.4.1.1. Memmap-builder implementációja

A szekvenciák mérkőzésenként egy-egy `.npz` fájlban kerültek kimentésre. A betöltés során az írási és olvasási műveletek száma és a párhuzamosított DataLoader-ek jelentették a szűk keresztmetszetet. A gyorsítás érdekében egy memmap-pal ellátott `npy` fájlba töltöttem az összes mérkőzés szekvenciáit, ennek segítségével hatékony cache-alapú szekvenciaszintű betöltést lehet megvalósítani. A művelet során két kimeneti állományt hoztam létre, a `merged_X.npy` egy $[N, 200, 131]$ dimenziójú adatmátrix, ami a szekvenciákat tartalmazza, valamint a $[N, 200]$ dimenziójú `merged_masks.npy` fájlt, ami az érvényes és érvénytelen frame-ek maszkját jelöli a hozzá tartozó szekvenciákban. A fájlok előállítása mérkőzésenként történik, annak érdekében, hogy egy mérkőzéshez tartozó `.npz` fájlt csak egyszer kelljen megnyitni, ezzel is spórolva a betöltési műveleteken.

A 2.3.9. szakaszban említésre került a szekvenciák és metaadataik kimentésének struktúrája. A tanítási folyamat során egy DataLoader objektum minden hívásával egyetlen szekvenciát tölt be a háttértárról, pontosan itt kerül hangsúly a közös memmap-pal ellátott szekvenciák alkalmazására.

A memmap során a metaadatok `game_id` szerint kerülnek feldolgozásra, és a szekvenciák egyetlen olvasási művelet segítségével kerülnek beépítésre a mátrixba az indexeik felhasználásának segítségével. A tanítás alatt a `MemmapNBADataset` osztály `mmap_mode='r'` módban nyitja meg a két memmap struktúrával rendelkező fájlt. A beolvasási művelettel

pedig egy szekvencia a teljes fájl memóriába történő betöltése nélkül kerül feldolgozásra, így ez nagyságrendekkel gyorsabb műveletvégzést eredményez.

2.4.1.2. Dataset implementáció

A PyTorch `Dataset`-ből leszármaztatott `MemmapNBADataset` osztály felelős a modell számára átadott konkrét bemeneti mátrix előállításáért. A `split_indices` paraméter segítségével az indexek alapján tartja nyilván, hogy egy adott szekvencia a tanító vagy validációs halmazhoz tartozik. Mind a tanító, mind a validációs halmaz külön `Dataset` objektumot kap, de ennek az index alapú logikának a segítségével képesek ugyanazon a memmap-pal ellátott fájlon egyszerre dolgozni. A szekvenciák 90%/10% arányban kerülnek szétosztásra tanító és validációs halmazokra. A rögzített `RandomState(42)` paraméter használatával viszont biztosítottam a reprodukálható futtatást, hogy a vágások a futtatások során determinisztikusan menjenek végbe. A `__getitem__` függvény egy betöltött szekvenciát ad vissza egy Python szótár formájában, ami tartalmazza a $[200, 131]$ dimenziójú bemeneti mátrixot és a hozzá tartozó [200] hosszú logikai maszkot.

A `Dataset` objektumokhoz `DataLoader`-ek tartoznak, amik elvégzik az adatok kötegelését és a modellbe történő hatékony betöltését. A `pin_memory=True` paraméter segítségével a CPU-ról GPU-ra történő feltöltést gyorsítottam, valamint a `persistent_workers=True` segítségével a párhuzamos működés optimalizálását próbáltam megoldani, ezzel az epoch-ok között fenntartjuk a párhuzamos worker folyamatokat és nem kell minden epoch elején újraindítani őket.

2.4.2. Adatok normalizációja és augmentáció

A `DataLoader` által tartalmazott adatok jellemzőik mentén már egységes értéktartományba lettek transzformálva, de egységes normalizálás hiányában még alkalmatlanok a modell tanításához, ugyanis a bemeneti jellemzők szórása még mindig eltérő, a folytonos és bináris változókból adódóan a nagyobb szórással rendelkező jellemzők torzítanák a Transformer-encoder figyelmi mechanizmusát. A normalizáció során a teljes tanítóhalmazon kiszámításra kerül az első 131 feature dimenzió átlaga (μ) és szórása (σ), ezek alapján kerül végrehajtásra a normalizáló művelet, amely változókon ez korábban még nem történt meg. Ezt követően hajtottam végre a szekvenciák augmentációs lépéseit is, amelyek szintén ebben az alfejezetben kerülnek bemutatásra.

A lépések a `normalize` függvényben kerültek implementálásra, aminek négy tenzor lesz a kimenete. Az `x_norm` mátrix a normalizált és augmentált értékeket tartalmazza, az `x_clean` a normalizált, de augmentáció mentes mátrix, ami az MLM és Velocity pretext feladatok számára szükséges a rekonstrukció elvégzéséhez. Ezeket a rekonstrukciókat ugyanis az augmentált zaj nélkül végeztem el. Az `x_aug` a normalizálatlan, de augmentált adatokkal a gráf-neurális háló `x_raw` bemenetéül szolgál, hogy a térbeli jellemzők kiszámítása az augmentációval együtt mehessen végbe. Az `m_aug` pedig a frissített maszkot tartalmazza, amely tartalmazza az idő alapú eltolási augmentáció miatt érvénytelenné vált frame-pozíciókat.

2.4.2.1. Y-tengely menti tükrözés

A taktikáknak léteznek bal és jobb irányba végrehajtott verzióik, ugyanakkor ezeket a taktikákat egy csoportba szeretném sorolni a későbbiekben. Annak érdekében, hogy a modell elég bemenetet kapjon a szekvencia mindkét irányból végrehajtott verziójáról az y-tengely tükrözésével végrehajtott augmentációt alkalmaztam. Ez `AUG_Y_FLIP_PROB` (= 0,5) valószínűséggel generál ellentétes oldalon elhelyezkedő párt minden szekvencia esetén. Az y-koordinátákat a pálya $y = 25{,}0$ láb tengelye mentén tükrözi, ami a normalizált

koordináták szerint `center_y_norm` = 25,0/53,0 ≈ 0,472 érték alapján történik. Emellett a szekvenciában szereplő v_y, y irányú sebességvektorok előjelét is megfordítottam.

A pálya x-tengelyére vett tükrözést a 2.3.7.2. szakaszban is említett okok miatt nem építettem be, ugyanis az említett bekezdésekben a cél a támadási irány egységesítése volt. Az x-tengely szerinti augmentáció pedig pont ezt a lépést semlegesítené.

2.4.2.2. Időbeli eltolás

Az időbeli eltoláson alapú augmentációval a célom az volt, hogy egy kötegben szereplő szekvenciák frame-jeit véletlenszerűen 5 időpillanattal (±`AUG_TIME_SHIFT_MAX` = 5) eltoljam valamelyik irányban. Ennek előnye, hogy a szekvencián belüli dinamikai szerkezet változatlan marad, de a modell nehezebben fogja a konkrét frame-pozíciókat memorizálni. A szekvencia elején és végén torzuló időpillanatokat érvénytelenné állítottam, 0 értékekkel történő kitöltéssel.

2.4.2.3. Gauss-zaj hozzáadása

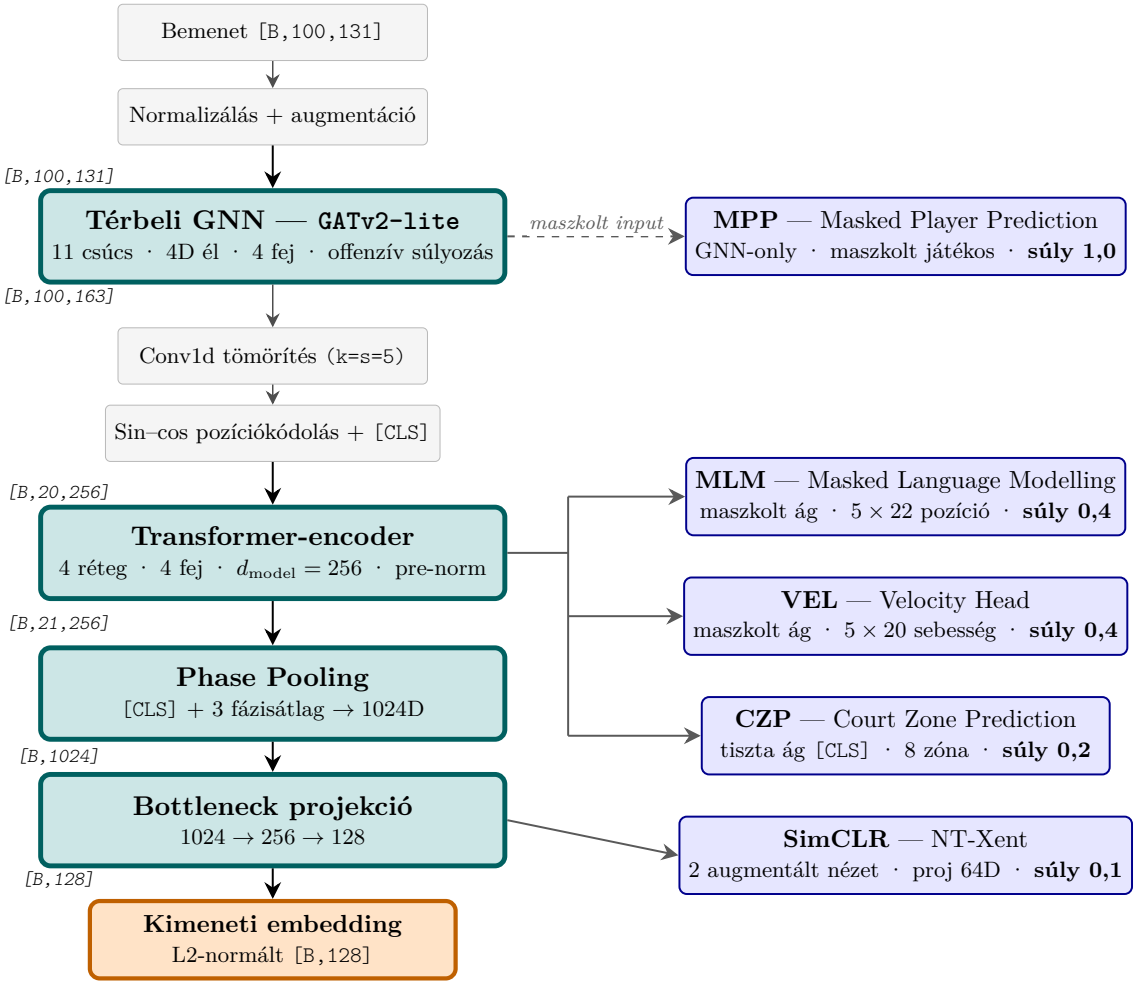
Az előzőekben említett lépéseket követően, az érvényes frame-ekre `AUG_NOISE_STD` = 0,02 szórású, normális eloszlású zajt adtam. A zaj szórását próbáltam úgy meghatározni, hogy kellően kicsi legyen ahhoz, hogy ne bontsa meg a szekvenciák taktikai struktúráját, ugyanakkor regularizációt valósítson meg a bemeneti adatokon. Emiatt a modell nehezebben memorizálja a konkrét koordinátaértékeket és sebességvektorokat.

2.4.2.4. Augmentáció szerepe a SimCLR-tanításban

Fontos megemlíteni, hogy az augmentációs folyamat a SimCLR kontrasztív tanulás alatt központi szerepet tölt be. A `make_simclr_view` függvény egymás után kétszer hívja meg a `normalize` függvényt egyetlen mintára, ezzel előállítva az augmentált párokat. Az egymást követő véletlen műveletek hatására újabb augmentált nézetek jönnek létre, ezért ez a két kimenet alkotja a SimCLR során alkalmazott pozitív párt. A kontrasztív veszteség célja, hogy az embedding térben ezek a pozitív párok egymáshoz közel kerüljenek, miközben az adott kötegben szereplő többi szekvencia, azaz a negatív párok távolodjanak ezektől. A SimCLR-tanítás további részletei a 2.4.5.3. alszakaszban kerülnek bemutatásra.

2.4.3. Modell architektúra

A 1. fejezetben már említésre kerültek az implementáció során használt megközelítések, újítások és a felhasznált modellelemek alapvető működési elvei. Ebben a fejezetben ezeknek a rendszerszintű implementációja kerül vizsgálatra. Részletesen kitérek a modellt felépítő elemek összeállítására, a pretext feladatok részletes bemutatására, a tanulási folyamat során alkalmazott veszteségfüggvényekre, valamint a futást felügyelő eszközökre is, mint az ütemező, a validáció és a korai megállás. A modell részletes architektúráját és a tenzorok dimenzióinak alakulását az adatfolyam során a 2.4. ábra foglalja össze.



2.4. ábra. A teljes modell architektúra felépítése

A rendszer magasabb szintű áttekintését a következő pontok mutatják be:

- **Bemenet**: a 2.3. szakaszban előállított szekvenciák, amelyekből a `SUBSAMPLE = 2` mintavételezés után minden második frame kerül megtartásra, kötegenként 100 frame, frame-enként 131 feature-t tartalmaznak, $[B, 100, 131]$.
- **Gráf-attention alapú GNN (GATv2-lite)**: 10 játékos és a labda alapján felépített 11-csúcsú teljes gráfban megvalósított frame-szintű gráf-attention mechanizmus. A csúcsok közötti kapcsolatokat 32 dimenziós reprezentációkká transzformálja, ami az ezek között kialakuló interakciókat modellezi. Kimenete a 131 dimenziós bemeneti mátrix mellé illesztve halad tovább a folyamatban, emiatt a jellemzők 163 dimenziósra bővülnek, $[B, 100, 163]$.
- **Conv1d tömörítés**: Egy egydimenziós konvolúciós réteg felel a szekvenciák további tömörítéséért, `kernel_size = stride = 5` paraméterrel. Szekvenciánként a 100 frame-et 20 hosszúságú tokenekre bontja, ezzel párhuzamosan a jellemzők dimenzionalitását tovább növeli 256-ra. A 1.3.4. alfejezetben tárgyalt NLP-kosárlabda analógia miatt ezek a tömörített tokenek felelnek meg a szövegelemzésben alkalmazott szavak tokenjeinek. $[B, 20, 256]$.
- **Sin-–cos pozíció-kódolás**: a 1.3.1. szakaszban említett okok miatt a tokenek kiegészülnek pozíció információkkal, hogy a Transformer rendelkezzen információval a szekvenciákon belüli sorrendről.

- **Transformer-encoder**: Egy 4 rétegű, 4 fejjel rendelkező, $d_{\text{model}} = 256$ dimenziós Transformer Encoder. A szekvencia elejére egy tanulható `[CLS]` token került elhelyezésre, ami a tokenek együttes reprezentációját szolgáltatja. A modellbe érkező tokenek dimenziója $1 + 20 = 21$-re bővül. A tokenek az önfigyelmi rétegeken áthaladva kódolják a bidirekcionális szekvenciális kontextust, hűen a 1.3.2. fejezetben tárgyalt BERT implementációjához. $[B, 21, 256]$.
- **Phase Pooling**: A 20 hosszú tokeneket három fázisra bontottam, majd a fázisok átlagait a `[CLS]` tokennel kombinálva egyetlen $4 \cdot 256 = 1024$ dimenziós vektorrá vontam össze. Ezt egy kétrétegű bottleneck réteg segítségével vetítettem vissza a végső 128 dimenzióval rendelkező kimeneti térbe. A célja, hogy még több információt kódoljak a `[CLS]` tokenhez a szekvenciák időbeliségéről.
- **Kimenet**: Az architektúra kimenetét az L2-normált 128 dimenziós embedding vektorok jelentik ($[B, 128]$). Ezek jelentik a további kiértékelési és klaszterezési lépések bemenetét, valamint a SimCLR-veszteség is felhasználja a kontrasztív tanulás megvalósításához.

A feldolgozási lánc lépéseit és a tenzorok dimenzióit a 2.4. ábra szemlélteti.

2.4.3.1. Térbeli figyelem

A feldolgozási lánc legelső eleme a gráf-attention-t megvalósító `GATv2-lite` alapú gráfneurális háló. Célja a szekvenciákon belül, a pályán detektált objektumok közötti térbeli kapcsolatok felismerése és tanulása. GNN nélkül, csak Transformer-encoder alkalmazásával a modellnek a nyers bemeneti koordináták és sebességvektorok értékeiből, valamint időbeli változásaiból kellene következtetnie a geometriai összefüggésekre. A 1. fejezetben is említett újításom, hogy a modell rendelkezzen külön térbeli és temporális időbeliséget megvalósító figyelmi mechanizmusokkal, amelyek együttesen fejtik ki hatásukat a tanulás során.

Az implementáció legelső lépése a gráf-struktúra felépítése volt a bemeneti adatokból. Ennek megvalósítása során fontos kritérium volt, hogy a detektált objektumok között olyan térbeli kapcsolatok kerüljenek beépítésre, hogy ezek alapján a térbeli figyelem ténylegesen megvalósítható legyen. A felépített adatszerkezet 11 csomópontból álló teljes gráfot képez, amely tartalmazza a labdát és mindkét csapat játékosait. Ennek megfelelően három csomóponttípust is definiáltam a labda, a támadó és védő csapatok játékosainak explicit megkülönböztetéséhez. A háló a csomópontokat ezen típusok alapján definiált tanulható `type_embedding`-gel is rendelkezik, így a csomópontok típusaira történő következtetés nemcsak a bemeneti jellemzők alapján, hanem egy tanulható paraméterrel történik. A csomópontokhoz tartozó 8 dimenziós jellemzővektor a labda esetén az abszolút x, y-koordinátákat és sebesség-vektorokat, játékosok esetén a labdától és a támadott gyűrűtől mért távolságokat, valamint a labda közelségét jelző logikai értéket is tartalmazza.

Teljes gráf bevezetésével bármely két csomópontot él köt össze, melyekhez egy-egy `[dist, rel_speed, same_team, has_ball]` felépítésű, 4 dimenziós jellemzővektor tartozik. A `dist` a két összekötött csomópont közötti euklidészi távolságot reprezentálja. A `rel_speed` az objektumok egymáshoz viszonyított relatív sebességét tartalmazza. A `same_team` egy logikai érték a csapattársak összeköttetése esetén logikai 1, egyébként 0 értéket vesz fel. A `has_ball` szintén logikai jelzést ad arról, hogy az adott él kapcsolódik-e a labdához. A fejlesztés során kipróbált gazdagabb él-jellemzők nem hoztak érdemi javulást a térbeli reprezentáció tanulásában, emiatt a végső implementációban ezt a 4 dimenziós változatot tartottam meg.

GATv2-lite figyelmi mechanizmus. A gráfon alkalmazott figyelmi mechanizmus a `GATv2` dinamikus figyelmi módszerén [12] alapul, ennek elméleti háttere a 1.3.3. alfejezetben bevezetésre került. Az általam alkalmazott *GATv2-lite* ennek egy számítási szempontból egyszerűsített változata, amely csökkentett rejtett rétegekkel és dimenziókkal rendelkezik. Működése során a beépített 4 párhuzamos figyelmi fej célja, hogy a teljes 64 dimenziós beágyazott reprezentációs tér 4 különálló, 16 dimenziós altérre bontása után, a tanítási folyamat ezekben az alterekben párhuzamosan történjen meg. A tér felbontásának a célja, hogy a fejek eltérő kapcsolatokat tanuljanak az alterekben, és ezzel a multi-task-szerű megvalósítással egy robusztusabb térbeli reprezentáció kerüljön előállításra. Ennek a részlete a F.3.4. kódban látható. A fejek kimenete a folyamat végén egy lineáris projekcióval összevonásra kerülnek, hogy a kimenet visszanyerje az eredeti 64 dimenziós méretét.

Fontos, hogy a figyelem nem kizárólag a csomópontok alapján kerül meghatározásra, hanem az él-jellemzőkből számított súlyok is befolyásolják a folyamatot. Egy kétrétegű MLP felelős a 4 dimenziós él-jellemzők dimenziójának csökkentéséért egyetlen skalárértékbe. Ezt 16 dimenziós rejtett rétegek és egy `GELU` aktiváció segítségével hajtja végre. Fontos, hogy a súlyok a gráf 4 tanulási fejen párhuzamosan kerülnek felhasználásra, ezért az MLP háló kimenete egy 4 dimenziós vektor, amely minden fejhez külön értéket állít elő.

Támadó csapat felülsúlyozása. A gráf neurális háló kimeneti reprezentációjának felépítése a már említett módon a csapatok szerint súlyozottan történik, a támadó csapat szerepének növelése érdekében. A $\text{BALL_WEIGHT} = 1.5, \text{TEAM_A_WEIGHT} = 1.0, \text{TEAM_B_WEIGHT} = 0.2$ súlyok megválasztásával a támadók térbeli struktúrája és a labda helyzete fog dominálni a kinyert reprezentációban, ahogy ezt a F.3.5. kódrészletben láthatjuk.

Az előállt kimenetet egy projekcióval a $\texttt{GNN_OUT_DIM} = 32$ dimenzióra csökkentettem, majd az eredeti bemeneti vektor mellé fűztem. Így ez a térbeli reprezentációval kiegészített $[B, T, 163]$ alakú mátrix képezi a bemenetet a további lépések számára, valamint GNN belső 64 dimenziós reprezentációja önállóan is felhasználásra kerül a 2.4.4.1. alfejezetben bemutatott Masked Player Prediction pretext feladat során. Az MPP ugyanis a Transformer kihagyásával, csak a gráfreprezentáció alapján végzi el a maszkolt játékosok rekonstrukcióját.

2.4.3.2. Időbeli, temporális tömörítés

A GNN által kibővített $[B, 100, 163]$ dimenziójú bemeneten a Transformer nem lenne képes sikeres tanulást végrehajtani, ugyanis a self-attention mechanizmus számítási és memória igénye a szekvencia hosszának négyzetével arányosan nő. Másodszor, a jelenlegi frameenkénti bontás túl részletes és zajos információt hordoz a modell számára, ugyanis két frame között túl kis idő, 1/25-öd másodperc telik el, aminek köszönhetően a frame-ek közötti mozgás szinte minimális. A frame-szintű felbontás tehát a taktikai elemek szegmentálása szempontjából nem feltétlenül szükséges.

Már a $\texttt{SUBSAMPLE} = 2$ műveletet is ezért vezettem be, ami minden második frame-et tartja meg a szekvenciákból. A GNN kimenete viszont további tömörítésen megy keresztül egy `nn.Conv1d` egydimenziós konvolúciós réteg segítségével. Ez a tömörített reprezentáció tölti be a szavak analógiáját a már 1.3.4. alfejezetben is említett NLP BERT szempontjából. A konvolúció kernel és stride paramétereit mindkét esetben 5-re állítottam, így 5 egymást követő és *nem átfedő* frame-ből készül egy tömörített $[B, 20, 256]$ dimenziós kimeneti token. A szekvencia ennek értelmében 20 darab tokenbe lett összesűrítve, ami a Transformer számára már sokkal erőforrás hatékonyabb lesz, valamint a fentebb említett felbontásból eredő lényegkiemelési probléma is megoldódik. A konvolúciós műveletet egy

GELU aktiváció és egy LayerNorm réteg követi. A normalizáció biztosítja, hogy a Transformer bemenetén továbbra is megfelelő értéktartományú normalizált adatok jelenjenek meg.

Fontos megemlíteni, hogy a kimenetben a feature dimenzió 163-ról 256-ra emelkedik. A Conv1d az out_channels-ben rögzített számú tanulható kimeneti szűrőt alkalmaz minden időablakra. Ezen szűrők segítségével jön létre a konvolúció kimenete, aminek így dimenzionalitása is 256-ra növekszik. Ezt az értéket szándékosan állítottam 256-ra, hogy a konvolúció kimenete közvetlenül átadható legyen a d_model = 256 bemeneti dimenzióval rendelkező Transformer számára. Ezzel megspóroltam a projekciós réteget a két komponens között, ami a dimenzionalitás átalakításáért felelt volna.

2.4.3.3. Szekvenciális kontextus, időbeli figyelem

A tömörítő konvolúciós művelet kimenete a paraméterezésnek köszönhetően egyből átadható a Transformer-encoder számára. Ez felel a 1.3.1. alfejezetben bevezetett figyelmi mechanizmus megvalósításáért. Célja a szekvenciákon belüli időbeli összefüggések megértése és a labdabirtoklás szekvenciák alapján megértett játékreprezentáció kinyerése. A modell 4 figyelmi fejjel implementálja a később kifejtésre kerülő pretext feladatok tanítását is.

Sin–cos pozíciókódolás. A 1.3.1. alfejezetben már említésre került, hogy a Transformer nem tesz különbséget a bemeneti tokenek sorrendje között, emiatt meg kell oldani az időbeli információ beépítését a reprezentációba. A [9] forrásban javasolt szinusz-koszinusz pozíciókódolást alkalmaztam. A kódolás lényege, hogy minden időpillanathoz egy fix, determinisztikus d_model-dimenziós vektort rendel, ahol eltérő frekvenciákkal, a páros pozíciójú koordináták értékei szinusz, a páratlan koordináták koszinusz függvényértékekkel kerülnek meghatározásra a 2.2. képlet alapján.

$$\begin{aligned} PE_{(\mathrm{pos},\,2i)} &= \sin\!\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right), \\ PE_{(\mathrm{pos},\,2i+1)} &= \cos\!\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right). \end{aligned} \tag{2.2}$$

Tanulható [CLS] token. A pozíciókódolt tömörített tokenek szekvenciái elé kerül beépítésre a tanulható [CLS] token, ahogy ez a 1.3.2. alfejezetben tárgyalt BERT alapú implementációban történik. A [CLS] felvételével a tokenek száma $1 + 20 = 21$-re bővül, így a Transformer $[B, 21, 256]$ dimenziójú kötegeket dolgoz fel. A vektor inicializálása $\mathcal{N}(0,0{,}02^2)$ eloszlású, kis szórású értékekkel történik meg, majd a tanítási folyamat során a Transformer önfigyelmi mechanizmusa a rétegeken áthaladó szekvenciák tokenjeinek globális reprezentációját aggregálja a szóban forgó 256 dimenziós [CLS] vektorba. Ezt a globális token-reprezentációt több másik komponens és pretext feladat is alkalmazza később, valamint az embedding vektorok előállításához ezek kerülnek felhasználásra.

Encoder-paraméterek. A beépített Transformer-encodert n_layers = 4 réteggel, és rétegenként n_heads = 4 figyelmi fejjel (attention head) láttam el. Az eredeti BERT implementáció 12 réteggel dolgozik, de ezt indokolja a feldolgozott szöveg esetén használt hosszabb szekvenciahossz és nagyobb tokenszám. A 2.4.3.2. alfejezetben ismertetett tracking adatok jellemző tulajdonságai miatt döntöttem a tömörítés és kisebb tokenszám alkalmazása mellett, és pontosan ez jelenti a megkötést a modell rétegeinek beállítása során is. A túltanulás elkerülése érdekében sekélyebb modellt implementáltam, emiatt döntöttem a 4 réteg mellett. A feed-forward előrecsatolt részre vonatkozó paramétert

dim_feedforward = 512 = 2 · d_model szerint állítottam be. Ez negyede az eredeti BERT cikkben javasolt értéknek, itt szintén a túltanulás megelőzése érdekében csökkentettem a paraméter értékét. Végül a regularizáció erősítésének érdekében a beépített dropout módszert alkalmaztam, ami a `dropout` = 0,1 paraméter beállításával 0,1 valószínűséggel hajtódik végre.

Pre-norm elrendezés. A `norm_first = True` kapcsoló alkalmazása engedélyezi a Transformer *pre-norm* módban történő futását. Ez lehetővé teszi a `LayerNorm` réteg self-attention és a feed-forward blokkok *előtti* végrehajtását. Ellenkező esetben, alapértelmezés szerint ez fordított sorrendben hajtódna végre, mint ahogy a [9] forrásban is láthatjuk. A pre-norm alkalmazásának előnye, hogy a Transformer gradiens viszonylag egyenletessé válik a tanítás kezdeti fázisában, valamint arányos és hasonló nagyságrendű gradiens frissítések történnek a hálóban, emiatt stabilabb tanítást tesz lehetővé [22]. A pre-norm alkalmazása mellett a 2.4.6.1. alfejezetben bemutatott ütemezőben egy rövid bemelegítési (warm-up) szakasz szolgál hasonló stabilizáló céllal, ami tovább biztosítja, hogy a kezdeti lépésekben ne váljon instabillá a gradiens.

Padding-frame-ek kezelése. Ahogy már a korábbiakban is említésre került, a modellbemenet tartalmazhat érvényességet jelző padding maszkokat. Ezt az augmentáció időbeli eltolása aktívan módosíthatja is a futás során. A korábban használt frame-szintű `padding_mask` viszont nem használható fel közvetlenül a tömörített 20 tokenen, az elvégzett tömörítések miatt. Ennek a problémának a feloldására egy `max_pool1d` max pooling alapú leképezést alkalmaztam, ami gyakorlatilag az eredeti maszkot véve alulmintavételezést végez, és megállapítja egy adott token tömörítés utáni érvényességét az eredeti magasabb felbontású frame-ekből kiindulva. Az implementáció akkor utasít el egy tokent, ha a hozzá tartozó összes frame érvénytelen volt, így ezzel a megközelítéssel a lehető legtöbb token felhasználásra kerül.

A tokenszintű maszkok első pozíciójába kerül a `[CLS]` token, ami a tokenek egységes reprezentációjáért felel. Ennek mindig érvényesnek kell lennie a maszkban, ugyanis a tanítás alatt végig aktívnak kell lennie. Az így kapott 21 hosszú logikai értékeket tartalmazó maszk átadásra kerül a Transformer-encoder részére a `src_key_padding_mask` paraméterén keresztül, és a tanulási folyamat során minden `True`, azaz maszkolt értéket kihagy, így csak érvényes tokenek befolyásolják a reprezentáció kialakulását.

A Transformer-encoder kimenete kötegenként egy [B, 21, 256] alakú tenzor, második pozíciójában a szekvenciához tartozó 21 token található, melyek közül legelső az adott szekvenciához tartozó aggregált `[CLS]` tokenreprezentáció. Ezek a tokenek, ahogy a korábbiakban is látható volt, 256 dimenziós vektorként vannak reprezentálva a mátrixban. A kimeneti mátrix szolgáltatja a feldolgozás során következő Phase Pooling művelet bemenetét, emellett közvetlenül a Transformer kimenetre épülnek a további pretext fejek feladatai is. Az MLM és Velocity Head feladatok a 20 token alapján végez rekonstrukciót, a CZP fej pedig a `[CLS]` tokenre épített osztályozási feladatot implementál.

2.4.3.4. Phase Pooling

A Transformer-encoder kimeneti [B, 21, 256] tenzorát további felhasználáshoz egy globális embedding-vektorrá kell összevonni, ahogy ezt a klasszikus BERT implementáció is teszi [10]. Ebben a reprezentációs embedding vektor szerepét a fentebb bemutatott `[CLS]` token ölti be, a maradék 20 snippet-token pedig nem kerül felhasználásra a későbbi feladatok során. Ellenben a mean pooling alkalmazása esetén a 20 kimeneti token vektor átlaga alapján történik a reprezentáció kinyerése, `[CLS]` token használata nélkül.

A Phase Pooling megközelítésem megpróbálja felhasználni mind a 20 tokent és a [CLS] tokent is egy együttes reprezentáció kialakításához. Azért fázis alapú megközelítésnek neveztem el, ugyanis a szekvenciák első, középső és utolsó harmada alapján történik az átlagolás végrehajtása. Ezekben a különböző fázisokban feltételezésem szerint a taktikák dinamikája jellemzően különböző lesz. A Phase Pooling módszerem a CLS token-nel együtt átlagolja a szekvenciát az általam meghatározott három fázis mentén és az így elkészülő 4 vektort illeszti össze.

A négy vektor egyetlen $4 \cdot 256 = 1024$ dimenziós reprezentációt állít elő, amit a későbbiekben egy kétrétegű projekciós háló csökkent vissza a 128 dimenziós térbe. A [CLS] továbbra is a tanult, szekvenciánkénti globális kontextust jelenti, a három fázisátlag pedig az időbeli felbontást erősíti.

A fázishatárok a tokenek harmadolásával kerültek meghatározásra. Az $S = 20$ tokenszám esetén ez a korai fázis 6, a középső és a késői pedig 7–7 hosszú fázisokat jelent. Az átlagolás során a 2.4.3.3. alfejezetben tárgyalt snippet-szintű padding-maszkot is figyelembe vettem. A Phase Pooling-hoz tartozó implementáció részlete a F.3.6. kódrészletben olvasható.

2.4.3.5. Bottleneck réteg bevezetése

Az 1024 dimenziós kimeneti vektort egy nemlineáris bottleneck háló alakítja a 128 dimenziós kimeneti térbe. Azért döntöttem a 128-as dimenzió paraméter mellett, hogy a modell kimenetén ténylegesen különböző taktikai elemek jelenjenek meg. Nagyobb dimenzió alkalmazásával a szegmentálás során szemmel nehezen észlelhető, de a modell által megtanult különbségek is megjelenhetnének a szekvenciák között. A dimenziócsökkentés során először egy `Linear(1024, 256)`, majd `GELU` aktiváció és `LayerNorm` normalizáció követően egy újabb `Linear(256, 128)` réteg fejti ki hatását. A dimenzionalitás több tanulható rétegben és lépésben történő csökkentése képes a vektor négy forrása ([CLS] és három fázis) közötti együttes interakciókat is tanulni. A záró lépés pedig a 128 dimenziós kimeneten történő L2-normalizáció elvégzése.

2.4.4. Önfelügyelt multi-task tanítási feladatok

A modell a 1.1. szakaszban bemutatott önfelügyelt reprezentációs tanulást megvalósítva 5 párhuzamosan végzett pretext feladat segítségével alakítja ki a kimeneti embedding teret. Ahogy ezt a 1.2.3. fejezetben is kifejtettem, a cél az, hogy a pretext feladatok együttesen egy robusztusabb reprezentációt állítsanak elő [14]. Fontos, hogy ezek mindegyike egy fontos aspektusát ragadja meg a szekvenciák szerkezetének, hogy a tanítás sikeresen végbemenjen. Ahogy már korábban említettem, a jól definiált pretext feladatok befolyásolják az embedding tér szerkezetét és a feladat által beépített tanulási cél jellege alapján tudnak többlet tudást beemelni a kialakuló reprezentációba. A következő alfejezetben ezeknek a pretext feladatoknak a részletes bemutatására kerül sor.

2.4.4.1. Masked Player Prediction

A Masked Player Prediction (MPP) feladat a térbeli struktúrák és a geometria megértéséért felel. Ehhez a 2.4.3.1. alfejezetben bemutatott attention-alapú gráf-neurális hálózatot használja. Fontos hangsúlyozni, hogy az architektúra Transformer ágától elkülönítve végzi el a kimaszkolt játékosok rekonstrukcióját. A tanítási folyamat során végrehajtott játékosmaszkolás kötegenként kiválaszt egy adott játékost, mind a támadó, mind a védő csapattagok közül és az ezekhez tartozó jellemző értékeket kinullázza az adott szekvenciákban. A maszkolás a normalizáló függvényem `x_norm` és `x_raw` kimenetén egyszerre megy végbe, ugyanis a gráf alapú figyelmi blokk mindkettőt bemenetként alkalmazza.

A modell a maszkolt bemenethez egy dedikált, csak a GNN-t használó forward művelettel állítja elő a csomópontokhoz tartozó teljes 64 dimenziós reprezentációt. A maszkolt játékosokhoz tartozó reprezentációkat pedig egy `torch.gather` művelet küldi a `Linear-GELU-LayerNorm-Linear` rétegekből felépített MPP fej irányába. A fej a 64 dimenziós belső reprezentációból egy 2 dimenziós leképezést hoz létre, ami a kitakart játékosok labdához viszonyított térbeli pozícióját próbálja rekonstruálni az MPP veszteségfüggvényének segítségével. A feladat sikeres végrehajtása tovább erősíti a GNN-ben implementált gráf-attention mechanizmus hatását, annak érdekében, hogy a modell a geometria mellett még hatékonyabban tanulja a játékosok közötti térbeli függőségeket is. A szekvenciákon belül elvégzett predikció frame-enként történik, így az MPP kimenete egy $[B, 100, 2]$ dimenziójú mátrix lesz.

A feladat során alkalmazott veszteségfüggvény a 2.4.5.1. alfejezetben kerül kifejtésre, ugyanakkor itt is fontos megjegyezni, hogy veszteség csak az érvényes frame-eken kerül kiszámításra. Valamint itt is megjelenik az aszimmetrikus súlyozás a támadó taktikák kiemelésének céljából, így a támadó csapat játékosának rekonstrukciós hibája az összesített MPP veszteségben jobban érvényesül a védő hibájával szemben.

2.4.4.2. Masked Language Modelling

A Masked Language Modelling (MLM) pretext feladat a 1.3.2 és 1.3.4. alfejezetekben bemutatott BERT alapú megközelítésben NLP során alkalmazott analógiát valósítja meg a kosárlabda kontextusában. A feladata a szekvenciákon belül kitakart frame-ek teljes rekonstrukciója, a Transformer-encoder segítségével. Az előző pretext feladattól eltérően ez a fej az architektúra Transformert tartalmazó ágából folytatódik. Az encoder bidirekcionális kontextussal rendelkezik a frame-ekről, ezt kihasználva az MLM feladat kikényszeríti, hogy a tokenek reprezentációjába ez a kétirányú időbeli információ is beépítésre kerüljön.

A maszkolás implementációja szintén követi az említett fejezetekben bemutatott BERT analógiát. Az érvényes frame-ek közül véletlenszerűen kiválasztott `MLM_MASK_PROB` = 15%-a kerül rekonstrukcióra. Az ezeken a frame-eken a maszkolást pedig a klasszikus BERT-féle [10] 80-10-10 stratégia alapján valósítottam meg, ami a rekonstrukcióra kijelölt frame-ek maszkolási stratégiájának megosztásán alapul. Az esetek 80%-ában a maszk egy tanulható, 131 dimenziós `mask_token`-t jelent, amit az `nn.Parameter` segítségével implementáltam. A háló a tanulási folyamat során ezeket a paramétereket célfüggvény szerint módosítja. 10%-ban a frame tartalmát a köteg egy másik frame-jének értékeivel helyettesítettem, a maradék 10%-ban pedig a frame érintetlen marad.

Fontos megemlíteni, hogy a bevezetett konvolúciós tömörítés miatt a rekonstrukcióhoz egy ezzel ellentétes irányú, frame-szintű dekódolás kellett implementálnom, hogy a rekonstrukció képes legyen frame szintű rekonstrukciót elvégezni. A `mlm_head`-et az MPP-hez hasonlóan egy (`Linear-GELU-LayerNorm-Linear` rétegekkel rendelkező) MLP valósít meg, valamint az MLM veszteségfüggvényében is megjelenik a korábban említett aszimmetrikus súlyozás.

2.4.4.3. Velocity Head

A Velocity Head az egyik általam bevezetett új megközelítés a pretext feladatok között, ami egy külön pretext fej segítségével végzi el a maszkolt frame-eken lévő játékosok sebességvektorainak rekonstrukcióját, ezzel kényszerítve a Transformert a mozgásdinamika és irányváltások kontextusának beépítésére a reprezentációba. Az eddig bemutatott feladatok inkább a játékosok pozíciójára fókuszáltak. Az itt definiált sebesség alapú feladattal tisztán dinamikai struktúrát szerettem volna reprezentálni. A különböző taktikák más tempóval

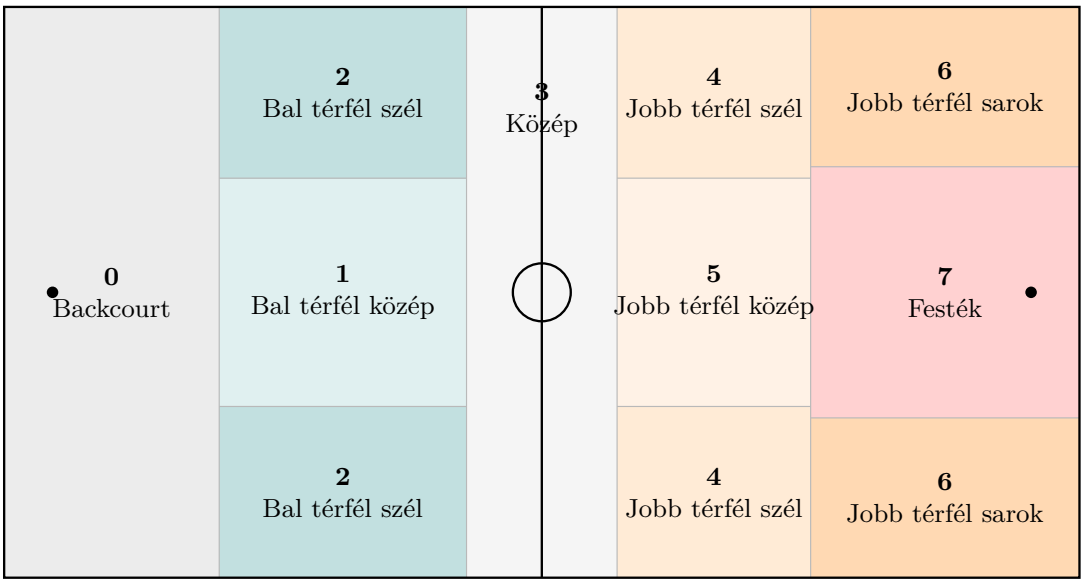
kerülnek végrehajtásra a játék során, ezért a játékosok sebességéről szerzett ismeretek fontos kontextus lesz a modell számára.

A Velocity Head az MLM-nél bemutatott maszkolási stratégiát közösen alkalmazza a szekvenciákon, így a két feladat ugyanazon frame-kijelölésre épül, valamint az eddigiekhez hasonlóan a fej architektúrája is megegyezik az előzőkben alkalmazott felépítésekkel. A kimenete viszont eltérő, ugyanis tokenenként hozza létre a rekonstrukciót, így a tokenen belüli 5 szomszédos frame sebesség-vektorait határozza meg ($[B, T, 5, 20]$). Ahogy eddig is, a veszteségfüggvényben szintén implementálásra került az aszimmetrikusan történő támadó súlyozás.

2.4.4.4. Court Zone Prediction

A Court Zone Prediction (CZP) célja a kosárlabdában ismert taktikai zónák térbeli ismeretének kikényszerítése. Az eddigi térbeli modellelemek a pályakoordináták segítségével szereztek információt a térbeli kontextusról. A CZP ezzel szemben a térbeli információit a domain ismeret alapján meghatározott pálya zónák alapján tanulja meg.

Fontos különbség, hogy az eddigi feladatokkal szemben ez a fej globálisan, szekvencia szinten (nem frame-szinten) implementál egy osztályozási feladatot. A működése során meghatározásra kerül, hogy a 2.5. ábrán látható zónák közül melyikben tartózkodott legtöbbet a labda a szekvencia során. A cél, hogy a CZP fej a Transformer kimenetének logit rétegén a `[CLS]` token alapján meghatározza a szekvenciához tartozó helyes zónát. A CZP veszteségfüggvénye pedig implicit módon kényszeríti a modellt a zónákra vonatkozó geometriai jellemzők beemelésére a megfigyelt kontextusba. A pálya zónáit a normalizált koordináták szerint osztottam fel 8 taktikai régióra. A kategóriák tartalmazzák a saját térfelet, a támadó térfél két 45 fokos oldalát, a sarok pozíciókat, a palánkhoz közeli festék zónát, a pálya közepén végigfutó sávot. A szekvencia-szintű zóna-címke előállítása két lépésben történik. Először frame-szinten kerül megállapításra, hogy a labda melyik zónában volt a legtöbbet, egy one-hot aggregálás segítségével. Az így előálló $\{0, 1, \ldots, 7\}$ értékű címkék szekvenciánként kerülnek átadásra, amin a CZP megvalósításáért felelős háló `cross_entropy` keresztentrópia veszteségfüggvénnyel tanul és hajtja végre a predikciót. A predikciót a `[CLS]` tokenjén alapuló `czp_head` végzi, amely egy `Linear, GELU, LayerNorm, Linear` rétegekkel rendelkező MLP háló. A CZP fejben alkalmazott hálóarchitektúra az előző fejezetekben bemutatott fejekhez hasonló felépítésű, paramétereiben viszont eltérő. A rejtett dimenziók száma szándékosan kisebb (64), mint az MLM/Velocity fejekben (256), mert a CZP egyszerűbb 8-osztályos klasszifikációs feladatot valósít meg. A CZP-fej emellett a normalizációs függvény tiszta, nem maszkolt kimenetét kapja, így a globális zóna-predikció tanulása a teljes, érintetlen szekvenciákon mehet végbe. Az aszimmetrikus támadó súlyozás szintén beépítésre került. A CZP megoldással a célom a domain ismereteken alapuló zónarendszer beépítése volt a modell által megtanult reprezentációba. Itt a 2.5. ábrán látható részletesebb zónafelosztást alkalmaztam, a 2.3.7.4. alfejezetben bemutatott helyett. A CZP zónái a jobb irány szerint egységesített támadási irány figyelembevételével és a taktikák jobb és bal oldali szimmetriájára alapozva kerültek kialakításra.



2.5. ábra. A CZP zónafelosztása.

2.4.4.5. SimCLR kontrasztív tanítás

A korábban tárgyalt négy pretext feladat (2.4.4.1, 2.4.4.2, 2.4.4.3, 2.4.4.4) rekonstrukciós eljárásokon alapultak, valamint a térbeli dinamika és a kontextus tanulására irányultak. Ugyanakkor ezek a módszerek csak implicit módon, a kontextus módosításával érik el a reprezentációs embedding tér struktúrájának szegmentálódását. A [13]-ban bemutatott SimCLR kontrasztív tanulási módszer célja, hogy közvetlenül az embedding-tér geometriáját módosítva kikényszerítse a taktikailag hasonló szekvenciák közeledését és a különbözőek távolodását az embedding-térben.

A kontrasztív tanulás alapelve, hogy minden eredeti szekvenciából két különböző, augmentált nézetet hozunk létre, mint pozitív pár. A tanulási cél az, hogy a pozitív párok között az embedding-térben vonzó erő alakuljon ki, miközben az adott köteg összes többi szekvenciájának nézeteitől, azaz a negatív pároktól eltávolodjanak. A nézetek előállítását a 2.4.2.4. alfejezetben bemutatott `make_simclr_view` függvénnyel hajtottam végre, ami a 2.4.2. alfejezetben bemutatott augmentációs eszközöket alkalmazza kétszer a feldolgozás alatt lévő köteg szekvenciáin.

A [13] alapján a SimCLR egyik kulcslépése, hogy az eredeti reprezentációs téret (h), egy nemlineáris projekció segítségével transzformáljuk ($z = g(h)$). A pretext feladatok továbbra is az (h) reprezentációs teret alkalmazzák, és az új kontrasztív tér csak a veszteség kiszámításához szükséges a SimCLR során. A `simclr_proj` egy MLP segítségével végez nemlineáris projekciót 128 dimenziós bemenet 64 dimenzióra csökkentése érdekében. Fontos lépése a kimeneten végrehajtott L2-normalizáció, ami a z kontrasztív teret egy egységgömb felületére transzformálja, ezáltal a páronként vett koszinusz hasonlóság közvetlenül értelmezhető a $z_1 \cdot z_2$ vektorok skalárszorzataként.

A SimCLR által alkalmazott *Normalized Temperature-scaled Cross Entropy* (NT-Xent) kontrasztív veszteségfüggvény a 2.4.5.3. alfejezetben kerül részletes bemutatásra. Működésének lényege, hogy a két nézet (z_1, z_2) kimenete egyetlen $2B$ (B a kötegméretet jelöli) méretű halmazba kerüljön összevonásra. Itt számításra kerül a $(2B, 2B)$ halmaz elemeinek páronkénti koszinusz hasonlósági mátrixa, amin a `SIMCLR_TEMPERATURE` = 0,1 SimCLR hőmérsékleti paraméterrel végrehajtottam egy skálázást a kontrasztív hatás módosíthatóságának érdekében. Az átló, azaz az elemek önmagukkal vett hasonlóságának $-\infty$-re történő frissítését követően a mátrix átló-értékei a későbbi softmax-

normalizációban nem vesznek részt. Egy bevezetett indexelés jelöli a pozitív páros pozícióit, amik egyben az augmentált pozitív párokat jelentik a mátrixban. A veszteségfüggvény célja a softmax alapú keresztentrópiával kiszámított normalizált hasonlóság maximalizálása a pozitív páron a $2B - 1$ negatív párok hasonlóságához képest. A SimCLR fej és veszteségfüggvényének implementációját a F.3.7. kódrészlet szemlélteti.

Az alacsony hőmérséklet ($T = 0{,}1$) értékének beállítása saját tudatos tervezési döntésem volt, ugyanis ennek az értéknek a növelése gyengíti a kontrasztív hatás mértékét az embedding-térben, így a kialakuló taktikai klaszterek elkülönülése nehezebben történne meg. A paraméter alacsonyan tartása viszont felerősíti a kontrasztív hatást, így a háló erősebben végzi a pozitív párok távolságának minimalizálását.

2.4.5. Veszteségfüggvények implementációja

A modell összetett veszteségfüggvényét a 2.4.4. alszakaszban bemutatott pretext feladatok hibájának súlyozott lineáris kombinációjaként állítottam elő az implementációmban a 2.3. képlet alapján. A legnagyobb súllyal (1,0) az MPP feladat által szolgáltatott térbeli struktúrát részesítettem. A frame-szintű MLM és Velocity Head rekonstrukció kisebb 0,4-es súllyal került beépítésre. A CZP feladat csak kiegészítő szerepet tölt be a teljes reprezentáció kialakításában 0,2-es súllyal, valamint a SimCLR regularizációs kontrasztív szerepet betöltve csak 0,1-es súllyal vesz részt a tanítási folyamatban. Valamint a kontrasztív tanulás paraméterezésének célja, hogy a folyamat a lehető legjobban összehúzza a pozitív párokat a térben, de ennek hatása ne nyomja el a más modellkomponensek által tanult együttes reprezentációt.

$$\mathcal{L}_{\text{total}} = 1{,}0 \cdot \mathcal{L}_{\text{MPP}} + 0{,}4 \cdot \mathcal{L}_{\text{MLM}} + 0{,}4 \cdot \mathcal{L}_{\text{VEL}} + 0{,}2 \cdot \mathcal{L}_{\text{CZP}} + 0{,}1 \cdot \mathcal{L}_{\text{SimCLR}} \tag{2.3}$$

2.4.5.1. MPP, MLM és Velocity loss

Az MPP, MLM és Velocity Head pretext feladatokban közös, hogy folytonos regressziós feladatot valósítanak meg, mivel a maszkolt értékeket skalárokkal prediktálják. Mindhárom esetében Smooth L1 veszteségfüggvényt alkalmaztam [23], ami a kis hibák esetén négyzetes MSE-szerű viselkedést, nagyobb hibák előfordulása során pedig lineáris büntetést használ. Ennek köszönhetően robusztusabban kezeli az értékekben fennmaradó kiugró értékeket, viszont a kis hibákat nem bünteti alul.

Az MPP pretext feladat a kimaszkolt játékosok rekonstrukciós predikcióinak optimalizálását szintén egy Smooth L1 veszteség végzi. Ugyanakkor fontos megemlíteni, hogy a két csapat játékosaira külön veszteség kerül kiszámításra, megvalósítva a csapatonkénti aszimmetrikus súlyozás logikáját, összhangban a támadók fókuszával. Ez a 2.4. képletben olvasható.

$$\mathcal{L}_{\text{MPP}} = \mathcal{L}_{\text{MPP},A} + \texttt{DEFENSIVE_MPP_SCALE} \cdot \mathcal{L}_{\text{MPP},B}, \qquad \texttt{DEFENSIVE_MPP_SCALE} = 0{,}3. \tag{2.4}$$

2.4.5.2. CZP loss

A CZP során definiált 8-osztályos pálya-zóna alapú osztályozás egy softmax-alapú keresztentrópia veszteségfüggvénnyel került implementálásra. A zóna eloszlását a tanítást megelőző vizsgálattal ellenőriztem. Ez megmutatta, hogy az osztályok eloszlása nem indokolja ezeknek a direkt módon történő súlyozását, így a keresztentrópiát egyenlő súlyozással alkalmaztam.

2.4.5.3. SimCLR NT-Xent loss

A SimCLR kontrasztív tanításhoz használt NT-Xent veszteség a 2.4.4.5. alszakaszban már részletesen bemutatásra került, mivel a módszer értelmezése a veszteségfüggvény működésének magyarázatát is indokolja. Fontos azonban kiemelni, hogy gyakorlatilag az NT-Xent veszteség egy keresztentrópia megvalósítását jelenti a $(2B, 2B)$ kontrasztív térbeli vektorokból felépített hasonlósági mátrixban. Az implementációt a F.3.7. kódrészlet bővebben részletezi.

2.4.6. Tanítási folyamat részletei

2.4.6.1. Optimalizáló és tanulási ráta ütemező

A tanítási folyamat során egy AdamW optimalizáló biztosítja az optimalizáció során a gradiens stabilitását. Kezdeti paramétereit `LEARNING_RATE` $= 1 \times 10^{-4}$ tanulási rátára és `WEIGHT_DECAY` $= 1 \times 10^{-4}$ állítottam be. A weight decay regularizációt valósít meg hibafüggvényhez adott büntetéssel, ami megakadályozza a hibásan túl nagy súlyok kialakulását. Az AdamW az Adam optimalizáló a múltbeli gradiensek első és második momentuma alapján meghatározott tanulási rátájájának módszerét egészíti ki, a gradiens-számítástól elkülönített weight decay meghatározásával. Ez a megközelítés további regularizációs hatással bír, ugyanis a weight decay értéktartománya független marad a kiszámított tanulási rátától, így ez hatékonyabban fejtheti ki hatását [24].

A tanulási ütemezését két fázisra bontottam, ami a tanulási ráta módosítására van hatással. Kezdetben egy lineáris bemelegítési fázis megy végbe, ahol az első `WARMUP_EPOCHS` $= 10$ epochban a tanulási ráta lineárisan nő. Célja, hogy a tanítás kezdetén a háló súlyait fokozatosan stabilizálja, elkerülve a gradiens instabilitását és robbanását, valamint az AdamW momentum-becslését is segíti.

A bemelegítést a koszinusz alapú csökkenést megvalósító, koszinusz-decay [25] fázis követi. Itt a tanulási ráta koszinusz alapú, folyamatos csökkenése kerül végrehajtásra a `MAX_EPOCHS` $= 150$ epoch eléréséig. A lassítás dinamikusan történik, a fázis kezdetén lassabb, a középső szakaszban felgyorsul, majd a tanítás végén újra lassuló ütemben csökken a tanulási ráta értéke. A csökkenés kontrollálása érdekében a tanulási ráta a kezdeti érték legkevesebb 5%-át érheti el.

2.4.6.2. Mixed Precision alkalmazása

A tanulási folyamat erőforrás hatékonyságának biztosítása érdekében a PyTorch *Automatic Mixed Precision* (AMP) [26] keretrendszerét használtam. Ez az AMP a futás alatt automatikusan dönt arról, hogy az erőforrás igényes számítási műveletek `float16` vagy `float32` számábrázolási pontossággal kerüljenek végrehajtásra. A számításigényes tenzorműveletek alacsonyabb, amíg a numerikusan érzékeny műveletek továbbra is a magasabb pontosságú reprezentációban kerülnek számításra. A végső gradiens-frissítések szintén `float32`-ben tárolódnak, így a Mixed Precision a tanulási stabilitást nem befolyásolja.

A `float16` számábrázolás használatának veszélyét a kis abszolút értékű gradiensek alulcsordulása jelenti. Ennek megoldására a `GradScaler` került beépítésre, ami veszteségértékeket a backward folyamat előtt dinamikus skálázással felszorozza, emiatt a backward alkalmazott gradiens értékek is a `float16` tartományában maradnak. Az optimalizáló lépés előtt a felskálázás visszaosztásra kerül, így a gradiensértékek az eredeti tartományukban kerülnek felhasználásra. Ha mégis alulcsordulás következne be, akkor az adott számítási lépés átugrása után a `GradScaler` skálázási faktora automatikusan csökkentésre kerül.

2.4.6.3. Validáció és Early Stopping implementációja

A validációs ciklus minden epoch után lefut a tanítóadatoktól elkülönített validációs halmazon. A különböző validációs értékek összehasonlíthatósága érdekében a validációs kiértékelés során nem hajtottam végre a nem determinisztikus dropout-ot és augmentációs műveleteket, valamint az MPP során a maszkolásra kiválasztott játékosok kijelölése sem véletlenszerűen történik.

A validációs metrikák az egységes veszteségfüggvény logikája alapján kerülnek kiszámításra külön a validációs halmazra, ami a 2.5. képlet tartalmaz. Ebbe a SimCLR-veszteséget szándékosan nem építettem be, ugyanis a kontrasztív veszteség a véletlenszerű augmentációra épül, valamint nem közvetlenül a modell által tanult rekonstrukcióról adna visszajelzést, hanem az embedding-tér klaszterezhetőségéről a validációs lépések során.

$$\mathcal{L}_{\text{val_pretext}} = 1{,}0 \cdot \mathcal{L}_{\text{MPP}} + 0{,}4 \cdot \mathcal{L}_{\text{MLM}} + 0{,}4 \cdot \mathcal{L}_{\text{VEL}} + 0{,}2 \cdot \mathcal{L}_{\text{CZP}}. \tag{2.5}$$

Az `Early Stopping` korai megállás célja, hogy a metrikákat monitorozva fenntartsa a folyamatos javulást a validációs halmazon, és megállítsa folyamatot a túltanulás bekövetkezése előtt. Az implementált szabály szerint a validációs értékek javulásának legalább `min_delta` $= 10^{-4}$ értékűnek kell lennie, azaz ennyivel kell csökkenjenek az eddigi legjobb metrikához képest. Ha `patience` $= 12$ beépített türelmi fázis alatt nem következik be további javulás a legjobb értékéhez képest, akkor a tanítás leáll. A folyamat az aktuális legjobb epoch nyilvántartása segítségével, minden javulás esetén felépíti a modellsúlyokból álló (`best_nba_bert_23.pt`) fájlt, így a tanítási folyamat végén ezek a súlyok jelentik a legjobb állapotot.

2.4.6.4. Silhouette Próba

Az eddigi validációs metrika a multi-head pretext feladatok túltanulását és rekonstrukciós minőségét szemlélteti, de a tényleges szegmentálásról és szeparálhatóságról nem nyújt információt. A Silhouette próba implementációjával az utóbbit folyamatosan monitorozni lehet. Ez 5 epochonként szolgáltat információt, ahol visszaméri a klaszterezhetőséget egy mini-batch alapú, a validációs adatok közül véletlenszerűen kiválasztott 2000 mintás embedding halmazon végrehajtott K-Means klaszterezéssel, aminek eredményén meghatározásra kerül a koszinusz távolság alapú Silhouette-score [27] metrika. A koszinusz metrika az L2-normált embedding vektortéren ekvivalens az euklidészi távolsággal, ugyanakkor stabilabb numerikus számítást eredményez.

A Silhouette-score [27] a klaszterezés minőségére bevezetett metrika, ami a klaszteren belüli és közötti távolságokon alapszik.A 0 körüli érték klaszter határon álló összemosódó klasztereket, a negatív értéke pedig azt jelzi, hogy a minták átlagosan közelebb vannak egy szomszédos klaszterhez, mint a sajátjukhoz.

A Silhouette Próba módszerrel a céljom az volt, hogy monitorozzam a tanulási folyamat alatt a Silhouette értékek változását. Az elvárt viselkedés ezeknek az értékeknek a folyamatos növekedése volt a struktúrálódó embedding-térben.

2.4.7. Embeddingek kinyerése

A sikeres tanítási folyamat végrehajtása után az `extract_embeddings_full()` metódus kinyeri a végső kimeneti tér embedding 128 dimenziós L2 normált vektorait. A folyamat ellátja a vektorokat az ezekhez tartozó metaadatokkal, mint a mérkőzés azonosítója, a labdabirtoklás határának megállapításához használt PBP eseménytípusok, valamint információt szolgáltat, hogy mely ablakok tartoznak egy adott labdabirtokláshoz.

2.5. Riportáló weblap felület bemutatása

Az eddig bemutatott modell architektúra kimenete nehezen értelmezhető egyszerű felhasználók és kosárlabda elemzők számára. A 3. fejezetben bemutatott klaszterező kiértékelési folyamat ugyan előállít különböző statisztikákat, ábrákat és videókat az eredményei alapján, de ezek megjelenítését lehetővé kellett tennem a felhasználók számára is. Ennek érdekében egy webes alkalmazást készítettem, amely a modellfutások klaszterezési folyamatának kimenetét interaktívan és felhasználóbarát módon jeleníti meg.

Ebben a szakaszban a webes felület bemutatására kerül sor, a klaszterezési folyamat, amely ehhez bemeneti forrást szolgáltat a fentebb említett 3. fejezetben kerül bemutatásra.

2.5.1. Tervezési döntések és célok

A felület tervezése során számos nézőpontot mérlegeltem, amelyek befolyásolták az alkalmazott keretrendszer kiválasztását és a rendszer tervezési döntéseit is.

A modell Python környezetben fut, kimenetei NumPy-tömbök, valamint JSON formátumban tárolt metaadatok és statisztikák. A webes felületet emiatt szerettem volna ugyanebben a Python környezetben implementálni, a kimenetek szerializációs lépéseinek elkerülése, valamint az egységes rendszer érdekében.

Másrészről fontosnak tartottam, hogy a weboldal felépítése kövesse a rendszertervezés alapelveit, azaz könnyen bővíthető legyen a modell és a kiértékelő folyamatok változtatása esetén is. A kiértékelő a kimeneteit egy mappastruktúrába rendezi. Ha a kiértékelési módszertan vagy a felépített mappastruktúra változik, akkor ezt a weblap megjelenítési felületének módosítása nélkül lehessen megvalósítani.

Fontos követelmény az elkészült rendszer tesztelhetősége is, a tanítási folyamat lefuttatása nélkül, ami rendkívül sok időt venne igénybe.

Az itt felsorolt megközelítéseket figyelembe véve a *Streamlit* [28] keretrendszert választottam az implementáció elvégzéséhez.

A Streamlit Python függvények segítségével deklaratívan felépített UI (felhasználói felület) rendszerek kialakítását támogatja. Fontos, hogy közvetlenül képes feldolgozni a NumPy és Pandas adat-objektumait, így közvetlen alkalmazható az eredmények megjelenítése érdekében. Pontosan emiatt rendkívül elterjedt eszköz adattudományi és mélytanulási projektek riportálásához.

A webes rendszeremet négy Python-modulból, valamint egy bash indítószkriptből építettem fel a 2.2. táblázat szerint.

Modul	Felelősség
`app.py`	Streamlit felhasználói felület, navigáció, CSS és nézet-renderelés
`run_loader.py`	Kiértékelő kimeneteinek egységes betöltése
`mock_data.py`	Szintetikus tesztadatok generálása
`test_webapp.py`	Tesztelés

2.2. táblázat. A webes alkalmazás moduljai és felelősségi körük.

2.5.2. Adatbetöltő réteg és kompatibilitási absztrakció

A `run_loader.py` a UI megjelenítő és a kiértékelő folyamat közé épített réteg. Célja a fentebb említett kompatibilitás és változtathatóság garantálása. Két Python `dataclass`-t tartalmaz, a `ClusteringView` egy klaszterezési módszer kimeneti adatait tartalmazza, a `RunView` pedig a teljes futtatás összes adatát gyűjti össze, amelybe az összes konfigurációs adat, statisztika, generált kép és videó beletartozik.

Ennek következtében különböző klaszterezési módszerek implementálása válik lehetővé, így a jelenlegi K-Means alapú módszer (3. fejezet) a jövőben bővíthető más algoritmusokkal is. A kiértékelő a `clustering` mappájába menti kimeneteit az algoritmusnak megfelelő almappába. A webes rendszer a `detect_many_dir()` függvénnyel ezt a mappát felismeri, és egy egységes interfészen keresztül szolgálja ki a megjelenítésért felelős modult. A K-Means alapú kiértékelő K-Means és hierarchikus kimenete közötti navigáció, pontosan emiatt az implementáció miatt, egyszerűen váltható a megjelenő nézetek során az összehasonlítás érdekében. Ugyanígy jelennének meg más algoritmussal előálló kiértékelési eredmények is.

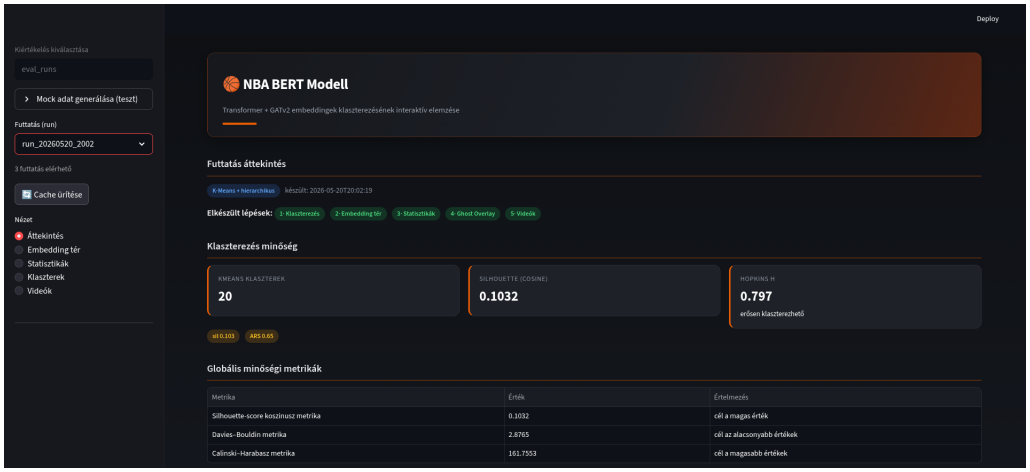
A nagy méretű adatok betöltése késleltetetten történik, csak amikor egy nézet ténylegesen hivatkozik ezekre, valamint a 2.4.1.1. alfejezetben említett NumPy memory-mapped módjában, a hatékonyság garantálása érdekében.

2.5.3. Felhasználói felület bemutatása

A Streamlit-ben a megjelenítés stílusát CSS-ben lehet testreszabni. Itt adtam meg a kinézetre vonatkozó paramétereket, azaz a sötét hátteret, a kosárlabdához illő narancsos színt és a metrikákat tartalmazó elemeket és a színes státuszjelzőket.

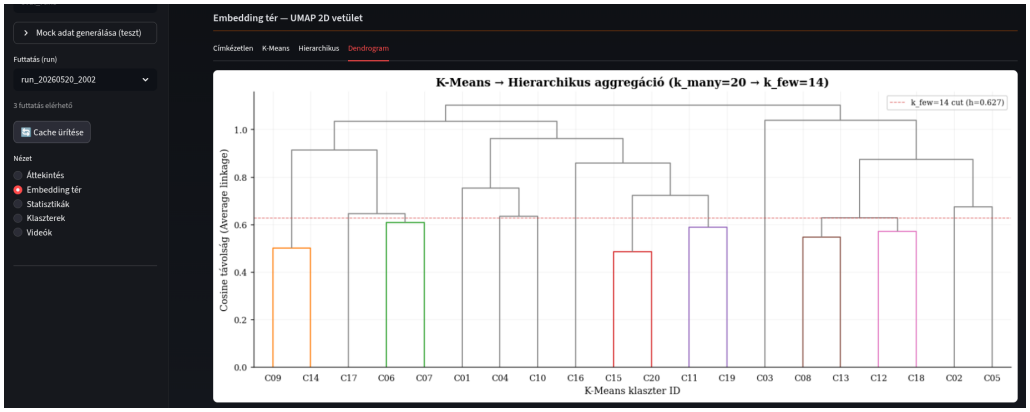
A bal oldalon egy *sidebar* található, ami a kiértékelési futtatások közötti váltást teszi lehetővé. A kiértékelési futtatások kimenetei egy `run_«időbélyeg»` mappába kerülnek mentésre, ezek pedig egységesen az `eval_runs` mappába kerülnek tárolásra. A felületen kiválasztható az adott futtatás időbélyeg azonosítója, melynek adatai a megfelelő mappa tartalma alapján kerülnek megjelenítésre. Ezek alatt található a cache-ürítő gomb, valamint a teszteléshez és fejlesztéshez alkalmazott mock-adatok generálásához használt panel. Ezek alatt lehet kiválasztani a jobb oldali fő megjelenítő terület nézeteit, melyek a következő bekezdések szerint végzik a megjelenítést.

Áttekintés : Tartalmazza az adott futtatás konfigurációját és paramétereit. A megjelenítés a 2.6. ábrán látható. A kiértékelő folyamat lépéseit jelző címkék zölden jelzik a sikeresen végrehajtott, pirosan a sikertelen lépéseket. Ezek alatt négy különböző Streamlit-kártyán jelenik meg a klaszterszám a silhouette metrika, a Hopkins-statisztika és a az adatok lefedettsége. Alattuk pedig a metrikák részletes táblázata egy Streamlit `st.dataframe`-ben.



2.6. ábra. A webalkalmazás Áttekintés nézete.

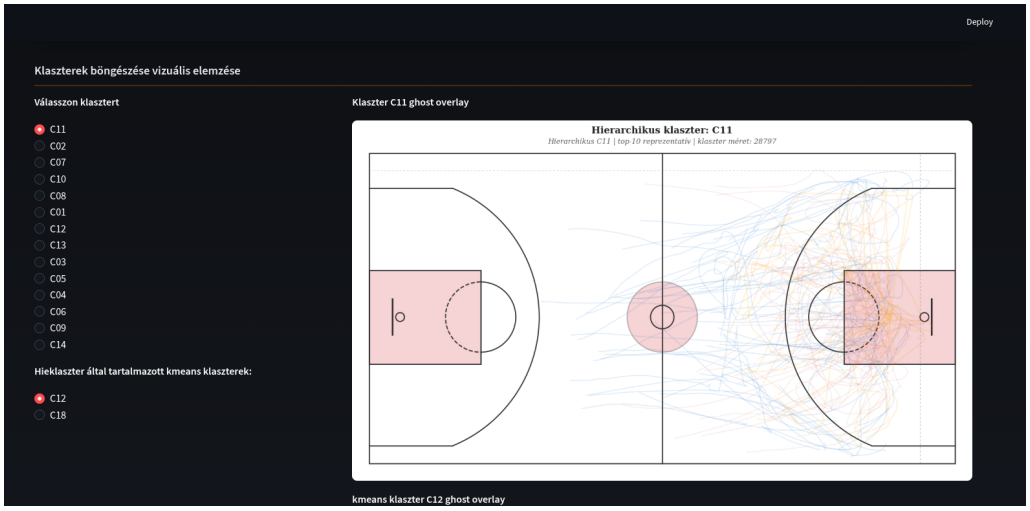
Embedding tér. Itt kerülnek megjelenítésre a kiértékelés során előállított embedding-tér vizualizációk, valamint a hierarchikus klaszterezés során előállított dendrogram ábrája is ami a 2.7. ábrán is látható.



2.7. ábra. Az Embedding-tér menü megjelenítése.

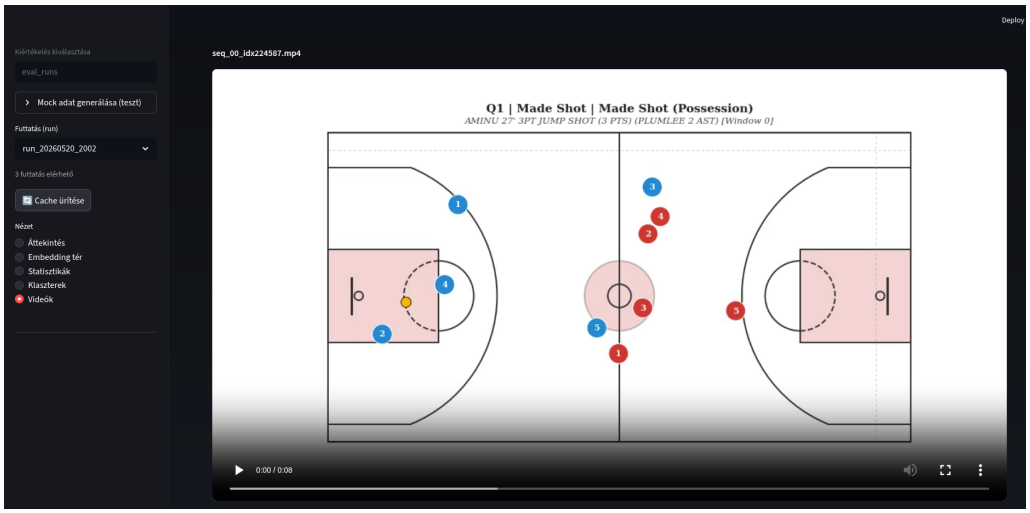
Statisztikák. Itt a futtatások statisztikáinak részletesebb megjelenítésére és a statisztikák alapján készített vizualizációk is megjelenítésre kerülnek. A Hopkins-statisztika és a bootstrap ARI-stabilitás (3 .fejezet) önálló kártyán, alattuk pedig a silhouette értékek és klasztertávolságokra vonatkozó ábrák foglalnak helyet.

Klaszterek. A menüben a felhasználó kiválaszthat egy klasztert rádiógombok segítségével (`st.radio`). Ez alapján betöltésre kerülnek a mappastruktúrából az overlay ábrák (a klaszterek legjellemzőbb játékainak pályán történő együttes ábrázolása a 3. fejezetben). Ez a 2.8. ábrán látható módon történik. Emellett megjelenik egy klaszterméret szerint csökkenően rendezett statisztika a klaszterekről.

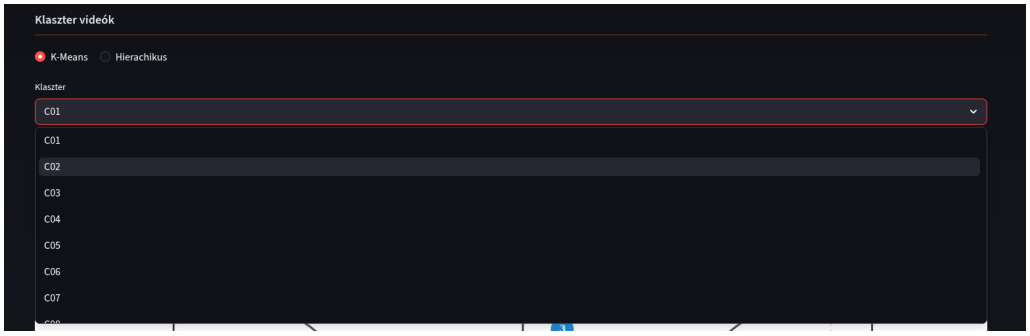


2.8. ábra. Klaszterek menüpont megjelenítése.

Videók. Itt a kiértékelés során generált klaszterenkénti videók tekinthetők meg. A kiválasztott klaszterhez tartozó videók egymás alatt jelennek meg, ahogyan a 2.9. ábrán, a kiválasztás menete pedig a 2.10. ábrán látható



2.9. ábra. Videók menüpont megjelenítése.



2.10. ábra. Klaszter kiválasztása a Videók nézetben.

2.5.4. Mock-adat alapú fejlesztés és tesztelhetőség

A tesztelhetőséghez és a fejlesztés megkönnyítése érdekében vezettem be a `mock_data.py` modult. Ennek célja a valódi kiértékelő folyamat mappastruktúrájával azonos kimeneti fájlok generálása szintetikus adatokkal. Ennek segítségével könnyen a kiértékelő futtatása nélkül tudtam előállítani adatokat a tesztelés elvégzéséhez. Emellett a fejlesztés párhuzamosan történhetett a modellel, ugyanis csak a definiált kimeneti struktúrát kellett betartani a kompatibilitás megőrzése érdekében, a modell fejlesztésének befejezéséig pedig a weboldal fejlesztése alatt használhattam a generált teszt adatokat. Ehhez a felület sidebar paneljén is megjelenik a mockadat-generálás gombja. Fontos, hogy a kiválasztható futtatások között a mappa elnevezésben és a felületen is megjelenő figyelmeztetés jelzi a tesztelés során, hogy nem valós tesztadatok láthatóak. Ez a figyelmeztetés a valós adatok betöltése során eltűnik.

2.6. A webes környezet tesztelése

A felület tesztelését a `pytest` [29] keretrendszer segítségével végeztem el.

A `test_webapp.py` modul végzi a tesztek előkészítését, felépítését és futtatását. Öt tesztcsoportot implementáltam, amelyek több tesztmetódust tartalmaznak.

TestMockGenerator. Ellenőrzi a tesztadatok generálása során felépített mappa struktúrát valamint, hogy a kötelező elemek és fájlok előálljanak. Ez azért szükséges, hogy a

mock adatokon történő fejlesztés után a valós adatokra való átállás egyszerűen elvégezhető legyen.

TestRunLoader. Teszteli a `list_runs()`, `detect_many_dir()`, `load_run()`, `get_per_cluster_table()` és `get_macro_grouping()` függvények működését, amelyek a `run_loader.py` adatbetöltő réteg működését implementálják. Ellenőrzésre kerül a helyes kimenet, valamint az adatok összegyűjtésének folyamata a mappastruktúra szerint.

TestCompatibility. Ellenőrzi, hogy az implementált kiértékelő folyamat által előállított kimenetek rendelkeznek-e a megfelelő struktúrával, valamint elérhetők-e a kiértékelőhöz tartozó annotációk és metaadatok a megjelenítéshez.

TestRobustness. Teszteli, hogy a megjelenítő rendszer képes-e betölteni részben lefutott vagy hiba miatt leállt kiértékelések tartalmát. Ebben a tesztcsoportban négy különálló esetet vizsgáltam. Az elsőben felépítettem egy mock-eredményt, amiben a kiértékelésnek csak az 1. lépése fejeződött be. A második esetben hiányzik a `config.json` fájl. A harmadikban sérült JSON-fájlokat adtam a bemenetre. A negyedik teszt pedig teljesen üres run mappát tartalmaz, ami azért fontos, hogy a kiértékelési lépések státuszát megjelenítő elem megfelelően működjön.

TestStreamlitApp. A rendszer végponttól végpontig történő futása kerül ellenőrzésre. Ezt az `AppTest` [30] keretrendszer segítségével implementáltam. Az `AppTest` a Streamlit-szkriptet böngésző használata nélküli futtatással eléri a megjelenítés során renderelt komponenseket. Itt az eddig említett összes komponens konkrét megjelenését ellenőriztem, azaz hogy mind az öt fő nézet előállt-e, a futtatások kiválasztásánál megjelenik-e a megfelelő bemeneti mappa, a metrikák megjelennek-e az összesítő blokkban, a klaszterek kiválasztási logikája megfelelően működik-e és teljesen üres bemeneti mappa esetén a felület kiadja-e figyelmeztetést a felhasználónak.

A TestStreamlitApp tesztjei ellenőrzik a UI azon részeit, amelyek a unit tesztekkel nehezen megvalósíthatóak vagy dinamikus működést tartalmaznak. Ugyanakkor ez csak a műveletek sikerességét ellenőrzi, a tartalmukat és kimenetüket nem, így a tesztek sikeres lefutása után vizuálisan is ellenőriztem a megjelenő értékeket, valamint kipróbáltam minden funkciót manuálisan is. A tesztelés során minden unit tesztem sikeresen lefutott.

3. fejezet

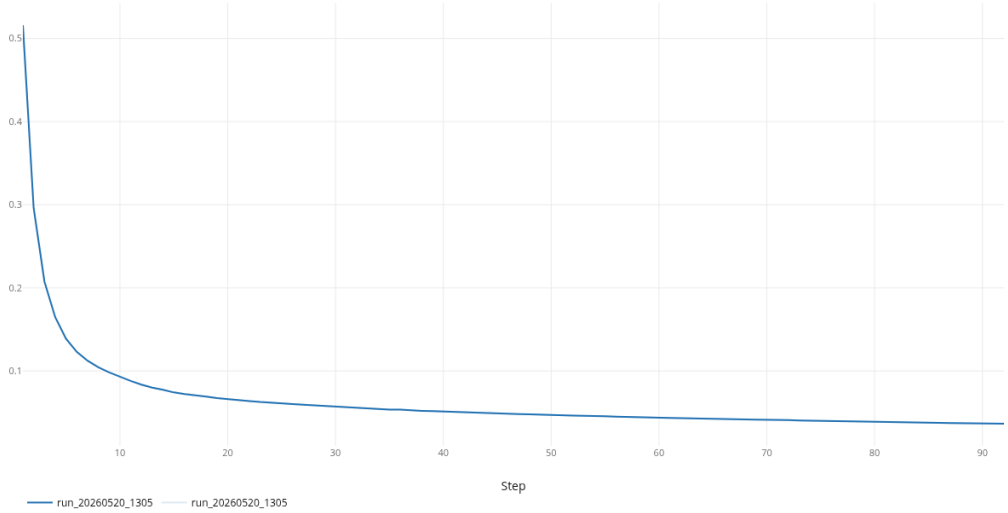
Modell eredményeinek kiértékelése

A következő fejezetben bemutatásra kerül a modell tanulási folyamatának lefutása a naplózott metrikák alapján, valamint a kimenetként előállt embedding-térre alkalmazott klaszterezési módszertan. Részletes bemutatásra kerül az embedding-vektorok szerkezetének vizsgálata és kiértékelése. Zárásként pedig felsorolásra kerül néhány továbbfejlesztési lehetőség az eredményekből levont konklúziók alapján.

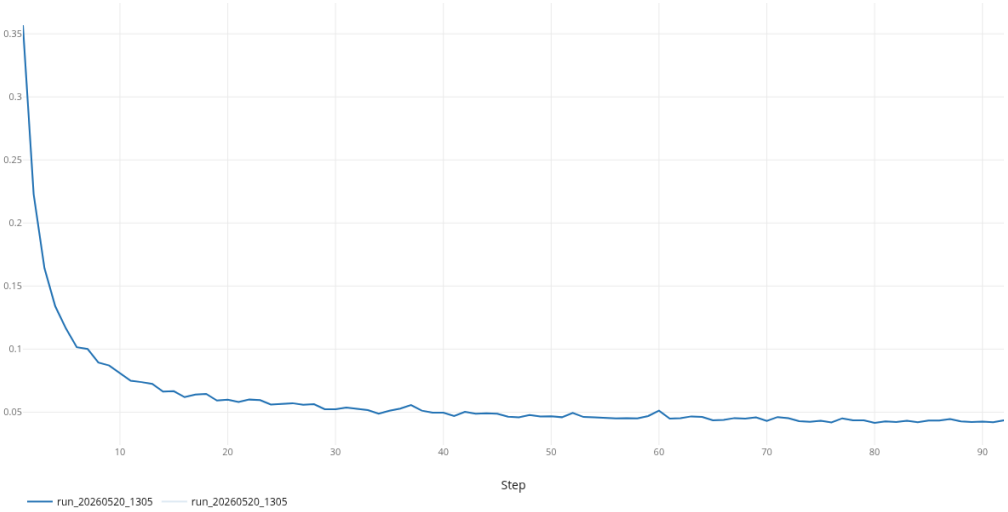
3.1. A tanítási folyamat lefutása

A tanítási folyamat során a modell viselkedését és metrikáinak változását az *MLflow* eszköz segítségével követtem nyomon, valamint a pontos dokumentálást is ez tette lehetővé. A tanulás 92 epoch-on keresztül futott, ahol az early stopping (korai megállás) mechanizmusa megszakította a további lépéseket, valamint egy epoch futási ideje a folyamat során stabilan 130 másodpercig tartott. Az L_val_pretext összetett veszteségfüggvény értékei folyamatos csökkenést mutattak, a kezdeti epoch 0,357-es értékéről egészen 0,0436-ig. Emellett a pretext veszteségek is monoton csökkenő trendet követtek. A tanulási folyamat alatt az `MLflow` segítségével rögzített veszteségmetrikák alakulása a 3.1. ábrán látható. A további veszteséggörbék a függelék F.2.1. ábra tartalmazza. Az early stopping 12 lépéses türelmi kritériuma értelmében a 80. epoch jelentette a legjobb modellállapotot, ahol a veszteségérték 0,04150 volt. Ennek a legjobb modellverziónak a súlyai kerültek kimentésre.

A *silhouette próba* ellenőrizte a pretext feladatok által kialakított embedding-tér szegmentációjának minőségét. A legmagasabb értéke (0,126) volt az 5. epoch-ban, majd a későbbi epoch-ok során a 0,10 körül stabilizálódott. A későbbiekben a teljes adathalmazon elvégzett kiértékelés eredményeinek koszinusz alapú silhouette metrikája is hasonló, 0,1032-es értéket mutatott, így ez alapján megállapítható, hogy a silhouette-probe valós szerkezeti visszajelzést adott a folyamatról.



(a) Tanítási összetett veszteség



(b) Validációs összetett veszteség

3.1. ábra. Tanítási és validációs veszteségek alakulása a tanítás alatt.

3.2. Az embedding-tér kiértékelési módszertana

Az implementált modellem kiértékelése során célom a kimeneti embedding-vektorok klaszterezésére épülő strukturális vizsgálat elvégzése volt. Helyes modell működés esetén az eltérő taktikai típusok eltérő helyen sűrűsödnek az embedding-térben. A vizsgálatot egyrészt a klaszterezés jóságát jelző metrikák, másrészt az adott klaszterek taktikai elemeinek vizuális ellenőrzése alapján végeztem el. A kiértékelést a teljes, 246510 mintát tartalmazó adathalmazon hajtottam végre.

3.2.1. Kétszintű klaszterezési pipeline

A klaszterezést két lépésben végeztem el. Kezdetben egy K-Means algoritmust futtattam le, koszinusz alapú távolságmetrikát alkalmazva, a stabilabb számítási végrehajtás érdekében. A modell inicializálásához használt klaszterszám finomhangolásra került a $\{20, 22, \ldots, 40\}$ tartományban, melyek közül a legmagasabb koszinusz alapú silhouette értéket eredményező érték került kiválasztásra. A második lépésben a K-Means kimenetére

épített hierarchikus klaszterezést alkalmaztam, szintén koszinusz távolság alkalmazásával. Az eredményül kapott dendrogram a $k \in \{8, 9, \ldots, 14\}$ szinteken került vágásra, majd az előző logikához hasonlóan a koszinusz silhouette score alapján határoztam meg az optimális értéket.

A kétlépéses módszerrel egyszerre próbáltam lehetővé tenni a finomabb és durvább taktikai különbségek vizsgálatát. A folyamat végén mindkét lépés eredménye külön mentésre kerül az összehasonlíthatóság érdekében.

3.2.2. Vizualizáció UMAP-pal

A 128 dimenziós embedding-tér 2 dimenziós vizualizációjához dimenzióredukciós eljárást szükséges végrehajtani, amit az *Uniform Manifold Approximation and Projection* UMAP algoritmus [31] alkalmazásával valósítottam meg. Inicializálásakor koszinusz metrikát állítottam be, valamint az `n_neighbors=30` és `min_dist=0,1` értékekkel paramétereztem. Az UMAP előnye, hogy próbálja megőrizni az adatok lokális és globális struktúráját a transzformáció végrehajtása közben. A dimenzióredukció kizárólag a megjelenítés során kerül felhasználásra, a klaszterezés és a metrikák továbbra is a 128 dimenziós térben történnek.

3.3. Alkalmazott kiértékelési metrikák és eredmények

A klaszterezés jóságának kiértékelésére csak geometria alapú, címkézés nélküli metrikákat alkalmaztam. Mindegyik különböző jelentéssel és interpretációval rendelkezik, ami alapján megítélhető az embedding-tér szegmentáltsága.

A *Hopkins-statisztika* [32] klaszterezésre alkalmazott változata megmutatja, hogy egy mintahalmaz, jelen esetben az embedding-tér, mennyire jól klaszterezhető, vagy inkább közelítőleg egyenletes eloszlású-e. A $H \approx 0{,}5$ érték esetén az adatok egyenletes eloszlásúak klaszterek nélkül, az ennél nagyobb értékek egyre jobb klaszterezhetőséget jelentenek. A számítását hatékonyságnövelés érdekében 1000 mintán végeztem el, ezen a viszonylag kevés mintaszámon számított Hopkins-statisztika $H = 0{,}7971$ eredményt adott. Ez ugyan erős klaszterezhetőségre utalna, ugyanakkor a mintaszám torzítása miatt fenntartásokkal kezeltem.

A Silhouette-score-t továbbra is kiértékelő metrikaként alkalmaztam, ez viszont a 2.4.6.4. alfejezetben már bemutatásra került. $[-1, 1]$ tartományú értékeket vehet fel, ahol a pozitív érték jelzi a jó szegmentációt.

A *Davies–Bouldin-mutató* [33] minden klaszterre meghatározza a vele leginkább összemosódott klasztert, meghatározza az összemosódásuk értékét, majd ezeknek az értékeknek veszi az átlagát. Ebből adódóan ennek a metrikának a minimalizálása a cél.

A *Calinski–Harabasz-mutató* [34] metrika a klaszterek között és klasztereken belüli variancia hányadosát határozza meg, így ennek maximalizálása jelzi a klaszterezés helyességét.

A klaszterek belső struktúrájának vizsgálatára alkalmaztam a *Klaszter kohézió* metrikát, ami a klaszterelemek centroidjától mért átlagos koszinusz távolságát adja meg. Minél alacsonyabb a mérőszám, annál tömörebb egy klaszter szerkezete.

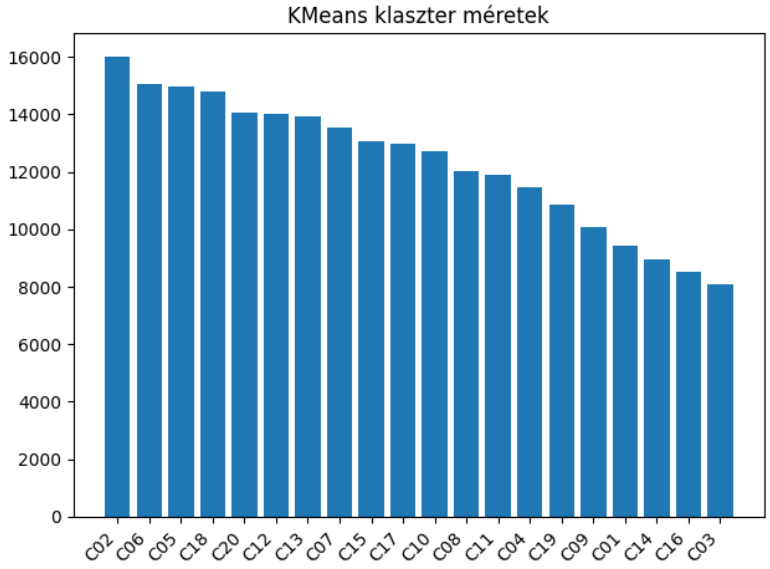
3.4. Eredmények

3.4.1. K-Means klaszterezés eredménye

A finomhangolás során a $k = 20$ klaszterszám paraméter került meghatározásra, így a K-Means 20 elkülönített klasztert hozott létre az embedding-térben. Ezekre az alábbi metrikák kerültek kiszámításra.

- silhouette-score: 0,1032
- Davies–Bouldin mutató: 2,876
- Calinski–Harabasz mutató: 161,76

A mérőszámok értelmezése alapján látható, hogy a klaszterek szegmentációja mérhetően helyes és értelmezhető, ugyanakkor kezdetleges és megjelenik közöttük összemosódás. Ezt a megállapítást alátámasztja, hogy a legkiemelkedőbb C03 jelölésű klaszter 0,343 silhouette értéket kapott. Ennek a vizuális ellenőrzése során szembetűnő, hogy a jobb térfélről indított bedobás utáni balra történő labdafelhozatalok kerültek ebbe a csoportba. Tehát látható, hogy a modell jól meg tudott különböztetni egy ilyen nagy különbséget, ebből következtetve, hogy a felállt támadások közötti különbség miért csak ennél kisebb silhouette értékeket kapott. A továbbiakban lesz szó a vizuális ellenőrzésről, ahol konkrét taktikai elemek is megjelennek az eredmények között. Az előző állítást, hogy a klaszterezés ténylegesen hordoz taktikai reprezentációt, az is alátámasztja, hogy nem lett negatív silhouette-score érték a klaszterek között. Itt fontosnak tartom megjegyezni a kosárlabda taktikák jellegét is, ugyanis egy taktika lefutása ritkán ugyanaz. A szemmel vizsgált taktikai elemek is összemosódnak, például egy bika (Horns offense) és egy középen lejátszott elzárás (pick-and-roll) játék esetén, amit így a modell is nehezen tud megkülönböztetni. Másrészről egy játék lejátszása több taktikai elemet is ötvözhet, ami szintén elmoshatja a klaszterezés határait. A 3.2. ábrán látható a 20 klaszter elemszáma. Ezt vizsgálva az eloszlásuk egyenletesnek bizonyult. Megállapítható tehát, hogy a klaszterezés nem alakított ki hatalmas domináns klasztereket, így az az implicit következtetés is levonható, hogy az embedding-tér struktúrájában sem alakultak ki hatalmas sűrűsödési csomópontok, amik összemosnák a különböző taktikákat.

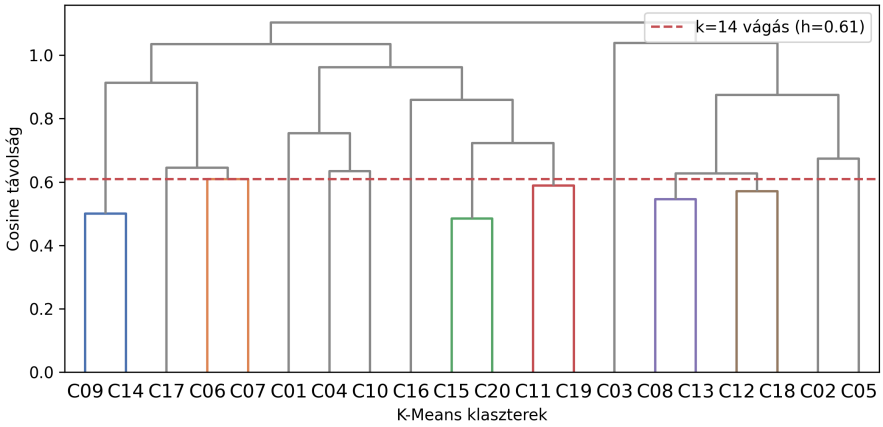


3.2. ábra. A K-Means klaszterméretei.

A Davies–Bouldin és Calinski–Harabasz mutatók a fejlesztés során voltak mérvadóak, ugyanis ezek javulása jelentette, hogy a modellen és a kiértékelő klaszterezésen véghezvitt iteratív fejlesztések hatásosnak bizonyultak.

3.4.2. Hierarchikus klaszterezés

A hierarchikus klaszterezés bevezetésével a K-Means által elkülönített csoportok robusztusságát akartam vizsgálni, az alapján, hogy hány klasztert von össze a hierarchikus megközelítés. Kevés összevonás igazolja, hogy már a K-Means is megfelelő klasztereket szegmentált. Az összevonásokra pedig elmondható, hogy ezek különválasztása viszonylag kis különbségeken alapult. A fentebb már említett silhouette-pontszám alapján történő vágás meghatározásával az eredeti 20 csoportot a 3.3. ábrán látható módon 14 tömörebb klaszterbe vonta össze a módszer, amiből 7 a K-Means által megállapított csoportok unióját képezi. Ez alapján megállapítható, hogy a különböző klaszterek többsége már eredetileg is eltérő elemeket tartalmazott a K-Means során ahhoz, hogy ne legyen szükség további összevonásokra.



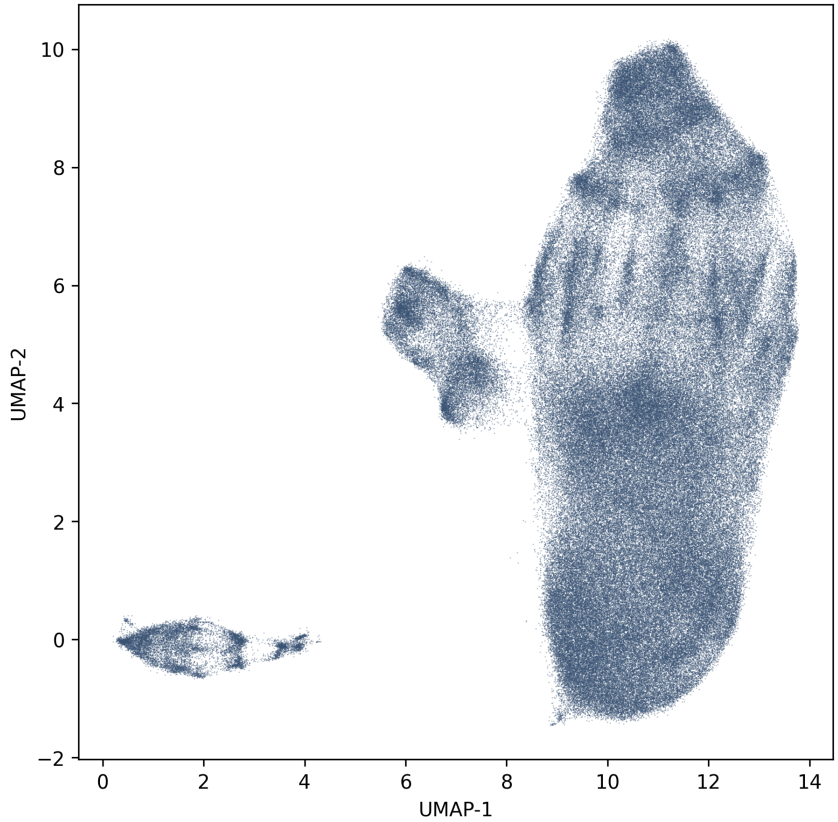
3.3. ábra. Hierarchikus klaszterezés dendrogramja.

3.4.3. Klaszterezés stabilitása

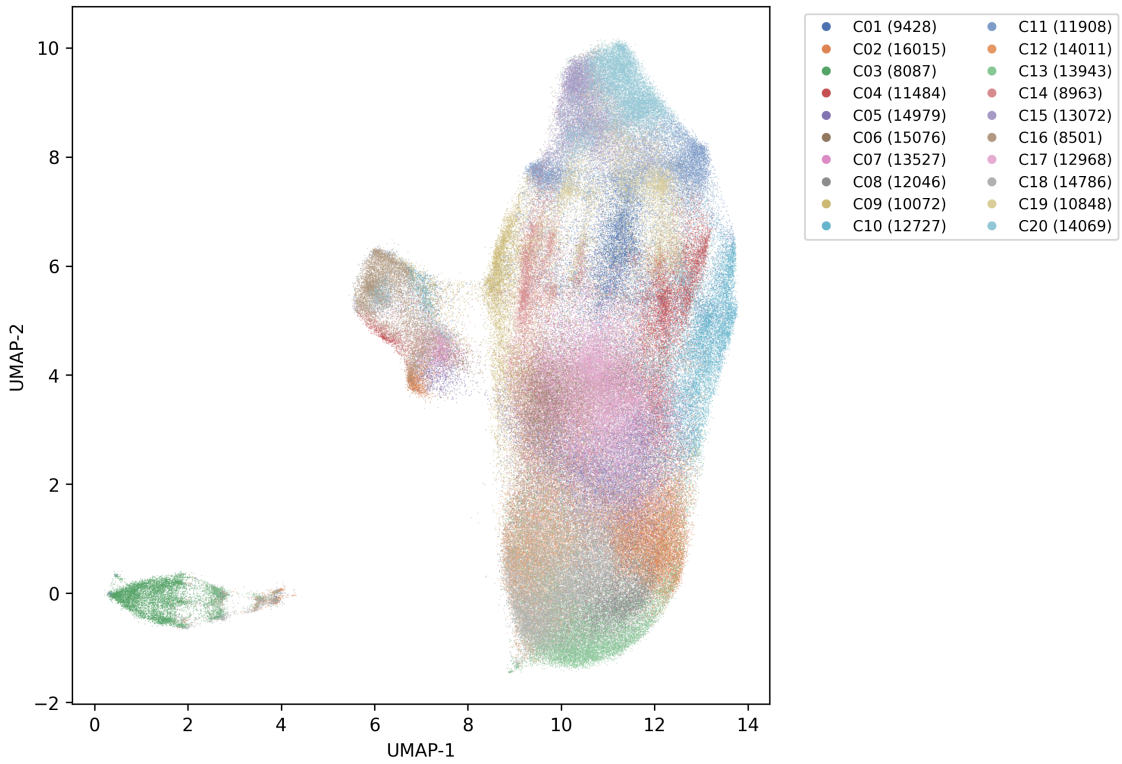
Közismert, hogy a K-Means algoritmus futása nem determinisztikus és nagyban függ a kezdeti paraméterek értékétől, valamint a megadott véletlen kezdőértéktől. Az implementációba ezért egy bootstrap alapú megközelítéssel 5 különböző random seed-del történő tesztet hajtottam végre annak ellenőrzésére, hogy mennyire befolyásolja a paraméterezés a klaszterek kialakulását. Ezek 30 000 elemszámú mintahalmazon kerültek végrehajtásra a futási idő csökkentése érdekében. Az 5 különböző klaszterezés kimenetének minden párosításán vizsgáltam a `ARI` (*Adjusted Rand Index*) értéket [35], ami két halmaz elemei közötti egyezést állapítja meg a mintapárok alapján. A forrás alapján `ARI` értéke $[-1, 1]$ értéket vehet fel. A teszt eredménye $\texttt{ARI} = 0{,}6519$ értéket adott, így az eljárásom stabilnak tekinthető.

3.4.4. Vizuális elemzés az embedding-térben

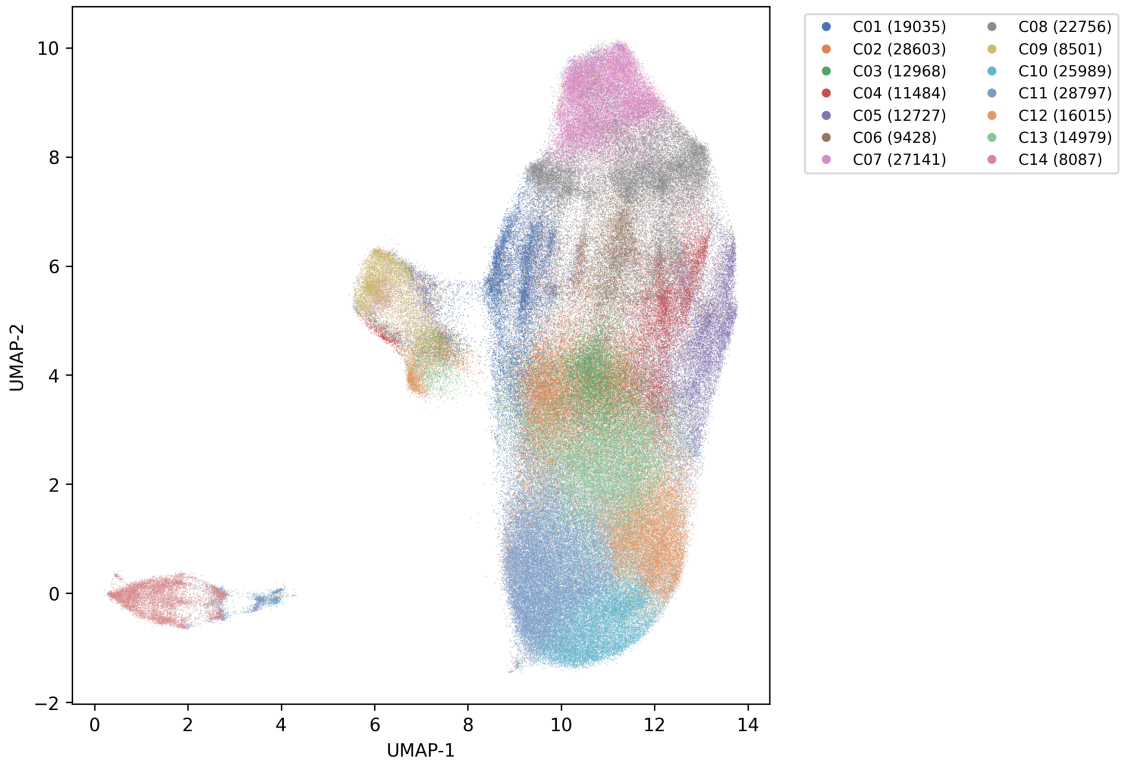
A metrikák alapján történő kiértékelés elengedhetetlen a módszertan vizsgálata szempontjából, de a felügyelet nélküli reprezentációtanulás miatt a referencia nélküli, tisztán geometriára és struktúrára épülő mérőszámok alapján önmagában nehéz megállapítani a modell jóságát. Ezért a mérőszámok vizsgálata után vizuális ellenőrzést is végeztem a kialakult klaszterekre. Ehhez a már korábban említett módszerrel előállított, 3.4. ábrán látható embedding-tér vizualizációját. A 3.5 és 3.6. ábrák pedig a kiértékelő klaszterezés eredményeit szemléltetik az embedding-térben. Ezeken jól megfigyelhetőek a reprezentáció alapú sűrűsödések, valamint teljesen különálló klaszterek és a hierarchikus összevonások hatásai is.



3.4. ábra. Az embedding tér struktúrája UMAP vizualizációval.



3.5. ábra. K-Means klaszterek az embedding-térben.

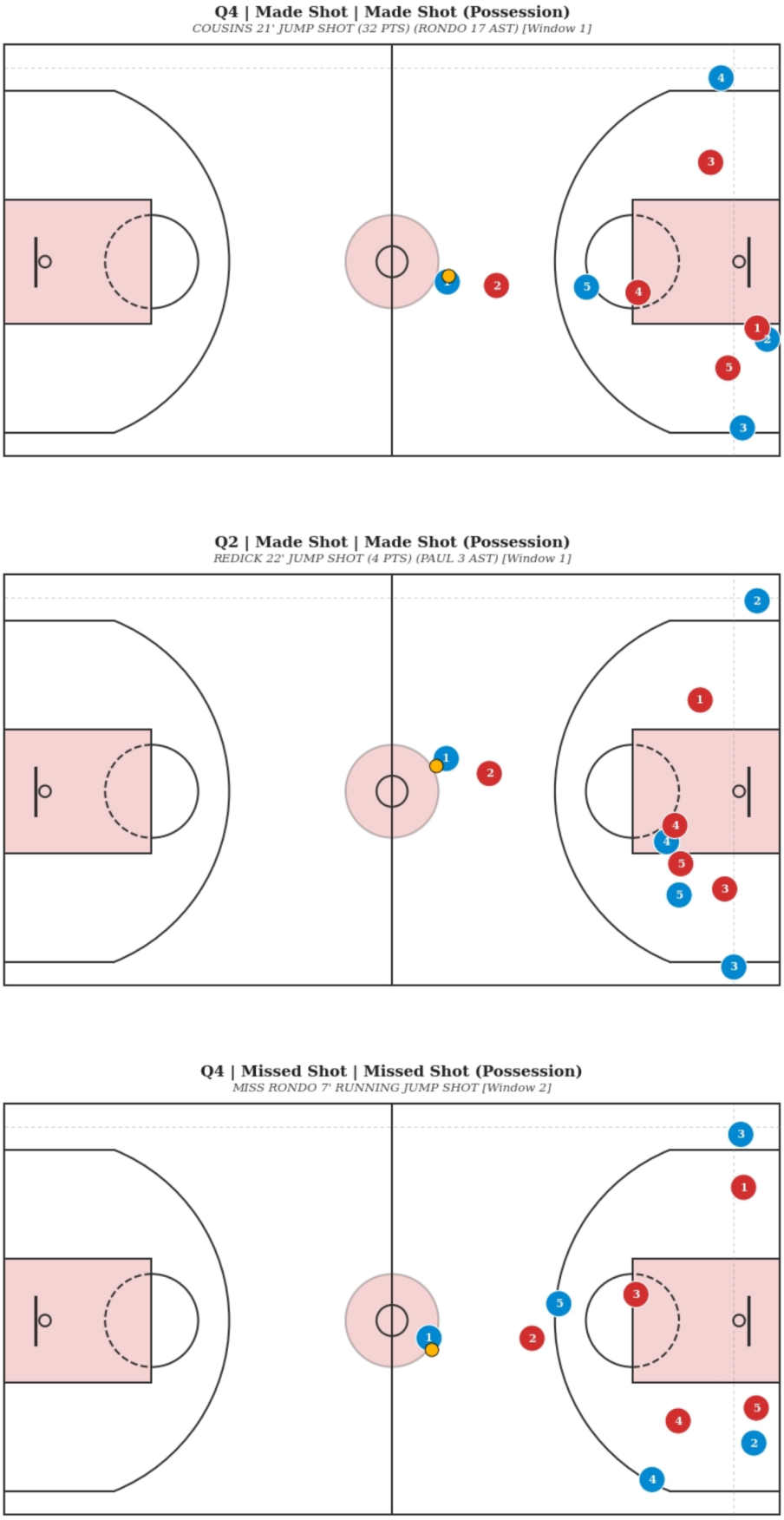


3.6. ábra. Hierarchikus összevont klaszterek az embedding-térben.

A klaszterek szekvenciáihoz tartozó vizualizációkat és videókat generáltam a tracking adatok alapján, valamint a hierarchikus klaszteren overlay ábrákat készítettem, ami az adott klaszterek centroid-távolság alapján meghatározott 10 legreprezentatívabb szekvenciáinak egymás felett történő ábrázolását jelenti. Ez segíti az ismételten előforduló taktikai mintázatok azonosítását az adott klaszteren belül.

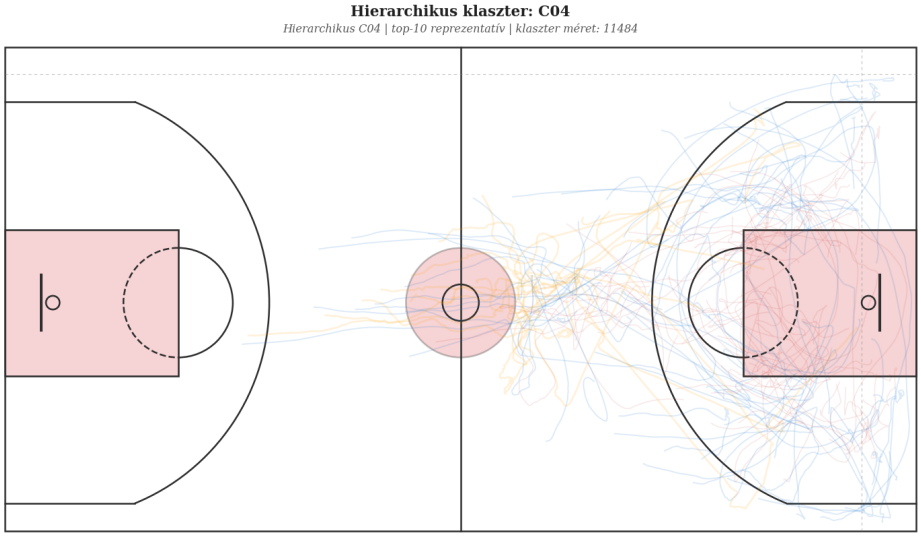
Ahogy a 3.7. ábrán a C4 videóinak pillanatképei láthatók. Mind a 3 szekvencia játékai taktikailag és felállás szerint hasonlítanak és egy középen bekövetkező elzárás (pick-and-roll) előtti felállást tartalmaznak.

A többi klasztert elemezve hasonló volt a tapasztalatom, hogy többnyire hasonló felállású játékokat gyűjtenek össze, a különbségek pedig a játékok lefutásából adódtak. Például az elzárásokat máshova adják, vagy más támadó pozícióba futnak a játékosok a játék végén, ami a játékvariációkból vagy a játékosok önálló döntéséből is adódhat.

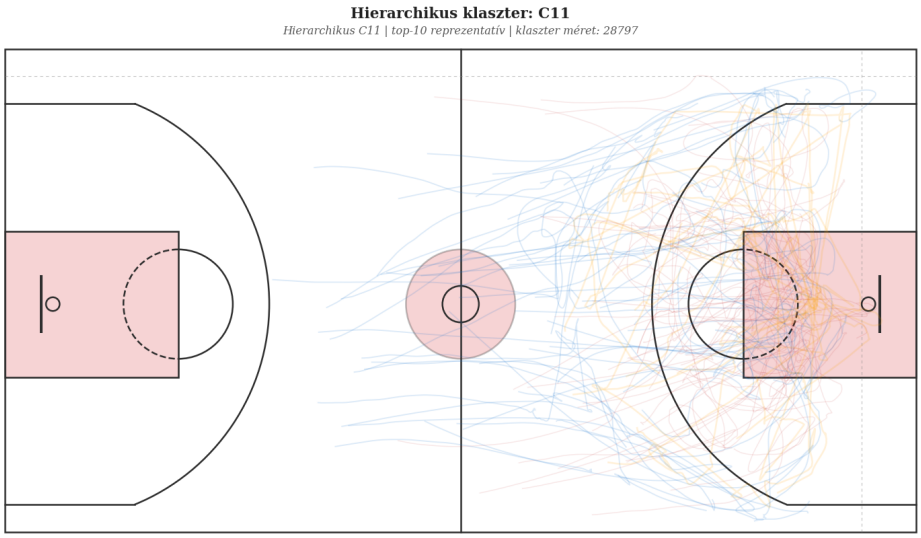


3.7. ábra. Támadó és védő játékosok térbeli elhelyezkedése a pálya különböző szituációiban.

Az overlay, mint domináns klaszterenkénti mozgások ábrázolása jól mutatja az adott klaszterben felismert tipikus játékok szerkezetét. Amíg a 3.8a. ábrán a C4-es klaszter esetén a középre adott elzárások dominálnak, addig a 3.8b. ábrán inkább a 45 fokról történő betörések jelennek meg.



(a) C04-es hierarchikus klaszter



(b) C11-es hierarchikus klaszter

3.8. ábra. A játékosok domináns klaszterenkénti mozgása.

Összességében elmondható a metrikák alapján végzett kiértékelés, az embedding-tér vizsgálata és a vizuális ellenőrzés alapján, hogy megjelennek klaszterenként tipikus játéktípusok és felállások. Ez bizonyítja, hogy a modell a reprezentációtanulás során képes volt megérteni a kosárlabdára vonatkozó tudást.

4. fejezet

Összefoglalás és továbbfejlesztési lehetőségek

A szakdolgozatomban azt vizsgáltam, hogy egy BERT-alapú, önfelügyelt tanulást megvalósító architektúra mennyire alkalmas a kosárlabdára jellemző taktika és játékdinamika megértésére.

Az implementált modellem újítása a térbeli és időbeli információ feldolgozásának különválasztása önálló modellkomponensekben. A teljes modell a multi-task tanulás megvalósítása érdekében öt párhuzamos önfelügyelt pretext feladaton hajtja végre a tanulási folyamatot.

A betanított modell teljesítményét az előállt embedding-tér klaszterezésével, a klaszterezési metrikákkal, valamint a klaszterek vizuális ellenőrzésével vizsgáltam. A metrikák alapján látható, hogy a klaszterek szegmentációja mérhetően helyes, ugyanakkor kezdetleges, és összemosódás jelenik meg közöttük. A $k = 20$ klaszterszámú K-Means klaszterezés 0,1032 silhouette-score-t, 2,876 Davies–Bouldin és 161,76 Calinski–Harabasz értékeket adott, a klaszterméretek eloszlása pedig egyenletesnek bizonyult, vagyis nem alakultak ki a taktikákat összemosó domináns sűrűsödések. A hierarchikus klaszterezés a 20 csoportot mindössze 14-be vonta össze, ami megerősíti, hogy már a K-Means is stabil csoportokat különített el. A klaszterek vizuális ellenőrzése során klaszterenként tipikus játéktípusok és felállások rajzolódtak ki, ami alátámasztja a korábbi metrikák eredményeit.

Összességében a metrikák, az embedding-tér vizsgálata és a vizuális ellenőrzés együttesen igazolja, hogy a megközelítésem önfelügyelt módon, multi-task tanulással, címkézés nélkül képes volt a kosárlabdára vonatkozó taktikai tudás megtanulására. A klaszterek között megjelenő összemosódás alapján ugyanakkor a reprezentáció még kezdetlegesnek tekinthető, így az éles környezetben történő alkalmazáshoz a rendszer további fejlesztése szükséges.

4.1. Továbbfejlesztési lehetőségek

A modell eredményeinek további javítása érdekében az alábbi fejlesztési lépéseket javasolom.

Változó méretű szekvenciák kezelése. Ugyan a modell a fix mérettől rövidebb szekvenciák hosszát maszkolással kezeli, jelenleg ezzel a fix tokenmérettel dolgozik. Ennek előnye, hogy nem kell kezelni a figyelem megoszlását a változó hosszú tokenek között, de ez egyben megkötést is jelent, mert az azonosított PBP szeleteket még tovább kellett bontanom. Ezek éles vágásokat és zajt jelentenek a modell számára, valamint a modell nem a teljes labdabirtoklást látja a kontextus vizsgálatakor, csak annak szeleteit. A csúszóablakos

megközelítés az éles vágásokat valamilyen szinten ellensúlyozza, és tompítja ezek hatását, de a jövőbeli fejlesztés célja lehetne a változó hosszú teljes labdabirtoklások kezelésének beépítése.

Támadási irány pontosabb meghatározása. A 3.4.1. alfejezetben már utaltam arra, hogy a támadási irányok még precízebb meghatározása fontos jövőbeli feladat lehet. Az irány egységesítése és garantálása érdekében számos szűrőt implementáltam, de például nem kezeli a fentebb említett esetet, hogy mi történik akkor, ha a támadás a megfelelő térfélen és irányban kezdődik, de mégis a másik palánk felé indulnak meg a támadók. Az adatok jelenlegi szűrése számos ilyen szélsőséges esetet tartalmaz, amiből az implementáció során a lehető legtöbbet próbáltam lefedni, de a jövőben ezeknek a szűrőknek a bővítése hasznos lehet.

Hiperparaméter optimalizálás további kiterjesztése. A modellt számos paramétere alapján optimalizáltam, például az alkalmazott súlyozások, veszteségfüggvények és regularizációs technikák bevezetésével. Ugyanakkor egy ilyen összetett modell esetén a hiperparaméterek optimalizálása soha véget nem érő feladat, így ennek folytatása is ajánlott.

Pretext feladatok fejlesztése. Ahogy a korábbiakban már említésre került, a helyes reprezentáció kialakulását nagyrészt a jól definiált pretext-feladatok eredményezik. A jövőben még több, a kosárlabda-domain ismereteit is alkalmazó pretext-feladat definiálása potenciálisan további javulást hozhat az eredményekben.

Irodalomjegyzék

[1] Alexander Franks – Andrew Miller – Luke Bornn – Kirk Goldsberry: Characterizing the spatial structure of defensive skill in professional basketball. *The Annals of Applied Statistics*, 9. évf. (2015) 1. sz., 94 – 121. p.
URL https://doi.org/10.1214/14-AOAS799.

[2] Avery McIntyre – Joel Brooks – John Guttag – Jenna Wiens: Recognizing and analyzing ball screen defense in the NBA. In *Proceedings of the MIT Sloan Sports Analytics Conference* (konferenciaanyag). Boston, MA, USA, 2016.
URL https://www.sloansportsconference.com/research-papers/recognizing-and-analyzing-ball-screen-defense-in-the-nba.

[3] Rajiv Shah – Rob Romijnders: Applying deep learning to basketball trajectories, 2016. URL https://arxiv.org/abs/1608.03793.

[4] Kuan-Chieh Wang – Richard Zemel: Classifying NBA offensive plays using neural networks. In *Proceedings of the MIT Sloan Sports Analytics Conference* (konferenciaanyag). Boston, MA, USA, 2016. URL https://www.cs.toronto.edu/~zemel/documents/1536-Classifying-NBA-Offensive-Plays-Using-Neural-Networks.pdf.

[5] Akhil Nistala – John Guttag: Using deep learning to understand patterns of player movement in the NBA. In *Proceedings of the MIT Sloan Sports Analytics Conference* (konferenciaanyag). Boston, MA, USA, 2019.
URL https://www.sloansportsconference.com/research-papers/using-deep-learning-to-understand-patterns-of-player-movement-in-the-nba.

[6] Ding Ding – H. Howie Huang: A graph attention based approach for trajectory prediction in multi-agent sports games, 2020. URL https://arxiv.org/abs/2012.10531.

[7] Xing Wang – Zitian Tang – Jianchong Shao – Sam Robertson – Miguel-Ángel Gómez – Shaoliang Zhang: HoopTransformer: Advancing NBA offensive play recognition with self-supervised learning from player trajectories. *Sports Medicine*, 54. évf. (2024) 10. sz., 2663–2673. p.
URL https://link.springer.com/article/10.1007/s40279-024-02030-3.

[8] Jianchong Shao – Xing Wang – Konstantinos Pelechrinis – Aaron T. Scanlan – Shaoliang Zhang: Hoop-mssl: Multitask self-supervised representation learning on basketball spatiotemporal data. *Intelligent Sports and Health*, 1. évf. (2025) 4. sz., 243–251. p. ISSN 3050-5445. URL https://www.sciencedirect.com/science/article/pii/S3050544525000465.

[9] Ashish Vaswani – Noam Shazeer – Niki Parmar – Jakob Uszkoreit – Llion Jones – Aidan N. Gomez – Lukasz Kaiser – Illia Polosukhin: Attention is all you need, 2023. URL https://arxiv.org/abs/1706.03762.

[10] Jacob Devlin – Ming-Wei Chang – Kenton Lee – Kristina Toutanova: Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
URL https://arxiv.org/abs/1810.04805.

[11] Petar Veličković – Guillem Cucurull – Arantxa Casanova – Adriana Romero – Pietro Liò – Yoshua Bengio: Graph attention networks, 2018.
URL https://arxiv.org/abs/1710.10903.

[12] Shaked Brody – Uri Alon – Eran Yahav: How attentive are graph attention networks?, 2022. URL https://arxiv.org/abs/2105.14491.

[13] Ting Chen – Simon Kornblith – Mohammad Norouzi – Geoffrey Hinton: A simple framework for contrastive learning of visual representations, 2020.
URL https://arxiv.org/abs/2002.05709.

[14] Carl Doersch – Andrew Zisserman: Multi-task self-supervised visual learning, 2017.
URL https://arxiv.org/abs/1708.07860.

[15] Ritchie Vink – Polars Contributors: Polars: Lightning-fast DataFrame library for Rust and Python. GitHub repository, https://github.com/pola-rs/polars, 2020–2025. Hozzáférve: 2026-05-23.

[16] Rajiv Shah: BasketballData: Raw SportVU NBA game logs (2015–16 season). GitHub repository, https://github.com/rajshah4/BasketballData, 2016. Forked from tcash21/BasketballData; Hozzáférve: 2026-05-23.

[17] Swar Patel: nba_api: An API client package to access the APIs for NBA.com. Python package on PyPI, https://pypi.org/project/nba_api/; GitHub: https://github.com/swar/nba_api, 2018–2025. Telepítés: pip install nba_api; Hozzáférve: 2026-05-23.

[18] National Basketball Association: Official NBA rulebook — rule 1: Court dimensions; equipment. https://official.nba.com/rule-no-1-court-dimensions-equipment/, 2024. Hozzáférve: 2026-05-23.

[19] Siu Kwan Lam – Antoine Pitrou – Stanley Seibert: Numba: a llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15 konferenciasorozat. New York, NY, USA, 2015, Association for Computing Machinery. ISBN 9781450340052.
URL https://doi.org/10.1145/2833157.2833162. 6 p.

[20] Abraham. Savitzky – M. J. E. Golay: Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36. évf. (1964) 8. sz., 1627–1639. p. URL https://doi.org/10.1021/ac60214a047.

[21] Eric W. Weisstein: Polygon area. From *MathWorld* – A Wolfram Web Resource. URL https://mathworld.wolfram.com/PolygonArea.html. Hozzáférve: 2026-05-23.

[22] Ruibin Xiong – Yunchang Yang – Di He – Kai Zheng – Shuxin Zheng – Chen Xing – Huishuai Zhang – Yanyan Lan – Liwei Wang – Tie-Yan Liu: On layer normalization in the transformer architecture, 2020.
URL https://arxiv.org/abs/2002.04745.

[23] Ross Girshick: Fast r-cnn, 2015. URL https://arxiv.org/abs/1504.08083.

[24] Ilya Loshchilov – Frank Hutter: Decoupled weight decay regularization, 2019. URL https://arxiv.org/abs/1711.05101.

[25] Ilya Loshchilov – Frank Hutter: Sgdr: Stochastic gradient descent with warm restarts, 2017. URL https://arxiv.org/abs/1608.03983.

[26] Paulius Micikevicius – Sharan Narang – Jonah Alben – Gregory Diamos – Erich Elsen – David Garcia – Boris Ginsburg – Michael Houston – Oleksii Kuchaiev – Ganesh Venkatesh – Hao Wu: Mixed precision training, 2018.
URL https://arxiv.org/abs/1710.03740.

[27] Peter J. Rousseeuw: Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20. évf. (1987), 53–65. p. ISSN 0377-0427. URL https://www.sciencedirect.com/science/article/pii/0377042787901257.

[28] Snowflake Inc.: Streamlit: A faster way to build and share data apps. https://docs.streamlit.io/, 2019–2025. Hozzáférve: 2026-05-23.

[29] Holger Krekel – pytest-dev team: pytest: helps you write better programs. https://docs.pytest.org/, 2004–2025. Hozzáférve: 2026-05-23.

[30] Snowflake Inc.: Streamlit App Testing API Reference. https://docs.streamlit.io/develop/api-reference/app-testing, 2024–2025. Hozzáférve: 2026-05-23.

[31] Leland McInnes – John Healy – James Melville: Umap: Uniform manifold approximation and projection for dimension reduction, 2020.
URL https://arxiv.org/abs/1802.03426.

[32] Richard G. Lawson – Peter C. Jurs: New index for clustering tendency and its application to chemical problems. *Journal of Chemical Information and Computer Sciences*, 30. évf. (1990) 1. sz., 36–41. p.
URL https://pubs.acs.org/doi/10.1021/ci00065a010.

[33] David L. Davies – Donald W. Bouldin: A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1. évf. (1979) 2. sz., 224–227. p.

[34] T. Caliński – J Harabasz: A dendrite method for cluster analysis. *Communications in Statistics*, 3. évf. (1974) 1. sz., 1–27. p.
URL https://doi.org/10.1080/03610927408827101.

[35] Lawrence Hubert – Phipps Arabie: Comparing partitions. *Journal of Classification*, 2. évf. (1985) 1. sz., 193–218. p.
URL https://link.springer.com/article/10.1007/BF01908075.

Függelék

F.1. Nyilatkozat generatív mesterséges intelligencia alkalmazásáról

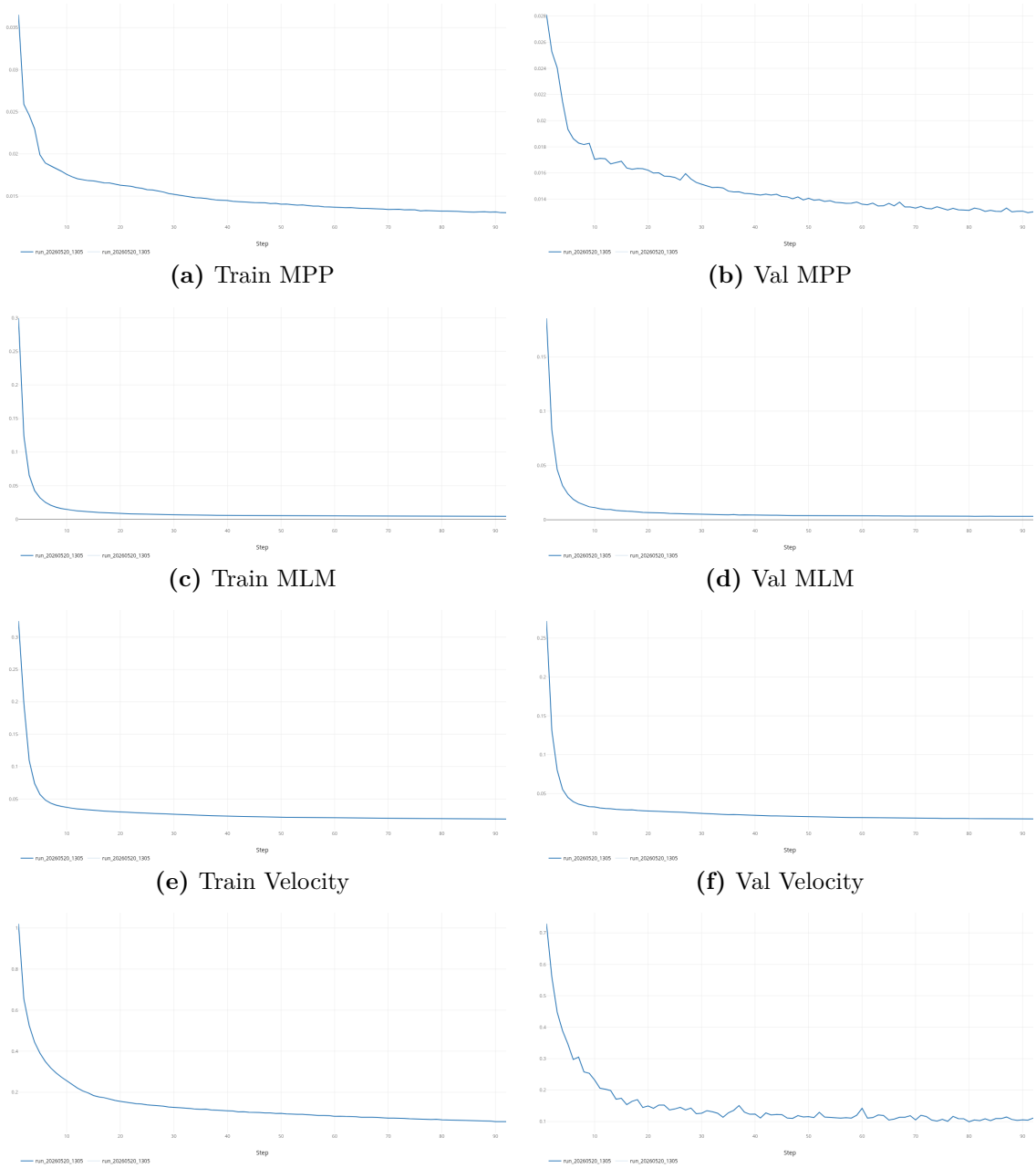
☐ **Nem használtam** semmilyen generatív MI segédeszközt. ☒ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	**Generatív MI eszköz(ök) neve**	**Érintett részek** (fejezet, oldalszám, hivatkozás)	**Használat becsült aránya** (felhasználási módonként)
Irodalomkutatás	Claude (Anthropic)	1.2–1.3. fejezet (2–5. o.), Irodalomjegyzék	~70%
Prompt lényegi része	Kosárlabda sportanalitika releváns szakirodalom keresése , BERT, GAT/GATv2, önfelügyelt multi-task tanulás és SimCLR témakörökben. Az MI által javasolt forrásokat minden esetben ellenőriztem és alkalmazhatóságukat mérlegeltem a konkrét feladatok esetén.		
Programkód generálása	Claude (Anthropic)	2.2–2.6. fejezet (Kódrészletek, függelék vonatkozó része)	~50%
Prompt lényegi része	Python kódvázak generálása PyTorch, Polars s Streamlit könyvtárakhoz. Az adatfeldolgozó folyamat, GATv2-lite gráfháló, Transformer-encoder, tanítási ciklus, Streamlit megjelnítés. Az MI által generált kódvázakat minden esetben átdolgoztam, kiegészítettem, integráltam, futtattam és ellenőriztem.		
Új ötletek, megoldási javaslatok generálása	Claude (Anthropic)	Architekturális és implementációs döntések (2. fejezet)	~15%

Prompt lényegi része	Néhány kisebb tervezési döntésnél, valamint lehetőségek közüli választás segítségére. A főbb architekturális újítások (térbeli és időbeli figyelem szétválasztása, aszimmetrikus támadó-súlyozás) saját ötletek.		
Vázlat létrehozása (szövegstruktúra, vázlatpontok)	Claude (Anthropic)	Tartalomjegyzék	~20%
Prompt lényegi része	A dolgozat fő szerkezetére valamint a fejezetek arányos felosztásához kértem ötleteket. Ezeket alapötletnek felhasználtam, de a konkrét felépítést és az alfejezetek felépítését saját döntéseim alapján írtam meg.		
Szövegblokkok létrehozása	Writefull (OpenAI GPT, Overleafben)	Teljes dolgozat szövege	~10%
Prompt lényegi része	A kezdeti szövegem stilisztikai javítását, szerkezetének pontosítását, elgépelések és helyesírási hibák javítását az Overleaf Writefull asszisztensének automatikus ajánlásaival javítottam. A szöveg tartalma és állításai viszont saját megfogalmazásom alapján kerültek megírásra.		
Képek generálása illusztrációs célból	Claude (Anthropic)	2.1, 2.2, 2.3. ábra (TikZ diagramok)	~20%
Prompt lényegi része	TikZ latex szintaxis segítése. Az ábrák tartalmát magam határoztam meg. A TikZ kód megírásában és vizuális finomításában segített az eszköz, az igényes és egységes megjelenés érdekében. Más generatív képgeneráló eszközt nem használtam.		
Adatvizualizáció, grafikonok generálása adatpontok alapján	Claude (Anthropic)	3. fejezet (3.1–3.6) ábrák matplotlib megjelenítése	~30%
Prompt lényegi része	A 3. fejezet kiértékelési ábrái a saját modellfutásaim eredményei. A matplotlib megjelenítési kód megjelenítésének finomításában segített. Az adatpontok és a kiértékelés tartalma saját eredmény. A weblapon megjelenített adatvizualizációk továbbfejlesztésében is alkalmaztam az eszköz segítségét.		
Prezentáció készítése	—	—	—
Prompt lényegi része	—		
Egyéb (nevezze meg)	—	—	—
Prompt lényegi része	—		
Összesített százalékos érték (a feladat érdemi részére nézve):			~35%
Összesített érték rövid, szöveges indoklása:			

A generatív MI eszközöket elsősorban forráskeresés, kódvázak generálása, TikZ/matplotlib megjelenítés finomítása és szöveg hibáinak javítások során használtam.
A dolgozat érdemi részei, azaz a modell architektúra megtervezése, a pretext-feladatok kiválasztása és kitalálása, a teljes adatfeldolgozási folyamat megtervezése, a kiértékelési módszer kitalálása, a tanítási folyamat valamint a kapott eredmények értelmezése, és a webes felület rendszerének és tesztjeinek tervezése a saját szakmai munkám eredményei.
Az mesterséges intelligencia által generált kimeneteket minden esetben ellenőriztem és átdolgoztam és továbbfejlesztettem.

F.2. A tanítás további veszteséggörbéi



(a) Train MPP **(b)** Val MPP

(c) Train MLM **(d)** Val MLM

(e) Train Velocity **(f)** Val Velocity

(g) Train CZP **(h)** Val CZP

F.2.1. ábra. A pretext feladatok további tanítási és validációs veszteséggörbéi.

F.3. Függelékben elhelyezett kódrészletek

```python
def extract_possession_bounds(pbp_df):
      pbp_sorted = pbp_df.sort(["quarter", "pbp_game_clock"], descending=[False, True])
      possession_bounds = []
      max_quarter = int(pbp_sorted["quarter"].max())
      for q in range(1, max_quarter + 1):
          q_clock_start = 720.0 if q <= 4 else 300.0
          q_pbp = pbp_sorted.filter(pl.col("quarter") == q)
          if q_pbp.height == 0:
              continue
          current_start = q_clock_start
          for i in range(q_pbp.height):
              row   = q_pbp.row(i, named=True)
              event = str(row["event_type"])
              if event in ["Made Shot", "Missed Shot", "Turnover", "Foul", "Violation"]:
                  end_clock = float(row["pbp_game_clock"])
                  if (current_start - end_clock) >= WINDOW_SEC:
                      possession_bounds.append({
                          "quarter":          int(q),
                          "start_clock":      current_start,
                          "event_clock":      end_clock,
                          "end_clock":        max(0.0, end_clock - POST_EVENT_BUFFER),
                          "base_event":       event,
                          "description":      str(row.get("description", "")).upper(),
                          "action_player_id": row.get("action_player_id", None),
                      })
                  current_start = end_clock
      return possession_bounds
```

F.3.1. lista. A labdabirtoklás-határok kinyerése a PBP eseménytípusok alapján.

```python
FPS               = 25
COURT_X_MAX       = 100.0
COURT_Y_MAX       = 53.0
WINDOW_SEC        = 8.0
STEP_SEC          = 4.0
FIXED_FRAMES      = int(WINDOW_SEC * FPS)   # = 200
POST_EVENT_BUFFER = 2.0
INPUT_DIM         = 131
```

F.3.2. lista. A szekvenciagenerálás és feature engineering modul rögzített paraméterei.

```python
    @njit
    def get_zone_one_hot(x_norm, y_norm):
        if x_norm < 0.0 or x_norm > 1.0 or y_norm < 0.0 or y_norm > 1.0:
            return 0.0, 0.0, 0.0, 1.0   # [festek, kozeptavoli, harmas, ervenytelen]

        # Visszaskalazas lab egysegre
        x_ft = x_norm * 100.0
        y_ft = y_norm * 53.0

        # Tavolsag a kozelebbi gyurutol
        dist_left  = np.sqrt((x_ft -  5.25) ** 2 + (y_ft - 25.0) ** 2)
        dist_right = np.sqrt((x_ft - 94.75) ** 2 + (y_ft - 25.0) ** 2)
        dist_hoop  = min(dist_left, dist_right)

        if (x_ft < 19.0 or x_ft > 81.0) and (17.0 < y_ft < 36.0):
            return 1.0, 0.0, 0.0, 0.0   # festek
        elif dist_hoop > 23.75:
            return 0.0, 0.0, 1.0, 0.0   # harmas
        else:
            return 0.0, 1.0, 0.0, 0.0   # kozeptavoli
```

F.3.3. lista. Pályazóna jellemzők meghatározása.

```python
    nh, hd = self.n_heads, self.head_dim         # 4 fej, fejenkent 16d
    q = self.q_proj(h).view(BT, N, nh, hd).transpose(1, 2)  # (BT, 4, N, 16)
    k = self.k_proj(h).view(BT, N, nh, hd).transpose(1, 2)
    v = self.v_proj(h).view(BT, N, nh, hd).transpose(1, 2)

    scale  = 1.0 / math.sqrt(hd)
    scores = torch.matmul(q, k.transpose(-2, -1)) * scale    # (BT, 4, N, N)
```

F.3.4. lista. GATv2 multi-head attention térfelbontása.

```python
    h_agg = torch.cat([
        BALL_WEIGHT   * h_ball,     # 1.5x: a labda kornyezet
        TEAM_A_WEIGHT * h_team_a,   # 1.0x: tamado csapat
        TEAM_B_WEIGHT * h_team_b,   # 0.2x: vedo csapat
    ], dim=-1)
    pooled = F.relu(self.out_proj(h_agg))
```

F.3.5. lista. Súlyozott GNN reprezentáció összeállítása.

```
    i1 = max(1, S // 3)
    i2 = max(i1 + 1, 2 * S // 3)

    valid = (~snippet_mask).float()                              # (B, S)

    def _masked_mean(seg, vmask):
        v     = vmask.unsqueeze(-1)                              # (B, k, 1)
        denom = v.sum(dim=1).clamp(min=1.0)
        return (seg * v).sum(dim=1) / denom                      # (B, D)

    ph_e = _masked_mean(snippets[:, :i1, :],   valid[:, :i1])  # korai fazis atlag
    ph_m = _masked_mean(snippets[:, i1:i2, :], valid[:, i1:i2])# kozepso fazis atlag
    ph_l = _masked_mean(snippets[:, i2:, :],   valid[:, i2:])  # kesoi fazis atlag

    h_concat = torch.cat([cls, ph_e, ph_m, ph_l], dim=-1)      # (B, 4*256 = 1024)
    h_emb    = self.phase_proj(h_concat)                       # (B, 128)
```

F.3.6. lista. Phase Pooling implementációja.

```
    # SimCLR Projekcios fej
    self.simclr_proj = nn.Sequential(
        nn.Linear(BOTTLENECK_DIM, PROJ_HIDDEN), nn.GELU(),   # 128 -> 128
        nn.Linear(PROJ_HIDDEN, PROJ_OUTPUT),                 # 128 -> 64
    )

    # A augmentacios nezetek
    z = F.normalize(self.simclr_proj(h), dim=-1, p=2)        # (B, 64), egyseggomb

    # NT-Xent loss
    z   = torch.cat([z1, z2], dim=0)                  # (2B, 64)
    sim = torch.mm(z, z.t()) / temperature            # (2B, 2B), T = 0.1
    mask_self = torch.eye(2*B, dtype=torch.bool, device=z.device)
    sim       = sim.masked_fill(mask_self, float('-inf'))

    # z1[i]- z2[i]; pozitiv par (index i+B); z2[i] - z1[i] pozitiv par (index i)
    targets = torch.cat([
        torch.arange(B, 2*B, device=z.device),
        torch.arange(0, B,   device=z.device),
    ])
    loss = F.cross_entropy(sim, targets)
```

F.3.7. lista. A SimCLR-fej implementációja.